



UNIVERSITAS INDONESIA

***PENGEMBANGAN JOINT DOCUMENT-LEVEL SENTIMEN-TOPIK
MODEL BERBASIS DEC UNTUK ANALISIS SENTIMEN***

SKRIPSI

**MOHAMMAD RAFFY ZEIDAN
2206051462**

**FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
PROGRAM STUDI STATISTIKA
DEPOK
2026**



UNIVERSITAS INDONESIA

***PENGEMBANGAN JOINT DOCUMENT-LEVEL SENTIMEN-TOPIK
MODEL BERBASIS DEC UNTUK ANALISIS SENTIMEN***

SKRIPSI

**Diajukan sebagai salah satu syarat untuk memperoleh gelar
Sarjana Statistika**

MOHAMMAD RAFFY ZEIDAN

2206051462

FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM

PROGRAM STUDI STATISTIKA

DEPOK

JUNI 2026

HALAMAN PERNYATAAN ORISINALITAS

**Skripsi ini adalah hasil karya saya sendiri,
dan semua sumber baik yang dikutip maupun dirujuk
telah saya nyatakan dengan benar.**

Nama : Mohammad Raffy Zeidan

NPM : 2206051462

Tanda Tangan :

Tanggal : 30 Juni 2026

HALAMAN PERNYATAAN PENGGUNAAN KECERDASAN ARTIFISIAL

Saya menyatakan bahwa dalam penyusunan Tugas Akhir ini, saya menggunakan teknologi Kecerdasan Buatan dengan *platform* Claude untuk tujuan penulisan dan saya bertanggung jawab penuh atas keakuratan, orisinalitas, dan integritas seluruh isi Tugas Akhir ini.

Nama : Mohammad Raffy Zeidan
NPM : 2206051462
Tanda Tangan :

Tanggal : 30 Juni 2026

HALAMAN PENGESAHAN

Skripsi ini diajukan oleh :

Nama : Mohammad Raffy Zeidan

NPM : 2206051462

Program Studi : Statistika

Judul Skripsi : *Pengembangan Joint Document-Level Sentimen-
Topik Model berbasis DEC* untuk Analisis Sentimen

Telah berhasil dipertahankan di hadapan Dewan Penguji dan diterima sebagai bagian persyaratan yang diperlukan untuk memperoleh gelar Sarjana Statistika pada Program Studi Statistika, Fakultas Matematika dan Ilmu Pengetahuan Alam, Universitas Indonesia.

DEWAN PENGUJI

Pembimbing I : Prof. Dr.rer.nat. Hendri Murfi, S.Si., M.Kom. ()

Pembimbing II : Dra. Nora Hariadi, M.Si. ()

Penguji I : Prof. Alhadi Bustamam, S.Si., M.Kom., Ph.D. ()

Penguji II : Kurnia Susvitasari, S.Si., M.Sc., Ph.D. ()

Ditetapkan di : Depok

Tanggal : 07 Juli 2026

KATA PENGANTAR

Puji syukur penulis panjatkan kepada Tuhan Yang Maha Esa atas segala berkat dan rahmat-Nya sehingga penulis dapat menyelesaikan skripsi ini dengan sebaik-baiknya. Selama masa penulisan skripsi ini, penulis telah mendapat banyak dukungan, doa, bantuan, inspirasi, dan motivasi dari berbagai pihak. Untuk itu, penulis ingin mengucapkan terima kasih kepada:

1. Bapak Prof. Dr. rer. nat. Hendri Murfi, S.Si., M.Kom. selaku Dosen Pembimbing I yang telah membimbing penulis dengan penuh kesabaran dan ketulusan. Arahkan dan ilmu yang beliau berikan tidak hanya membantu penulis menyelesaikan penelitian ini, tetapi juga membentuk cara penulis berpikir dalam menghadapi permasalahan secara lebih mendalam.
2. Ibu Dra. Nora Hariadi, M.Si. selaku Dosen Pembimbing II yang dengan teliti telah memberikan arahan terkait tata cara penulisan skripsi. Perhatian beliau terhadap setiap detail penulisan sangat membantu penulis dalam menyusun karya ilmiah yang sistematis dan sesuai kaidah akademik.
3. Bapak Prof. Alhadi B., S.Si., M.Kom., Ph.D. selaku Dosen Penguji I yang telah memberikan kritik dan saran yang tajam dan membangun, sehingga mendorong penulis untuk berpikir lebih kritis dalam menyempurnakan skripsi ini.
4. Ibu Kurnia Susvitasari, S.Si., M.Sc., Ph.D. selaku Dosen Penguji II yang telah memberikan masukan dan perspektif yang memperkaya pembahasan dalam skripsi ini.
5. Ibu Prof. Dr. Titin Siswantining, D.E.A. selaku Dosen Pembimbing Akademis yang senantiasa memberikan arahan, nasihat, dan dukungan selama penulis menjalani masa perkuliahan.
6. Orang tua penulis, Bapak Timu dan Ibu Yuyun Dharmayanti, yang telah memberikan segalanya. Doa, dukungan, dan kesediaan untuk selalu ada baik di saat semangat maupun di saat lelah menjadi kekuatan terbesar yang menopang penulis hingga sampai di titik ini.

7. Teman-teman Group DKM (Bagja College) yang telah menjadi bagian dari perjalanan penulis sejak masa SMA dan kebersamaannya terus berlanjut hingga saat ini.
8. Farhan, Dafa, Nicholas, dan Aditya, teman sepersementeran yang hadir dari hari-hari pertama kuliah hingga akhir. Kehadiran mereka membuat perjalanan yang panjang ini terasa lebih ringan.
9. RISTEK Fasilkom UI yang melalui kompetisi *data science*-nya telah membuka jalan bagi penulis menuju dunia yang kini menjadi bagian besar dari kehidupan akademik dan karier penulis.
10. PT Profina Edvisor yang telah memberikan kepercayaan dan ruang bagi penulis untuk tumbuh sebagai seorang *data scientist*, serta dukungan yang tidak pernah surut selama proses penyelesaian skripsi ini.
11. Seluruh pihak yang tidak dapat disebutkan satu per satu, yang dengan caranya masing-masing telah turut mengambil peran dalam perjalanan panjang ini.

Penulis menyadari bahwa skripsi ini masih memiliki keterbatasan dan kekurangan, baik dari segi penulisan maupun substansi keilmuan. Oleh karena itu, kritik dan saran yang membangun sangat penulis harapkan sebagai bahan perbaikan ke depan. Penulis berharap skripsi ini dapat bermanfaat bagi pembaca dan menjadi bagian dari kontribusi nyata dalam pengembangan ilmu pengetahuan, khususnya di bidang yang dikaji dalam penelitian ini.

Depok, 30 Juni 2026

Mohammad Raffy Zeidan

HALAMAN PERNYATAAN PERSETUJUAN PUBLIKASI TUGAS AKHIR UNTUK KEPENTINGAN AKADEMIS

Sebagai sivitas akademik Universitas Indonesia, saya yang bertanda tangan di bawah ini:

Nama : Mohammad Raffy Zeidan
NPM : 2206051462
Program Studi : Statistika
Departemen : Matematika
Fakultas : Matematika dan Ilmu Pengetahuan Alam
Jenis Karya : Skripsi

demikian pengembangan ilmu pengetahuan, menyetujui untuk memberikan kepada Universitas Indonesia **Hak Bebas Royalti Noneksklusif** (*Non-exclusive Royalty Free Right*) atas karya ilmiah saya yang berjudul:

Pengembangan Joint Document-Level Sentimen-Topik Model berbasis DEC untuk Analisis Sentimen

berserta perangkat yang ada (jika diperlukan). Dengan Hak Bebas Royalti Noneksklusif ini, Universitas Indonesia berhak menyimpan, mengalihmedia/formatkan, mengelola dalam bentuk pangkalan data (*data base*), merawat, dan memublikasikan tugas akhir saya selama tetap mencantumkan nama saya sebagai penulis/pencipta dan sebagai pemilik Hak Cipta.

Demikian pernyataan ini saya buat dengan sebenarnya.

Dibuat di : Depok
Pada tanggal : 30 Juni 2026

Yang menyatakan

(Mohammad Raffy Zeidan)

ABSTRAK

Nama: Mohammad Raffy Zeidan

Program Studi: Statistika

Judul: *Pengembangan Joint Document-Level Sentimen-Topik Model berbasis DEC untuk Analisis Sentimen*

Pertumbuhan data teks dari platform digital, seperti ulasan produk dan media sosial, mendorong kebutuhan akan metode otomatis yang mampu menganalisis sentimen dan mengidentifikasi topik secara bersamaan. Pendekatan analisis sentimen konvensional umumnya mengasumsikan satu sentimen untuk setiap dokumen sehingga belum mampu merepresentasikan ulasan yang membahas lebih dari satu topik. Penelitian ini mengembangkan model *Joint Document-level Sentiment-Topic* berbasis *Deep Embedded Clustering* (JDST-DEC) yang mengintegrasikan *Deep Embedded Clustering* (DEC) ke dalam kerangka *Multi-Task Learning* (MTL) dengan encoder BERT yang dibekukan. Pelatihan dilakukan dalam dua tahap, yaitu pra-pelatihan *autoencoder* untuk membentuk representasi laten dari *embedding* [CLS] BERT, kemudian dilanjutkan dengan optimasi gabungan fungsi *loss* sentimen berbasis *cross-entropy* dan *loss* klusterisasi berbasis *KL divergence*. Evaluasi dilakukan pada dataset IMDb dan Yelp dengan variasi jumlah topik $K = 30, 50, \text{ dan } 80$. Pada tugas klasifikasi sentimen, JDST-DEC memperoleh *F1-score* terbaik sebesar 0,8986 pada dataset IMDb dan 0,9020 pada dataset Yelp pada konfigurasi $K = 50$, lebih tinggi dibandingkan NN-BERT yang memperoleh 0,8612 dan 0,8715. Pada tugas pemodelan topik, BERTopic menghasilkan nilai *Topic Coherence* dan *Topic Diversity* yang lebih tinggi dibandingkan JDST-DEC. Meskipun demikian, JDST-DEC tetap mampu menghasilkan kualitas topik yang memadai sambil melakukan klasifikasi sentimen dan pemodelan topik secara simultan dalam satu kerangka *Multi-Task Learning*. Secara keseluruhan, hasil penelitian menunjukkan bahwa pendekatan yang diusulkan mampu meningkatkan performa klasifikasi sentimen dibandingkan pendekatan *single-task*, meskipun kualitas pemodelan topik masih dapat ditingkatkan.

Kata kunci: *Deep Embedded Clustering, Joint Document-level Sentiment-Topic Modeling, Multi-Task Learning, BERT.*

EXTENDED ABSTRACT

EXTENDED ABSTRACT

Undergraduate Thesis

Development of a Joint Document-Level Sentiment–Topic Model Based on DEC for Sentiment Analysis

Student Name : Mohammad Raffy Zeidan

Student ID Number (NPM) : 2206051462

Study Program : Statistika

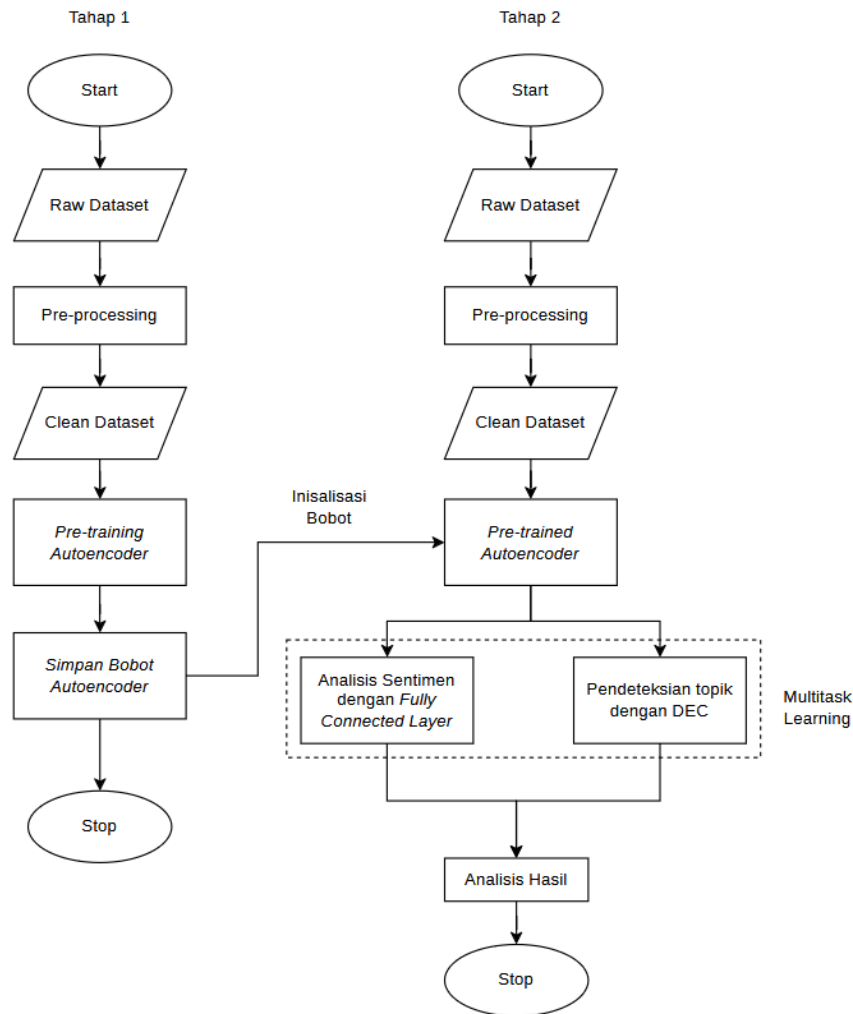
Every day, review platforms and social media generate enormous volumes of user-generated text. Automatically extracting sentiment and thematic content from this data has become a central challenge in Natural Language Processing (NLP). Conventional document-level sentiment analysis reduces each document to a single polarity label, which overlooks the fact that a review often discusses multiple aspects with differing opinions. Yet existing joint sentiment-topic approaches either require aspect-level supervision or rely on pipeline architectures that do not allow the two objectives to reinforce each other during training. This limitation motivates end-to-end joint approaches that simultaneously model sentiment and topics, offering richer output without requiring aspect-level annotations.

To address this, the present study proposes a *Joint Document-Level Sentiment-Topic* model based on *Deep Embedded Clustering* (**JDST-DEC**) within a *Multi-Task Learning* (MTL) framework. The model operates in two stages. In the first stage, the 768-dimensional [CLS] representation extracted by a frozen BERT encoder, which reduces computational cost and preserves stable pre-trained representations, is fed into a symmetric fully connected autoencoder with architecture $768 \rightarrow 2022 \rightarrow 2022 \rightarrow 256$, trained using mean squared error (MSE) to learn a compact 256-dimensional latent representation. In the second stage, this latent space is jointly optimised for sentiment classification via cross-entropy loss ($\mathcal{L}_{\text{sent}}$) and topic clustering via DEC with KL divergence loss ($\mathcal{L}_{\text{clust}}$), under the joint objective:

$$\mathcal{L} = \eta \mathcal{L}_{\text{sent}} + \gamma \mathcal{L}_{\text{clust}}, \quad \eta = \gamma = 0.5,$$

where equal weighting balances the sentiment classification and clustering objectives without favouring either. Cluster centroids are initialised using k -means on the

pre-trained latent space. The overall research workflow is summarised in Figure 1.



Gambar 1: Schematic diagram of the research framework.

Experiments were conducted on two benchmark datasets: IMDB Movie Reviews (Maas *et al.*, 2011b) (50,000 reviews) and the Yelp Review Dataset (X. Zhang, J. Zhao, dan LeCun, 2015b) (30,000 reviews), both with binary sentiment labels. Three topic cardinalities $K \in \{30, 50, 80\}$ were evaluated. **JDST-DEC** was compared against NN-BERT as a single-task sentiment baseline, MTL-GUI (Gui *et al.*, 2022), BERTopic-HDBSCAN, BERTopic-KMeans, and MambaQuin. All baselines were re-evaluated under identical dataset splits and evaluation protocols to ensure a fair comparison. Sentiment performance was measured using accuracy and F1-score; topic quality was assessed using *Topic Coherence* based on NPMI (TC-NPMI) and *Topic Diversity* (TD).

Pre-training results show that reconstruction loss drops sharply in the early epochs and stabilises after epoch 20, reaching final training losses of approximately

1.41×10^{-4} on IMDB and 1.55×10^{-4} on Yelp. Validation loss remains close to training loss throughout, indicating no overfitting. Selected pre-training loss values are shown in Table 1.

Tabel 1: Selected pre-training *loss* values on the IMDB and Yelp datasets.

Epoch	IMDB		Yelp	
	<i>Train</i>	<i>Val.</i>	<i>Train</i>	<i>Val.</i>
1	2.370×10^{-3}	2.544×10^{-3}	3.790×10^{-3}	4.062×10^{-3}
10	3.780×10^{-4}	4.120×10^{-4}	2.780×10^{-4}	3.040×10^{-4}
20	1.370×10^{-4}	1.550×10^{-4}	1.670×10^{-4}	1.860×10^{-4}
30	1.410×10^{-4}	1.590×10^{-4}	1.550×10^{-4}	1.730×10^{-4}

For sentiment classification, **JDST-DEC** consistently outperforms NN-BERT and MTL-GUI across all values of K on both datasets (Table 2). On IMDB, **JDST-DEC** achieves its best F1-score of 0.8986 at $K = 50$, compared with 0.8612 for NN-BERT and 0.8317 for MTL-GUI, corresponding to improvements of approximately 3.7 and 6.7 percentage points respectively. On Yelp, **JDST-DEC** reaches an F1-score of 0.9020 at $K = 50$, compared with 0.8715 and 0.8401, representing gains of approximately 3.1 and 6.2 percentage points. Performance remains stable across all values of K , indicating that sentiment classification quality is relatively insensitive to topic cardinality. MambaQuin records the highest scores on IMDB across all K settings and is directly comparable on that dataset. Its Yelp results are not directly comparable due to a different data sampling procedure (shaded red in Table 2) and are therefore excluded from column-wise comparison on Yelp. Among the comparable methods, **JDST-DEC** achieves the highest sentiment scores on Yelp and the second-highest on IMDB.

Tabel 2: Comparison of sentiment classification results on IMDB and Yelp.

Method	IMDB						Yelp					
	Acc		F1				Acc		F1			
NN-BERT	0.8581		0.8612				0.8664		0.8715			
	$K = 30$		$K = 50$		$K = 80$		$K = 30$		$K = 50$		$K = 80$	
	Acc	F1	Acc	F1	Acc	F1	Acc	F1	Acc	F1	Acc	F1
JDST-DEC	0.8957	0.8963	0.8967	0.8986	0.8941	0.8930	0.8965	0.8944	0.8983	0.9020	0.8935	0.8953
MTL-GUI	0.8327	0.8326	0.8317	0.8317	0.8334	0.8334	0.8385	0.8385	0.8380	0.8380	0.8402	0.8401
MambaQuin	0.9085	0.9085	0.9083	0.9083	0.9086	0.9086	0.9433	0.9433	0.9416	0.9416	0.9428	0.9428

■ highest comparable value in column; ■ MambaQuin (Yelp) — not directly comparable due to different data sampling;

All baselines evaluated under identical dataset splits and protocols.

For topic modelling, BERTopic achieves the highest TD values across most configurations (Table 3), with BERTopic-KMeans reaching 0.9567 on IMDB at $K = 30$. For TC-NPMI, MambaQuin leads on IMDB at $K = 30$ (0.1589), while BERTopic-HDBSCAN and BERTopic-KMeans lead at $K = 50$ and $K = 80$ respectively. On Yelp, BERTopic variants achieve the highest comparable TC-NPMI values; MambaQuin’s Yelp results are excluded from column-wise comparison due to different data sampling. **JDST-DEC** produces lower TC-NPMI and TD values than BERTopic, as joint optimisation prioritises sentiment-discriminative representations over purely coherent topic structures. Among the joint models, **JDST-DEC** consistently outperforms MTL-GUI on both metrics and both datasets across every value of K .

Tabel 3: Comparison of topic modelling results on IMDB and Yelp.

Method	IMDB						Yelp					
	$K = 30$		$K = 50$		$K = 80$		$K = 30$		$K = 50$		$K = 80$	
	TC-NPMI	TD	TC-NPMI	TD	TC-NPMI	TD	TC-NPMI	TD	TC-NPMI	TD	TC-NPMI	TD
JDST-DEC	0.0803	0.2256	0.0551	0.2100	0.0675	0.2875	0.0788	0.3267	0.0369	0.2280	0.0563	0.3150
BERTopic-HDBSCAN	0.1714	0.7966	0.1823	0.7680	0.1751	0.7625	0.1092	0.7233	0.1666	0.7041	0.1646	0.6671
BERTopic-KMeans	0.1566	0.9567	0.1596	0.9000	0.1874	0.8413	0.1077	0.7800	0.1120	0.6740	0.1465	0.6438
MTL-GUI	0.0200	0.5833	0.0108	0.5800	0.0060	0.4913	0.1578	0.6600	0.1086	0.6720	0.1123	0.5000
MambaQuin	0.1589	0.2400	0.1561	0.2040	0.1606	0.1575	0.2557	0.2533	0.2727	0.2060	0.2699	0.1588

■ highest comparable value in column; ■ MambaQuin (Yelp) — not directly comparable due to different data sampling;

All baselines evaluated under identical dataset splits and protocols.

Analysis of the joint training dynamics shows that $\mathcal{L}_{\text{sent}}$ decreases consistently from early iterations, while $\mathcal{L}_{\text{clust}}$ tends to increase throughout training. This reflects the DEC mechanism, in which the target distribution is iteratively updated based on current cluster assignments, keeping the KL divergence active and allowing the cluster structure to continue evolving during joint optimisation. The opposing trends of the two losses suggest that the model progressively trades raw reconstruction accuracy for a latent geometry that is simultaneously sentiment-discriminative and cluster-coherent.

Interpretability analysis using occlusion-based token attribution on the Yelp dataset ($K = 30$) shows that tokens such as “*super*” and “*average*” carry attributions consistent with their expected polarity, while certain tokens exhibit signs inconsistent with their lexical polarity, indicating that the model captures context-sensitive interactions rather than relying on surface-form cues alone.

Limitations. The current study has several limitations. The experiments are limited to binary sentiment classification on English-language datasets; performance on

multi-class or multilingual settings remains to be established. The use of a frozen BERT encoder, while computationally efficient, may limit the model's ability to adapt its representations to domain-specific sentiment signals. The autoencoder architecture and DEC objective are fixed throughout training, and it is unclear whether deeper or alternative architectures would improve topic quality without sacrificing sentiment accuracy. Finally, evaluation relies on automated topic metrics (TC-NPMI, TD), which do not fully capture human judgements of topic usefulness.

In future work, these limitations motivate exploring encoder fine-tuning strategies that balance adaptation with stability, deeper autoencoder architectures and alternative clustering objectives to improve topic diversity, and extensions to multilingual and multi-class sentiment settings. Incorporating aspect-level evaluation would further characterise the granularity of the learned topic representations.

In conclusion, **JDST-DEC** demonstrates an effective mechanism for joint sentiment-topic learning: integrating DEC into an MTL framework with a frozen BERT encoder improves sentiment classification over both single-task and existing joint-model baselines, and produces interpretable topic clusters, all without requiring aspect-level annotations. By showing that a single latent space can serve both objectives simultaneously, this work opens a practical path toward richer, annotation-efficient document understanding in large-scale review and social-media settings.

Keywords: *Deep Embedded Clustering, Joint Document-Level Sentiment-Topic Modeling, Multi-Task Learning, Topic Modeling, BERT.*

DAFTAR ISI

HALAMAN JUDUL	i
LEMBAR PERSETUJUAN	ii
LEMBAR PERNYATAAN ORISINALITAS	ii
LEMBAR PERNYATAAN PENGGUNAAN KECERDASAN ARTI-FISIAL	iii
LEMBAR PENGESAHAN	iv
KATA PENGANTAR	v
LEMBAR PERSETUJUAN PUBLIKASI ILMIAH	vi
ABSTRAK	viii
Extended Abstract	ix
DAFTAR ISI	xiv
DAFTAR GAMBAR	xvii
DAFTAR TABEL	xix
1 PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Rumusan Permasalahan	5
1.3 Tujuan Penelitian	6
1.4 Manfaat Penelitian	6
1.5 Ruang Lingkup Penelitian	7
2 LANDASAN TEORI	8
2.1 Analisis Sentimen	8
2.2 Pendeteksian Topik	9
2.3 Neural Network	9
2.3.1 Fungsi Aktivasi	11

2.3.1.1	<i>Softmax</i>	11
2.3.1.2	<i>Rectified Linear Unit</i> (ReLU)	12
2.3.1.3	<i>Gaussian Error Linear Unit</i> (GELU)	12
2.3.2	Fungsi <i>Loss</i>	13
2.3.3	Backpropagation	13
2.3.4	Teknik Optimasi	14
2.3.4.1	<i>Stochastic Gradient Descent</i> (SGD)	14
2.3.4.2	<i>Adaptive Moment Estimation</i> (Adam)	15
2.3.5	Regularisasi	16
2.3.6	Metrik Evaluasi	16
2.4	Autoencoder	17
2.4.1	Encoder dan Decoder	18
2.4.2	Fungsi Objektif	19
2.5	Bidirectional Encoder Representations from Transformers (BERT) .	19
2.5.1	Arsitektur BERT	19
2.5.2	Mekanisme <i>Self-Attention</i>	20
2.5.3	Representasi Input BERT	21
2.6	Multi-Task Learning	22
2.6.1	Formulasi MTL	22
2.6.2	Hard Parameter Sharing	23
2.6.3	Soft Parameter Sharing	24
2.7	Clustering	25
2.7.1	K-Means Clustering	26
2.8	Interpretabilitas Model <i>Deep Learning</i>	27
2.8.1	<i>Occlusion-based Token Attribution</i>	28
2.8.2	<i>Integrated Gradients</i>	29

3 METODE MODEL *JOINT DOCUMENT-LEVEL SENTIMENT-TOPIC* (JDST) BERBASIS *DEEP EMBEDDED CLUSTERING* UNTUK ANALISIS SENTIMEN

30

3.1	<i>Bidirectional Encoder Representations from Transformers</i> (BERT) Sebagai Model Representasi Teks	30
3.2	Alur Pelatihan Model <i>Multitask</i>	33
3.2.1	Implementasi <i>Autoencoder</i>	33
3.2.2	<i>Deep Embedded Clustering</i> (DEC)	34
3.2.3	Pelatihan <i>Multitask</i> : JDST-DEC	36
3.2.3.1	<i>Clustering Head</i>	36
3.2.3.2	Analisis Sentimen Menggunakan <i>Sentiment Head</i>	37

3.2.3.3	Pelatihan <i>Joint</i> dengan <i>Multitask Loss</i>	38
3.2.3.4	Algoritma JDST-DEC	39
3.3	Interpretasi Model dengan Analisis Atribusi Token	40
3.4	Evaluasi Topik	41
3.4.1	<i>Topic Coherence</i> Berbasis NPMI	41
3.4.2	<i>Topic Diversity</i>	42
4	HASIL DAN SIMULASI DAN PEMBAHASAN	44
4.1	Spesifikasi Mesin dan Perangkat Lunak	44
4.2	Tahapan Umum Simulasi	44
4.3	Data	46
4.3.1	Pengumpulan Data	46
4.3.2	Prapengolahan Data	47
4.3.3	Representasi Teks	49
4.3.4	Pembagian Data	49
4.4	Implementasi Model	50
4.4.1	Konfigurasi JDST-DEC	50
4.4.2	Konfigurasi <i>Joint Training</i>	51
4.4.3	Konfigurasi Model Pembanding	52
4.4.3.1	NN-BERT	52
4.4.3.2	BERTopic	53
4.5	Analisis Kinerja Model	54
4.5.1	Analisis Kinerja Pra-pelatihan	54
4.5.2	Analisis Kinerja <i>Joint Training</i>	56
4.5.2.1	Klasifikasi Sentimen	56
4.5.2.2	Pemodelan Topik	57
4.5.3	Analisis Multiloss	58
4.5.3.1	Dinamika pada Data Eksperimen	58
4.5.3.2	Eksperimen Dataset Kecil	59
4.6	Analisis Interpretabilitas Model	59
5	Penutup	62
5.1	Kesimpulan	62
5.2	Saran	63
	LAMPIRAN	1

DAFTAR GAMBAR

Gambar 1	Schematic diagram of the research framework.	x
Gambar 1	Ilustrasi <i>neural network</i> secara biologis (Aggarwal, 2018) .	10
Gambar 2	<i>Artificial Neural Network</i> dengan representasi bobot sinapsis (Aggarwal, 2018)	10
Gambar 3	Arsitektur <i>feed-forward neural network</i> (Aggarwal, 2018) .	11
Gambar 4	Grafik fungsi aktivasi: <i>Softmax</i> , ReLU, dan GELU	13
Gambar 5	Ilustrasi graf komputasi pada proses <i>backpropagation</i> (Aggarwal, 2018)	14
Gambar 6	Ilustrasi <i>bias-variance tradeoff</i> terhadap kompleksitas model (Aggarwal, 2018, telah diolah kembali)	16
Gambar 7	Arsitektur <i>autoencoder</i> menunjukkan proses <i>encoding</i> dan <i>decoding</i> (Aggarwal, 2018)	18
Gambar 8	Arsitektur BERT dengan beberapa lapisan <i>Transformer Encoder</i> (Devlin <i>et al.</i> , 2018)	20
Gambar 9	Representasi <i>input</i> BERT yang terdiri dari <i>token</i> , <i>segment</i> , dan <i>position embedding</i> (Devlin <i>et al.</i> , 2018)	21
Gambar 10	Ilustrasi arsitektur <i>hard parameter sharing</i> dalam <i>Multi-Task Learning</i> (Ruder, 2017, telah diolah kembali)	24
Gambar 11	Ilustrasi arsitektur <i>soft parameter sharing</i> (Ruder, 2017, telah diolah kembali)	25
Gambar 1	Alur metode analisis sentimen dan pendeteksian topik	30
Gambar 2	Representasi teks menggunakan BERT ((Devlin <i>et al.</i> , 2018); telah diolah kembali).	31
Gambar 3	Ilustrasi <i>autoencoder</i> lengkap (kiri) dan komponen <i>encoder</i> setelah <i>decoder</i> dilepas (kanan).	34
Gambar 4	Arsitektur <i>clustering head</i> berbasis DEC.	37
Gambar 5	Arsitektur <i>sentiment head</i>	37
Gambar 1	Diagram alur simulasi penelitian.	45
Gambar 2	Dinamika $\mathcal{L}_{\text{clust}}$ dan $\mathcal{L}_{\text{sent}}$ (ternormalisasi <i>min-max</i>) selama <i>joint training</i> pada dataset IMDB ($K = 50$) dan Yelp ($K = 30$).	58

Gambar 3	Skenario dataset kecil: $\mathcal{L}_{\text{sent}}$ mencapai kondisi stabil lebih dahulu, kemudian $\mathcal{L}_{\text{clust}}$ mulai menurun dan kedua komponen menunjukkan pola yang lebih seimbang.	59
Gambar 4	Ringkasan hasil atribusi token pada 30 klaster. Panel (a) menunjukkan nilai atribusi tertinggi pada setiap klaster, sedangkan panel (b)–(g) menampilkan token dengan kontribusi terbesar pada beberapa klaster terpilih berdasarkan skor atribusi rata-rata.	60

DAFTAR TABEL

Tabel 1	Selected pre-training <i>loss</i> values on the IMDB and Yelp datasets.	xi
Tabel 2	Comparison of sentiment classification results on IMDB and Yelp.	xi
Tabel 3	Comparison of topic modelling results on IMDB and Yelp.	xii
Tabel 1	Algoritma K-Means <i>Clustering</i>	27
Tabel 1	Algoritma <i>Deep Embedded Clustering</i> (DEC)	36
Tabel 2	Algoritma JDST-DEC (<i>Joint Document-level Sentiment-Topic</i>)	39
Tabel 1	Spesifikasi mesin dan perangkat lunak	44
Tabel 2	Cuplikan data ulasan IMDB (Maas <i>et al.</i> , 2011b).	47
Tabel 3	Cuplikan data ulasan Yelp (X. Zhang, J. Zhao, dan LeCun, 2015b).	47
Tabel 4	Contoh hasil prapengolahan pada dataset IMDB dan Yelp.	48
Tabel 5	Contoh proses tokenisasi menggunakan WordPiece BERT.	49
Tabel 6	Contoh representasi vektor [CLS] hasil ekstraksi BERT.	49
Tabel 7	Konfigurasi pra-pelatihan <i>autoencoder</i>	50
Tabel 8	Konfigurasi komponen DEC	51
Tabel 9	Konfigurasi <i>joint training</i> JDST-DEC	52
Tabel 10	Konfigurasi model NN-BERT	53
Tabel 11	Konfigurasi model BERTopic	54
Tabel 12	Perkembangan <i>loss</i> pra-pelatihan pada dataset IMDB	55
Tabel 13	Perkembangan <i>loss</i> pra-pelatihan pada dataset Yelp	55
Tabel 14	Perbandingan hasil <i>Sentiment Classification</i>	56
Tabel 15	Perbandingan hasil <i>Topic Modelling</i>	57

BAB 1

PENDAHULUAN

1.1 Latar Belakang

Dunia kontemporer dibanjiri oleh arus deras informasi tekstual yang dihasilkan setiap saat, bersumber dari platform media sosial, ulasan daring, artikel berita, repositori ilmiah, dan banyak lagi. Kumpulan data tekstual masif ini, yang sebagian besar tidak terstruktur, mengandung potensi pengetahuan yang signifikan mengenai sentimen masyarakat, dinamika pasar, hingga tren diskursus publik. Akan tetapi, volume dan keragaman data tersebut menjadikan analisis manual suatu hal yang tidak praktis dan efisien. Menjawab tantangan ini, disiplin *Natural Language Processing* (NLP), sebagai bagian dari kecerdasan artifisial yang menjembatani komunikasi antara mesin dan bahasa alami manusia, hadir menawarkan metodologi komputasional untuk mengolah dan memahami data linguistik dalam skala besar.

Dalam spektrum kapabilitas NLP, dua teknik yang memiliki relevansi tinggi untuk mengekstrak makna dari himpunan data tekstual adalah analisis sentimen (*Sentiment Analysis*) dan pemodelan topik (*Topic Modeling*) (Blei, Ng, dan Jordan, 2003). Analisis sentimen secara khusus dirancang untuk menyelidiki dan menentukan muatan subjektif atau nada emosional yang terkandung dalam suatu teks, lalu menggolongkannya sebagai ekspresi positif, negatif, ataupun netral. Pengaplikasian teknik ini sangat berharga untuk melacak citra merek, menganalisis tanggapan konsumen, memetakan opini publik terkait isu tertentu, serta mengidentifikasi ujaran problematik. Kemampuan mengukur sentimen secara kolektif memberikan landasan bagi entitas bisnis maupun riset untuk memahami persepsi dan mengambil langkah strategis.

Di sisi lain, pemodelan topik menangani tugas komplementer, yakni mengungkap tema-tema atau pokok bahasan esensial yang tersembunyi (*latent thematic structures*) di dalam koleksi dokumen berjumlah besar, atau sering disebut *corpora* (Blei, Ng, dan Jordan, 2003). Berbeda dengan fokus pada nuansa afektif per teks, tujuan utama pemodelan topik adalah menyajikan intisari tematik dari keseluruhan data. Kegunaannya terbentang luas, mulai dari fasilitasi eksplorasi dan pengorganisasian arsip digital, identifikasi pola baru dalam korpus literatur atau berita, hingga penyempurnaan sistem rekomendasi konten dan pemahaman struktur narasi dalam

skala makro. Dengan mengabstraksikan topik-topik dominan, metode ini membantu mereduksi kompleksitas informasi dan menyajikan gambaran umum konten. Walaupun memiliki fokus yang berbeda, kedua pendekatan ini berpotensi menghasilkan pemahaman yang lebih komprehensif jika diintegrasikan, menyingkap *apa* subjek pembicaraan sekaligus *bagaimana* pandangan atau perasaan terhadap subjek tersebut.

Pendekatan tradisional dalam pemodelan topik, seperti *Latent Dirichlet Allocation* (LDA), umumnya berbasis probabilistik dan mengandalkan metode faktorisasi matriks untuk mengidentifikasi distribusi kata dalam topik (Blei, Ng, dan Jordan, 2003; Blei, 2012). Meskipun metode ini telah terbukti efektif dalam banyak kasus, keterbatasannya dalam menangkap hubungan semantik yang kompleks dalam teks membuatnya kurang optimal untuk tugas yang lebih kompleks, seperti klasifikasi sentimen yang dilakukan secara simultan dengan deteksi topik. Salah satu kelemahan utama pendekatan probabilistik seperti LDA adalah skalabilitasnya yang terbatas, terutama saat diaplikasikan pada *dataset* berukuran besar dengan dimensi fitur tinggi (Boyd-Graber, Hu, dan Mimno, 2017). Selain itu, model berbasis probabilistik sering kali memerlukan waktu komputasi yang signifikan dan sulit diadaptasi untuk menangani data teks modern yang bersifat dinamis dan tidak terstruktur (McAuliffe dan Blei, 2008).

Dalam klasifikasi sentimen, pendekatan berbasis pembelajaran mendalam (*deep learning*) telah menunjukkan keunggulan signifikan dibandingkan metode tradisional. Salah satu model yang paling efektif adalah *Bidirectional Encoder Representations from Transformers* (BERT), yang mampu memahami konteks kata dalam teks dengan mempertimbangkan hubungan antar kata di kedua arah (*bidirectional*) (Devlin *et al.*, 2018). BERT telah banyak digunakan dalam tugas-tugas NLP dan terbukti unggul dalam klasifikasi sentimen, menghasilkan akurasi yang lebih tinggi dibandingkan pendekatan berbasis fitur atau metode statistik konvensional (Sun *et al.*, 2019).

Seiring dengan berkembangnya model *deep learning*, pendekatan berbasis jaringan saraf telah terbukti lebih unggul dalam menangani data teks yang tidak terstruktur dibandingkan metode berbasis probabilistik seperti LDA (Mikolov, K. Chen *et al.*, 2013). Model *neural network* dapat mengekstrak representasi semantik yang lebih kaya dan menangkap hubungan kontekstual antar kata dalam teks dengan lebih baik (Yinhan Zhang, H. Zhao, dan M. Li, 2020). Selain itu, pendekatan *neural network* dapat mengatasi keterbatasan skalabilitas pada metode probabilistik

karena dapat memanfaatkan arsitektur yang lebih efisien seperti *transformers* dan teknik *embedding* untuk mengurangi dimensi data (Vaswani *et al.*, 2017).

Salah satu pendekatan berbasis *neural network* yang semakin populer dalam pemodelan topik adalah *BERTopic*. Berbeda dengan LDA yang hanya mengandalkan distribusi kata untuk menentukan topik, *BERTopic* memanfaatkan representasi teks dari model *transformer* seperti BERT untuk menangkap hubungan semantik yang lebih dalam antar kata (Grootendorst, 2022). Metode ini menggabungkan *embedding* berbasis *transformer* dengan teknik klusterisasi seperti *UMAP* dan *HDBSCAN* untuk mengidentifikasi topik secara lebih akurat dan stabil, bahkan dalam *dataset* berukuran besar atau dengan struktur yang kompleks. Keunggulan utama *BERTopic* dibandingkan LDA adalah kemampuannya dalam menangani variasi bahasa yang lebih luas, memahami sinonim dan kontekstualisasi kata, serta tidak bergantung pada asumsi distribusi kata seperti pada metode probabilistik (Grootendorst, 2022).

Namun demikian, sebagian besar metode pemodelan topik modern seperti *BERTopic* masih berfokus pada pemisahan topik semata tanpa mempertimbangkan aspek lain seperti sentimen atau struktur laten yang lebih dalam secara eksplisit. Selain itu, integrasi antara pemodelan topik dan klasifikasi sentimen masih sering dilakukan secara terpisah, sehingga informasi semantik yang berharga dalam teks belum sepenuhnya dimanfaatkan secara terpadu. Pemisahan ini mengakibatkan redundansi komputasi dan hilangnya peluang untuk memanfaatkan sinergi antara kedua tugas dalam satu arsitektur model yang kohesif.

Dalam konteks pembelajaran representasi untuk data teks berdimensi tinggi, *Autoencoder* telah menunjukkan kemampuannya sebagai teknik pembelajaran tanpa supervisi (*unsupervised learning*) yang efektif untuk reduksi dimensi dan ekstraksi fitur (Hinton dan Salakhutdinov, 2006). *Autoencoder* bekerja dengan cara mempelajari representasi laten (*latent representation*) yang lebih kompak dari data masukan melalui proses *encoding* dan *decoding*. Arsitektur ini memaksa model untuk mengekstrak informasi paling esensial dari data dengan mengompresi masukan ke dalam ruang dimensi yang lebih rendah, kemudian merekonstruksi kembali data asli dari representasi tersebut. Proses ini memungkinkan model untuk menangkap struktur intrinsik dan pola-pola penting dalam data teks, sekaligus mengurangi *noise* dan redundansi fitur.

Berbeda dengan teknik reduksi dimensi tradisional seperti *Principal Component Analysis* (PCA) yang bersifat linear, *Autoencoder* mampu menangkap hubungan non-linear yang kompleks dalam data (Hinton dan Salakhutdinov, 2006). Hal ini

sangat penting dalam pemrosesan teks, di mana hubungan antar kata dan dokumen seringkali bersifat non-linear dan kontekstual. *Autoencoder* juga memiliki fleksibilitas arsitektur yang memungkinkan penyesuaian kedalaman jaringan sesuai dengan kompleksitas data dan tugas yang dihadapi. Dalam konteks NLP, penggunaan *Autoencoder* telah terbukti efektif untuk berbagai aplikasi, termasuk pembelajaran representasi dokumen, deteksi anomali dalam teks, dan ekstraksi fitur untuk tugas-tugas *downstream* seperti klasifikasi dan klusterisasi (Vincent *et al.*, 2010).

Untuk menjawab keterbatasan integrasi antara pemodelan topik dan klasifikasi sentimen, penelitian ini bertujuan untuk mengembangkan pendekatan pemodelan topik yang lebih terintegrasi melalui kombinasi metode representasi laten berbasis *Autoencoder* dan teknik *Deep Embedded Clustering* (DEC). Dengan memanfaatkan *Autoencoder* sebagai komponen *encoder* untuk menghasilkan representasi teks yang lebih efisien dan bermakna, serta DEC untuk melakukan klusterisasi yang mempertahankan struktur semantik dalam ruang laten, model diharapkan mampu mengidentifikasi topik-topik secara lebih akurat dan representatif. *Autoencoder* dalam penelitian ini berfungsi sebagai fondasi untuk pembelajaran representasi yang *robust*, yang kemudian dioptimalkan lebih lanjut melalui proses klusterisasi DEC untuk menghasilkan pembagian topik yang lebih koheren dan terpisah dengan baik.

Lebih lanjut, penelitian ini mengusulkan pengembangan arsitektur *multi-task learning* (MTL) yang memungkinkan integrasi klasifikasi sentimen dan pemodelan topik secara bersamaan dalam satu kerangka kerja. Pendekatan MTL memiliki keunggulan dalam memanfaatkan hubungan implisit antara tugas-tugas terkait untuk meningkatkan generalisasi model (Caruana, 1997). Dalam konteks penelitian ini, klasifikasi sentimen dan pemodelan topik memiliki keterkaitan semantik yang kuat; sentimen suatu teks seringkali berkorelasi dengan topik yang dibahas, dan pemahaman topik dapat membantu dalam identifikasi sentimen yang lebih akurat. Dengan berbagi representasi laten yang sama melalui *Autoencoder*, kedua tugas dapat saling memperkaya dan menghasilkan performa yang lebih baik dibandingkan jika dilatih secara terpisah.

Pendekatan ini diharapkan dapat meningkatkan efisiensi dan efektivitas analisis teks dengan memanfaatkan representasi laten yang sama untuk dua tugas sekaligus, mengurangi redundansi komputasi, serta memberikan kontribusi terhadap pengembangan model-model NLP yang lebih adaptif dan skalabel. Dengan mengintegrasikan *Autoencoder* untuk pembelajaran representasi, DEC untuk klusterisasi

topik, dan arsitektur MTL untuk klasifikasi sentimen simultan, penelitian ini berupaya menghadirkan solusi komprehensif yang mengatasi keterbatasan pendekatan-pendekatan sebelumnya dalam pemrosesan bahasa alami.

1.2 Rumusan Permasalahan

Dalam pemrosesan bahasa alami (*Natural Language Processing*, NLP), klasifikasi sentimen dan pemodelan topik merupakan dua tugas utama yang sering dilakukan secara terpisah. Pendekatan tradisional seperti *Latent Dirichlet Allocation* (LDA) berbasis probabilistik memiliki keterbatasan dalam menangkap hubungan semantik yang kompleks, sehingga kurang optimal untuk tugas yang membutuhkan pemahaman konteks yang lebih dalam.

Selain itu, pemisahan antara tugas klasifikasi sentimen dan pemodelan topik menyebabkan hilangnya potensi peningkatan performa yang bisa diperoleh dengan memanfaatkan hubungan alami antar tugas tersebut. Model berbasis *deep learning*, seperti *Bidirectional Encoder Representations from Transformers* (BERT), telah terbukti unggul dalam tugas klasifikasi sentimen, namun pendekatan ini belum sepenuhnya diintegrasikan dengan pemodelan topik dalam satu *framework* yang efisien.

Beberapa permasalahan utama yang ingin dijawab dalam penelitian ini adalah:

1. Bagaimana mengembangkan model berbasis *Fully Neural Network* yang dapat menangani klasifikasi sentimen dan pemodelan topik secara simultan?
2. Bagaimana pengaruh pendekatan *Multi-Task Learning* (MTL) terhadap performa model dibandingkan dengan model *single-task*?
3. Bagaimana penerapan *Autoencoder* dapat membantu mengatasi permasalahan dimensi tinggi dalam data teks?
4. Sejauh mana model yang dikembangkan dapat meningkatkan efisiensi dan akurasi dibandingkan dengan metode sebelumnya, seperti analisis sentimen menggunakan BERTopic dan model sentimen BERT?

Melalui penelitian ini, diharapkan dapat diperoleh solusi yang lebih efisien dan akurat dalam menangani kedua tugas secara simultan dengan memanfaatkan pendekatan pembelajaran mendalam berbasis *Fully Neural Network*.

1.3 Tujuan Penelitian

Penelitian ini bertujuan untuk mengembangkan dan mengevaluasi model berbasis *Fully Neural Network* yang mampu melakukan klasifikasi sentimen dan pemodelan topik secara simultan menggunakan pendekatan *Multi-Task Learning* (MTL). Dengan pendekatan ini, model diharapkan dapat memanfaatkan hubungan antara kedua tugas tersebut untuk meningkatkan performa keseluruhan dan efisiensi dalam pemrosesan bahasa alami (*Natural Language Processing*, NLP).

Secara spesifik, penelitian ini memiliki beberapa tujuan utama sebagai berikut:

1. Mengembangkan arsitektur berbasis *Fully Neural Network* yang dapat secara bersamaan melakukan klasifikasi sentimen dan pemodelan topik dalam satu model terpadu. Dengan integrasi ini, model diharapkan mampu menangkap hubungan semantik yang lebih dalam dan menghasilkan representasi teks yang lebih bermakna.
2. Menganalisis pengaruh *Multi-Task Learning* (MTL) terhadap efisiensi dan performa model dalam menangani kedua tugas secara simultan. Evaluasi akan dilakukan untuk memahami bagaimana strategi berbagi parameter antara dua tugas ini dapat meningkatkan akurasi, stabilitas, dan efisiensi model dibandingkan dengan pendekatan *single-task*.
3. Menerapkan *Autoencoder* untuk reduksi dimensi guna meningkatkan representasi fitur dan mengoptimalkan performa model *Fully Neural Network*. *Autoencoder* akan digunakan untuk mengekstraksi informasi laten dari teks secara lebih efektif, mengurangi redundansi fitur, dan membantu model dalam menangkap struktur data yang lebih informatif.

Melalui penelitian ini, diharapkan dapat diperoleh model yang lebih efisien, akurat, dan tangguh dalam menangani klasifikasi sentimen dan pemodelan topik secara simultan. Implementasi *Fully Neural Network* dengan MTL dan *Autoencoder* diharapkan dapat memberikan kontribusi dalam peningkatan teknik pemrosesan bahasa alami, terutama dalam konteks analisis teks skala besar dan kompleks.

1.4 Manfaat Penelitian

Penelitian ini diharapkan dapat memberikan manfaat baik secara teoretis maupun praktis, yang diuraikan sebagai berikut:

1. Manfaat Teoretis

Penelitian ini diharapkan mampu memperluas wawasan dalam bidang *Natural Language Processing* (NLP), terutama dalam hal perancangan model yang menggabungkan analisis sentimen dan pemodelan topik dalam satu arsitektur secara simultan. Penggunaan *Deep Embedded Clustering* (DEC) sebagai komponen pemodelan topik berbasis *unsupervised learning* dalam kerangka *joint model* diharapkan dapat menambah khasanah penelitian mengenai integrasi dua tugas yang berbeda dalam satu sistem pembelajaran. Penelitian ini diharapkan dapat menjadi acuan bagi penelitian berikutnya dalam membangun sistem analisis teks yang lebih terintegrasi dan efisien.

2. Manfaat Praktis

Hasil penelitian ini diharapkan dapat digunakan sebagai alat bantu dalam memahami isi ulasan teks secara lebih menyeluruh, baik dari sisi sentimen maupun topik yang terkandung di dalamnya, dalam satu kali proses inferensi. Sistem yang dikembangkan berpotensi diterapkan pada berbagai domain, seperti pemantauan ulasan produk, evaluasi layanan pelanggan, hingga analisis tren opini publik di platform digital. Pendekatan *joint model* ini juga diharapkan dapat mengurangi beban komputasi dibandingkan menjalankan kedua analisis secara terpisah.

1.5 Ruang Lingkup Penelitian

Batasan dalam penelitian ini ditetapkan sebagai berikut:

1. Penelitian ini menggunakan analisis sentimen biner yang terdiri atas dua kelas, yaitu positif dan negatif.
2. Data yang digunakan merupakan *dataset* IMDB dan Yelp berupa kumpulan teks ulasan yang diperoleh melalui platform Hugging Face, yaitu *dataset* IMDB dan *dataset* Yelp Polarity.
3. Jumlah data yang digunakan terdiri atas 50.000 ulasan pada *dataset* IMDB dan 30.000 ulasan pada *dataset* Yelp. Kedua *dataset* disusun dengan distribusi kelas yang seimbang antara sentimen positif dan negatif.

BAB 2

LANDASAN TEORI

Bab ini memaparkan landasan teori yang menjadi dasar penelitian. Pembahasan dimulai dari konsep *analisis sentimen* beserta tingkatan dan pendekatannya, dilanjutkan dengan *pendeteksian topik* dan berbagai metodenya. Selanjutnya diuraikan dasar-dasar *neural network*, mulai dari arsitektur, fungsi aktivasi, *backpropagation*, hingga metrik evaluasi. Bab ini juga membahas *Bidirectional Encoder Representations from Transformers* (BERT) sebagai model representasi teks berbasis *transformer*, *autoencoder* sebagai komponen kompresi representasi, serta *multi-task learning* sebagai kerangka pelatihan model untuk beberapa tugas sekaligus.

2.1 Analisis Sentimen

Platform daring seperti media sosial, forum diskusi, dan situs ulasan produk memungkinkan pengguna mengekspresikan pendapat secara bebas. Hal ini menghasilkan data opini dalam jumlah besar yang terus bertambah, sehingga muncul kebutuhan akan metode komputasional yang mampu mengolahnya secara otomatis.

Analisis sentimen, yang juga dikenal sebagai *opinion mining*, merupakan bidang dalam pemrosesan bahasa alami yang berfokus pada identifikasi dan ekstraksi opini, sentimen, serta emosi yang terkandung dalam teks (B. Liu, 2012). Secara umum, analisis sentimen mengklasifikasikan opini ke dalam beberapa kategori: positif, negatif, atau netral. Klasifikasi ini dapat dilakukan pada berbagai tingkatan, mulai dari keseluruhan dokumen, per kalimat, hingga aspek tertentu dalam teks. Pada penelitian ini, analisis dilakukan pada tingkat dokumen, artinya setiap ulasan diklasifikasikan ke dalam satu label sentimen. Pendekatan ini sesuai karena dataset yang digunakan, yakni Yelp dan IMDB, masing-masing berisi ulasan dari satu pengguna tentang satu entitas.

Analisis sentimen umumnya dilakukan melalui dua pendekatan (Pang, L. Lee, dan Vaithyanathan, 2002). Pendekatan berbasis leksikon menggunakan kamus sentimen yang berisi kata-kata beserta nilai sentimennya untuk menentukan sentimen suatu teks. Pendekatan berbasis *machine learning* melatih model pada data berlabel agar dapat mempelajari pola bahasa yang berkorelasi dengan sentimen tertentu. Pendekatan ini lebih mampu menangani variasi bahasa, konteks, dan domain yang beragam, terutama pada data berskala besar. Penelitian ini menggunakan pen-

dekatan berbasis *machine learning* sebagai landasan utama.

2.2 Pendeteksian Topik

Topik merujuk pada tema utama yang terkandung dalam sebuah teks. Pendeteksian topik bertujuan menemukan tema-tema yang tersebar dalam kumpulan dokumen tanpa label yang ditentukan sebelumnya (Blei, 2012). Setiap topik direpresentasikan sebagai distribusi probabilitas atas sekumpulan kata, dan maknanya dapat ditelusuri dari kata-kata yang paling sering muncul di dalamnya. Metode ini termasuk ke dalam *unsupervised learning* karena tidak memerlukan anotasi data.

Salah satu metode pendeteksian topik yang umum digunakan adalah BERTopic (Grootendorst, 2022), yang menggabungkan representasi kontekstual dari model bahasa besar dengan *clustering* untuk menghasilkan topik yang koheren secara semantis.

Pada penelitian ini, pendeteksian topik dilakukan dengan pendekatan *clustering*. Dokumen-dokumen yang berdekatan maknanya dikelompokkan ke dalam satu klaster, dan setiap klaster diinterpretasikan sebagai satu topik. Pengelompokan dilakukan pada ruang laten hasil *autoencoder*, sebuah jaringan saraf yang mengompresi teks menjadi representasi yang lebih padat namun tetap mempertahankan informasi semantiknya.

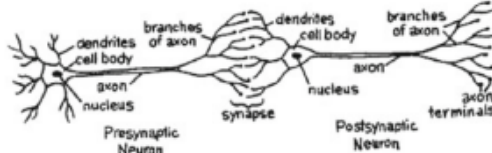
2.3 Neural Network

Jaringan saraf tiruan (*Artificial Neural Network/ANN*) adalah model komputasi yang dirancang meniru cara kerja sistem saraf biologis manusia (Aggarwal, 2018). Seperti neuron biologis yang saling terhubung melalui sinapsis, ANN terdiri dari unit-unit komputasi yang saling terhubung melalui bobot yang dapat disesuaikan. Proses pembelajaran berlangsung melalui penyesuaian bobot-bobot tersebut berdasarkan data pelatihan.

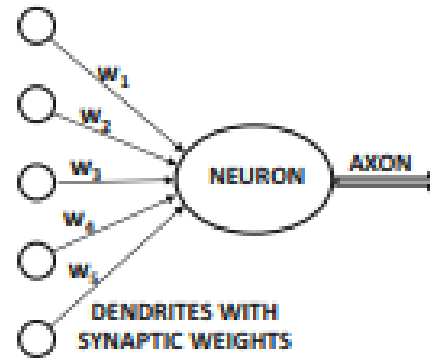
Secara umum, arsitektur ANN tersusun atas tiga komponen utama (Silva *et al.*, 2017). Pertama, *input layer*, yaitu lapisan yang menerima data mentah dari luar sistem dan meneruskannya ke lapisan berikutnya tanpa melakukan transformasi. Kedua, *hidden layer*, yaitu lapisan yang menerapkan fungsi aktivasi untuk mengidentifikasi pola kompleks dari data. Ketiga, *output layer*, yaitu lapisan yang menghasilkan prediksi akhir berdasarkan representasi yang telah dibangun oleh lapisan-lapisan sebelumnya.

Gambar 1 dan Gambar 2 menunjukkan perbandingan antara jaringan saraf biol-

ogis dan ANN.



Gambar 1: Ilustrasi *neural network* secara biologis (Aggarwal, 2018)



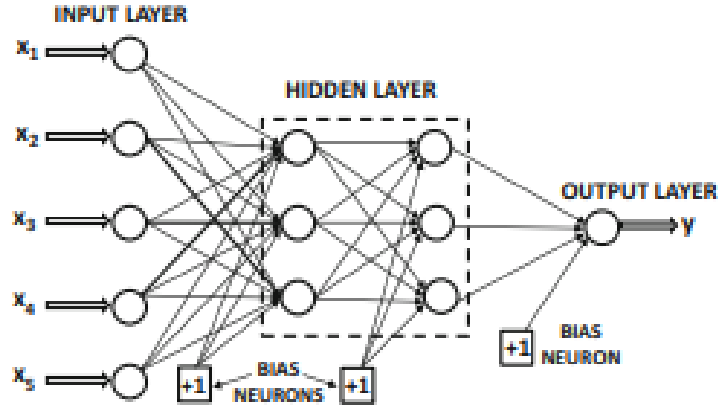
Gambar 2: *Artificial Neural Network* dengan representasi bobot sinapsis (Aggarwal, 2018)

Dalam bentuk paling sederhananya, sebuah *neural network* hanya memiliki satu lapisan komputasi yang disebut *perceptron*. Pada arsitektur ini, terdapat n neuron *input* yang meneruskan fitur ke satu neuron *output* melalui sekumpulan bobot. Nilai *output* diperoleh dari penjumlahan bobot yang dikalikan dengan masing-masing nilai *input*, ditambah sebuah nilai *bias* b yang berfungsi menggeser ambang batas keputusan model. Secara matematis, prediksi *output* dirumuskan sebagai:

$$z = \mathbf{w} \cdot \mathbf{x} + b = \sum_{i=1}^n w_i x_i + b. \quad (2.1)$$

dengan $\mathbf{w}$ adalah vektor bobot, $\mathbf{x}$ adalah vektor input, dan b adalah skalar bias.

Perceptron hanya mampu memodelkan hubungan linier antara *input* dan *output*, sehingga tidak dapat menangani pola yang lebih kompleks. Untuk mengatasi keterbatasan ini, digunakan *multilayer neural network* yang memiliki satu atau lebih *hidden layer* di antara *input layer* dan *output layer*. Setiap lapisan menerima input dari lapisan sebelumnya dan meneruskannya searah menuju *output layer*. Pola aliran satu arah seperti ini dikenal sebagai *feed-forward*, dan arsitekturnya ditunjukkan pada Gambar 3.



Gambar 3: Arsitektur *feed-forward neural network* (Aggarwal, 2018)

Misalkan terdapat k *hidden layer* dengan masing-masing $p_1, \dots, p_k$ neuron, $\mathbf{x}$ sebagai vektor input berisi n neuron, $\mathbf{h}_1, \dots, \mathbf{h}_k$ sebagai keluaran tiap *hidden layer* setelah melewati fungsi aktivasi, dan $\mathbf{o}$ sebagai vektor output berisi m neuron. Matriks bobot $\mathbf{W}_1$ berdimensi $n \times p_1$ menghubungkan *input layer* dengan *hidden layer* pertama, $\mathbf{W}_r$ berdimensi $p_{r-1} \times p_r$ menghubungkan antar *hidden layer*, dan $\mathbf{W}_{k+1}$ berdimensi $p_k \times m$ menghubungkan *hidden layer* terakhir ke *output layer*. Proses komputasi pada tiap lapisan dirumuskan sebagai:

$$\mathbf{h}_1 = \phi_1(\mathbf{W}_1^\top \mathbf{x}), \quad (2.2)$$

$$\mathbf{h}_{r+1} = \phi_{r+1}(\mathbf{W}_{r+1}^\top \mathbf{h}_r), \quad r \in \{1, \dots, k-1\}, \quad (2.3)$$

$$\mathbf{o} = \phi_0(\mathbf{W}_{k+1}^\top \mathbf{h}_k). \quad (2.4)$$

dengan ϕ_i adalah fungsi aktivasi yang diterapkan pada lapisan ke- i . Output akhir $\mathbf{o}$ merupakan hasil transformasi bertingkat dari input $\mathbf{x}$ melalui seluruh lapisan jaringan secara berurutan.

2.3.1 Fungsi Aktivasi

Fungsi aktivasi digunakan dalam *neural network* untuk memberikan sifat non-linier pada model. Tanpa fungsi aktivasi, output jaringan hanya merupakan transformasi linier dari input sehingga tidak mampu menangkap pola yang kompleks (Goodfellow, Bengio, dan Courville, 2016). Berikut adalah fungsi aktivasi yang digunakan dalam penelitian ini.

2.3.1.1 Softmax

Fungsi *softmax* digunakan untuk mengubah vektor bernilai real menjadi distribusi probabilitas pada K kelas. Melalui proses ini, setiap nilai keluaran berada

pada rentang 0 hingga 1, dengan total seluruh probabilitas sama dengan 1. Untuk vektor masukan $\mathbf{v} = (v_1, v_2, \dots, v_K)$, fungsi *softmax* pada elemen ke- j didefinisikan sebagai:

$$\phi(\mathbf{v})_j = \frac{e^{v_j}}{\sum_{k=1}^K e^{v_k}}. \quad (2.5)$$

dengan e adalah bilangan Euler bernilai sekitar 2,718. Elemen dengan nilai v_j yang lebih besar menghasilkan e^{v_j} yang lebih besar, sehingga probabilitas $\phi(\mathbf{v})_j$ yang diperoleh juga lebih tinggi. Fungsi *softmax* umum digunakan pada lapisan keluaran jaringan untuk tugas klasifikasi multikelas karena keluarannya dapat langsung diinterpretasikan sebagai probabilitas prediksi tiap kelas.

2.3.1.2 Rectified Linear Unit (ReLU)

Rectified Linear Unit (ReLU) mengatasi masalah *vanishing gradient* dengan cara meneruskan nilai *input* apa adanya jika positif, dan mengembalikan nol jika negatif:

$$\phi(v) = \max\{v, 0\}. \quad (2.6)$$

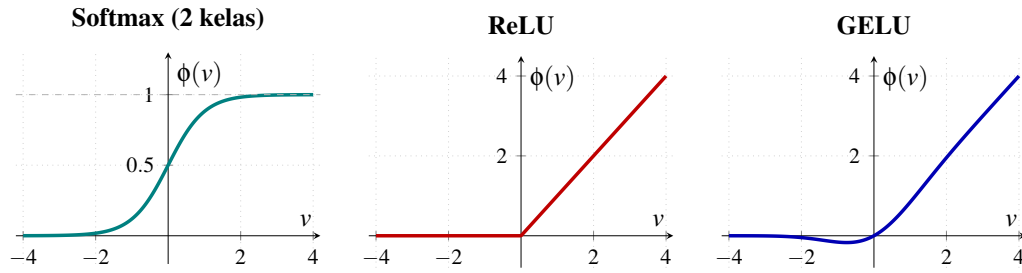
ReLU lebih efisien secara komputasi dan lebih mudah dilatih pada jaringan yang dalam, karena gradiennya selalu bernilai 1 untuk *input* positif sehingga informasi pembaruan tidak melemah saat merambat mundur melalui banyak lapisan.

2.3.1.3 Gaussian Error Linear Unit (GELU)

Gaussian Error Linear Unit (GELU) digunakan sebagai fungsi aktivasi pada arsitektur *Transformer* seperti BERT (Hendrycks dan Gimpel, 2016). Berbeda dengan ReLU yang mengubah seluruh nilai negatif menjadi nol, GELU mengatur kontribusi setiap nilai *input* berdasarkan probabilitas kumulatif distribusi Gaussian pada nilai tersebut:

$$\phi(v) = v \cdot \Phi(v) = v \cdot \frac{1}{2} \left[1 + \operatorname{erf}\left(\frac{v}{\sqrt{2}}\right) \right]. \quad (2.7)$$

dengan $\Phi(v)$ adalah fungsi distribusi kumulatif Gaussian standar, dan $\operatorname{erf}(\cdot)$ adalah fungsi error dengan rentang nilai $[-1, 1]$, sehingga $\Phi(v) \in [0, 1]$. Untuk v besar dan positif, GELU mendekati perilaku ReLU, sedangkan di sekitar nol transisinya berlangsung mulus.



Gambar 4: Grafik fungsi aktivasi: *Softmax*, ReLU, dan GELU

Perbandingan ketiga fungsi aktivasi ditunjukkan pada Gambar 4.

2.3.2 Fungsi Loss

Fungsi *loss* mengukur seberapa jauh prediksi model menyimpang dari nilai yang sebenarnya. Untuk tugas klasifikasi multikelas, *categorical cross-entropy loss* merupakan pilihan yang umum digunakan (Aggarwal, 2018):

$$L = - \sum_{i=1}^n \sum_{j=1}^k y_{ij} \log(\hat{y}_{ij}). \quad (2.8)$$

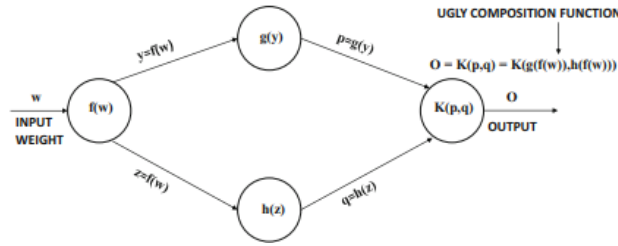
dengan n adalah jumlah sampel, k adalah jumlah kelas, y_{ij} adalah label aktual dalam bentuk *one-hot encoded* bernilai 1 jika sampel ke- i termasuk kelas ke- j dan 0 jika tidak, serta $\hat{y}_{ij}$ adalah probabilitas prediksi model untuk sampel ke- i pada kelas ke- j .

2.3.3 Backpropagation

Setelah nilai *loss* diperoleh, langkah berikutnya dalam proses pelatihan adalah menghitung gradien terhadap parameter model untuk menentukan arah pembaruan bobot. Proses tersebut dilakukan menggunakan algoritma *backpropagation* (Aggarwal, 2018).

Backpropagation bekerja dengan merambatkan informasi kesalahan dari lapisan keluaran menuju lapisan sebelumnya untuk menghitung seberapa besar kontribusi masing-masing parameter terhadap nilai *loss*. Perhitungan tersebut dilakukan menggunakan aturan rantai (*chain rule*), karena keluaran model umumnya diperoleh melalui serangkaian fungsi yang saling bergantung.

Hubungan tersebut dapat dijelaskan menggunakan graf komputasi pada Gambar 5.



Gambar 5: Ilustrasi graf komputasi pada proses *backpropagation* (Aggarwal, 2018)

Pada Gambar 5, parameter w diproses melalui beberapa tahapan transformasi hingga menghasilkan keluaran akhir o . Keluaran tersebut diperoleh dari dua jalur komputasi yang digabungkan melalui fungsi:

$$o = K(g(f(w)), h(f(w))) \quad (2.9)$$

Persamaan 2.9 menunjukkan bahwa keluaran merupakan fungsi komposit terhadap parameter w . Oleh karena itu, pengaruh perubahan parameter terhadap nilai *loss* dihitung menggunakan aturan rantai sebagai berikut:

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial o} \left(\frac{\partial o}{\partial p} \cdot \frac{\partial p}{\partial y} \cdot \frac{\partial y}{\partial w} + \frac{\partial o}{\partial q} \cdot \frac{\partial q}{\partial z} \cdot \frac{\partial z}{\partial w} \right) \quad (2.10)$$

Pada Persamaan 2.10, $\partial L / \partial o$ menyatakan perubahan nilai *loss* terhadap perubahan keluaran model, sedangkan suku di dalam tanda kurung menyatakan pengaruh parameter terhadap keluaran melalui masing-masing jalur komputasi. Gradien diperoleh dengan mengalikan turunan lokal pada setiap tahapan transformasi dan menjumlahkan kontribusi dari seluruh jalur yang memengaruhi keluaran.

Proses penyebaran gradien dari keluaran menuju parameter pada tahapan komputasi sebelumnya inilah yang disebut sebagai *backpropagation*.

2.3.4 Teknik Optimasi

Proses optimasi merupakan tahapan pencarian parameter yang menghasilkan nilai fungsi *loss* sekecil mungkin selama pelatihan (A. Zhang *et al.*, 2023). Gradien yang diperoleh dari *backpropagation* digunakan untuk memperbarui bobot model secara iteratif.

2.3.4.1 Stochastic Gradient Descent (SGD)

Stochastic Gradient Descent (SGD) adalah algoritma optimasi yang diperkenalkan oleh (Rumelhart, Hinton, dan Williams, 1986) untuk meminimalkan fungsi *loss* $L(\mathbf{w})$ secara iteratif dengan memperbarui bobot ke arah negatif gradiennya:

$$\mathbf{w}^{[t+1]} = \mathbf{w}^{[t]} - \eta \nabla L(\mathbf{w})^{[t]}. \quad (2.11)$$

dengan $\mathbf{w}^{[t]}$ adalah vektor bobot pada iterasi ke- t , $\nabla L(\mathbf{w})^{[t]}$ adalah vektor gradien fungsi *loss* terhadap bobot pada iterasi ke- t , dan η adalah skalar *learning rate* yang mengontrol besar langkah pembaruan. Nilai η yang terlalu kecil memperlambat konvergensi, sedangkan nilai yang terlalu besar dapat membuat pelatihan tidak stabil.

2.3.4.2 Adaptive Moment Estimation (Adam)

Adaptive Moment Estimation (Adam), yang dikemukakan oleh (Kingma dan Ba, 2015), mengembangkan SGD dengan mengadaptasi *learning rate* secara individual untuk setiap parameter. Adaptasi ini didasarkan pada dua estimasi momen gradien: momen pertama $\mathbf{v}^{[t]}$ terhadap gradien itu sendiri, dan momen kedua $\mathbf{s}^{[t]}$ terhadap kuadrat gradien:

$$\mathbf{v}^{[t]} = \beta_1 \mathbf{v}^{[t-1]} + (1 - \beta_1) \nabla L(\mathbf{w})^{[t]}. \quad (2.12)$$

$$\mathbf{s}^{[t]} = \beta_2 \mathbf{s}^{[t-1]} + (1 - \beta_2) \left(\nabla L(\mathbf{w})^{[t]} \right)^2. \quad (2.13)$$

dengan β_1 dan β_2 adalah skalar koefisien peluruhan yang mengontrol pengaruh gradien lama terhadap estimasi saat ini. Karena kedua momen diinisialisasi dengan nol, estimasinya cenderung bias ke nol pada iterasi awal. Koreksi bias dilakukan dengan normalisasi:

$$\hat{\mathbf{v}}^{[t]} = \frac{\mathbf{v}^{[t]}}{1 - \beta_1^t}, \quad \hat{\mathbf{s}}^{[t]} = \frac{\mathbf{s}^{[t]}}{1 - \beta_2^t}. \quad (2.14)$$

Seiring bertambahnya t , nilai β_1^t dan β_2^t mendekati nol sehingga koreksi ini semakin tidak berpengaruh. Pembaruan bobot dilakukan menggunakan kedua momen yang telah dikoreksi:

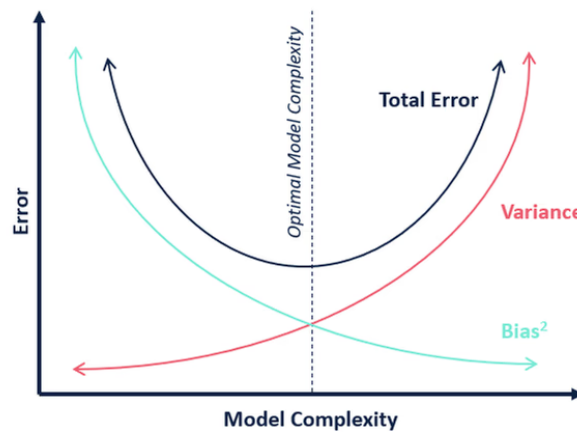
$$\mathbf{w}^{[t+1]} = \mathbf{w}^{[t]} - \frac{\eta}{\sqrt{\hat{\mathbf{s}}^{[t]} + \epsilon}} \hat{\mathbf{v}}^{[t]}. \quad (2.15)$$

dengan $\epsilon = 10^{-8}$ adalah skalar untuk mencegah pembagian dengan nol (A. Zhang *et al.*, 2023). Pembagian dengan $\sqrt{\hat{\mathbf{s}}^{[t]}}$ memperkecil langkah pembaruan untuk parameter dengan gradien besar dan berfluktuasi, serta memperbesar untuk parameter dengan gradien kecil dan konsisten.

2.3.5 Regularisasi

Dalam melatih model, terdapat dua kondisi yang perlu dihindari, yaitu *underfitting* dan *overfitting*. *Underfitting* terjadi ketika model gagal mempelajari pola data pelatihan, ditandai dengan bias yang tinggi. *Overfitting* terjadi ketika model terlalu menghafalkan data pelatihan sehingga performanya menurun pada data baru, ditandai dengan variansi yang tinggi (Goodfellow, Bengio, dan Courville, 2016).

Gambar 6 menunjukkan hubungan antara kompleksitas model dengan bias, variansi, dan total error. Bias menurun seiring meningkatnya kompleksitas model, sementara variansi justru meningkat. Total error yang merupakan kombinasi keduanya mencapai nilai minimum pada titik kompleksitas optimal.



Gambar 6: Ilustrasi *bias-variance tradeoff* terhadap kompleksitas model (Aggarwal, 2018, telah diolah kembali)

Untuk mencegah *overfitting*, salah satu teknik yang digunakan adalah *weight decay* atau L_2 regularization. Teknik ini menambahkan penalti pada fungsi *loss* yang proporsional terhadap besar bobot model, sehingga model cenderung menghasilkan bobot bernilai kecil:

$$\tilde{L}(\mathbf{w}) = L(\mathbf{w}) + \frac{\lambda}{2} \|\mathbf{w}\|^2. \quad (2.16)$$

dengan $L(\mathbf{w})$ adalah fungsi *loss* asal, $\|\mathbf{w}\|^2$ adalah jumlah kuadrat seluruh bobot dalam model, dan λ adalah skalar yang mengontrol kekuatan penalti.

2.3.6 Metrik Evaluasi

Setelah model dilatih, performa klasifikasinya diukur menggunakan data yang belum pernah dilihat selama pelatihan. Dasar dari evaluasi ini adalah *confusion*

matrix, yaitu tabel yang membandingkan prediksi model terhadap label aktual. Dari tabel ini diperoleh empat komponen dasar: *True Positive* (TP) yaitu sampel positif yang diprediksi benar, *True Negative* (TN) yaitu sampel negatif yang diprediksi benar, *False Positive* (FP) yaitu sampel negatif yang salah diprediksi sebagai positif, dan *False Negative* (FN) yaitu sampel positif yang salah diprediksi sebagai negatif.

Dari keempat komponen tersebut, diturunkan empat metrik utama. Akurasi mengukur proporsi keseluruhan prediksi yang benar:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}. \quad (2.17)$$

Precision mengukur seberapa tepat model ketika menyatakan sesuatu sebagai positif, artinya dari semua yang diprediksi positif, berapa banyak yang benar-benar positif:

$$\text{Precision} = \frac{TP}{TP + FP}. \quad (2.18)$$

Recall mengukur seberapa lengkap model dalam menangkap semua sampel positif yang ada, artinya dari semua yang sebenarnya positif, berapa banyak yang berhasil ditemukan:

$$\text{Recall} = \frac{TP}{TP + FN}. \quad (2.19)$$

Kedua metrik ini sering kali saling bertentangan. Model dengan *precision* tinggi cenderung memiliki *recall* rendah, dan sebaliknya. *F1-Score* menggabungkan keduanya melalui rata-rata harmonik:

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2 \times TP}{2 \times TP + FP + FN}. \quad (2.20)$$

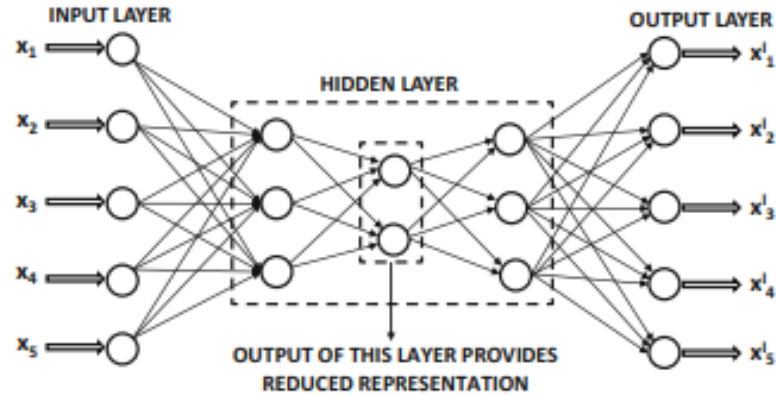
F1-Score berguna pada dataset yang tidak seimbang karena tidak memberikan bobot berlebihan pada kelas yang mendominasi.

2.4 Autoencoder

Pemahaman tentang arsitektur *feed-forward neural network* menjadi landasan untuk memahami *autoencoder*, yang pada dasarnya adalah sebuah jaringan saraf yang dilatih untuk merekonstruksi inputnya (Goodfellow, Bengio, dan Courville, 2016). Karena tidak memerlukan label, *autoencoder* termasuk ke dalam kategori *unsupervised learning* (Hinton dan Salakhutdinov, 2006).

Arsitektur *autoencoder* (Vincent *et al.*, 2010) terdiri dari dua komponen utama: *encoder* yang memetakan representasi input dari dimensi tinggi ke dimensi yang

lebih rendah, dan *decoder* yang merekonstruksi representasi tersebut ke dimensi semula. Lapisan tersempit di antara keduanya disebut *bottleneck* atau ruang laten. Arsitektur lengkapnya ditunjukkan pada Gambar 7.



Gambar 7: Arsitektur *autoencoder* menunjukkan proses *encoding* dan *decoding* (Aggarwal, 2018)

2.4.1 Encoder dan Decoder

Secara formal, misalkan $\mathbf{x} \in \mathbb{R}^D$ adalah vektor input berdimensi D . *Encoder* memetakan $\mathbf{x}$ ke dalam vektor laten $\mathbf{z} \in \mathbb{R}^d$ dengan $d < D$:

$$\mathbf{z} = f_{\theta}(\mathbf{x}) = \sigma_h(\mathbf{W}_h \mathbf{x} + \mathbf{b}_h). \quad (2.21)$$

dengan $\mathbf{W}_h \in \mathbb{R}^{d \times D}$ adalah matriks bobot *encoder*, $\mathbf{b}_h \in \mathbb{R}^d$ adalah vektor bias, dan $\sigma_h(\cdot)$ adalah fungsi aktivasi. Simbol θ merepresentasikan seluruh parameter pada bagian *encoder*.

Selanjutnya, *decoder* memetakan kembali vektor laten $\mathbf{z}$ ke dalam ruang dimensi asal untuk menghasilkan rekonstruksi $\hat{\mathbf{x}} \in \mathbb{R}^D$:

$$\hat{\mathbf{x}} = g_{\theta'}(\mathbf{z}) = \sigma_o(\mathbf{W}_o \mathbf{z} + \mathbf{b}_o). \quad (2.22)$$

dengan $\mathbf{W}_o \in \mathbb{R}^{D \times d}$ adalah matriks bobot *decoder*, $\mathbf{b}_o \in \mathbb{R}^D$ adalah vektor bias, dan $\sigma_o(\cdot)$ adalah fungsi aktivasi output. Simbol θ' merepresentasikan seluruh parameter pada bagian *decoder*. Secara keseluruhan, *autoencoder* melakukan transformasi:

$$\hat{\mathbf{x}} = g_{\theta'}(f_{\theta}(\mathbf{x})). \quad (2.23)$$

2.4.2 Fungsi Objektif

Autoencoder dilatih dengan meminimalkan selisih antara *input* asal dan hasil rekonstruksinya. Pada penelitian ini, fungsi *loss* yang digunakan adalah *Mean Squared Error* (MSE):

$$\mathcal{L}(\theta, \theta') = \frac{1}{n} \sum_{i=1}^n \|\mathbf{x}_i - g_{\theta'}(f_{\theta}(\mathbf{x}_i))\|^2. \quad (2.24)$$

dengan n adalah jumlah sampel dan $\|\cdot\|^2$ adalah kuadrat norma Euclidean. Parameter *encoder* θ dan *decoder* θ' diperbarui secara bersama-sama menggunakan *backpropagation* dan optimasi Adam (Vincent *et al.*, 2010).

2.5 Bidirectional Encoder Representations from Transformers (BERT)

Bidirectional Encoder Representations from Transformers (BERT) merupakan model bahasa yang diperkenalkan oleh (Devlin *et al.*, 2018). Berbeda dari model bahasa sebelumnya yang membaca teks secara satu arah, BERT memahami makna suatu kata dengan mempertimbangkan konteks dari kiri maupun kanan secara bersamaan. Kemampuan ini dimungkinkan oleh arsitekturnya yang didasarkan pada *Transformer Encoder*, yaitu mekanisme yang memungkinkan setiap token menghitung relevansinya terhadap token lain dalam kalimat secara paralel.

Sebelum digunakan, BERT dilatih pada korpus teks berskala besar, yaitu Wikipedia bahasa Inggris dan BookCorpus. Pada penelitian ini, bobot BERT yang sudah terlatih tidak diperbarui selama pelatihan. BERT digunakan murni sebagai ekstraktor fitur yang dibekukan (*frozen*), sehingga representasi yang dihasilkan langsung digunakan sebagai fitur untuk tahap berikutnya.

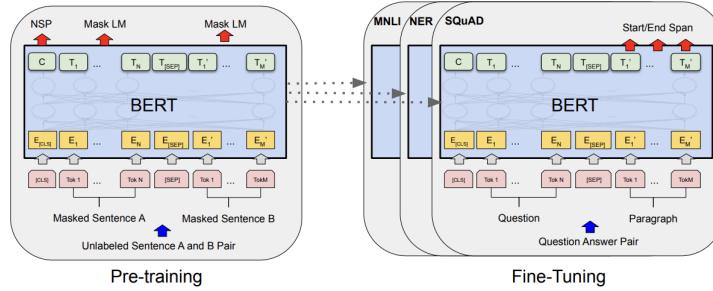
2.5.1 Arsitektur BERT

Arsitektur BERT tersusun dari L lapisan *Transformer Encoder* yang identik, masing-masing terdiri dari dua sub-lapisan: *Multi-Head Self-Attention* dan *Position-wise Feed-Forward Network*. Keluaran setiap lapisan, seperti ditunjukkan pada Gambar 8, dihitung sebagai:

$$\mathbf{H}^{(\ell)} = \text{LayerNorm}\left(\mathbf{H}^{(\ell-1)} + \text{MultiHeadAttention}(\mathbf{H}^{(\ell-1)})\right), \quad (2.25)$$

$$\mathbf{H}_{\text{out}}^{(\ell)} = \text{LayerNorm}\left(\mathbf{H}^{(\ell)} + \text{FFN}(\mathbf{H}^{(\ell)})\right). \quad (2.26)$$

dengan $\mathbf{H}^{(\ell)}$ adalah matriks *hidden state* pada lapisan ke- ℓ , dan LayerNorm adalah normalisasi lapisan yang membantu stabilitas pelatihan. Jumlah lapisan L , dimensi *hidden state* H , dan jumlah *attention head* A bergantung pada varian model yang dipilih.



Gambar 8: Arsitektur BERT dengan beberapa lapisan *Transformer Encoder* (Devlin *et al.*, 2018)

2.5.2 Mekanisme *Self-Attention*

BERT menggunakan mekanisme *self-attention* untuk menentukan seberapa penting suatu kata terhadap kata-kata lain dalam kalimat yang sama. Input $\mathbf{X} \in \mathbb{R}^{n \times d}$ dengan n token dan dimensi d diproyeksikan ke tiga representasi melalui matriks bobot yang dipelajari saat pelatihan, yaitu *query* $\mathbf{Q}$, *key* $\mathbf{K}$, dan *value* $\mathbf{V}$:

$$\mathbf{Q} = \mathbf{X}\mathbf{W}_Q \in \mathbb{R}^{n \times d_k}, \quad (2.27)$$

$$\mathbf{K} = \mathbf{X}\mathbf{W}_K \in \mathbb{R}^{n \times d_k}, \quad (2.28)$$

$$\mathbf{V} = \mathbf{X}\mathbf{W}_V \in \mathbb{R}^{n \times d_v}. \quad (2.29)$$

dengan $\mathbf{W}_Q, \mathbf{W}_K \in \mathbb{R}^{d \times d_k}$ dan $\mathbf{W}_V \in \mathbb{R}^{d \times d_v}$ adalah matriks bobot yang dapat dipelajari. *Query* merepresentasikan kata yang sedang mencari konteks, *key* merepresentasikan kata-kata yang berpotensi menjadi konteks, dan *value* adalah informasi yang diambil apabila konteks tersebut dianggap relevan.

Skor relevansi antara setiap pasangan kata dihitung melalui perkalian *query* dan *key*, lalu dinormalisasi dengan $\sqrt{d_k}$ untuk menjaga skala nilai. Hasilnya kemudian diubah menjadi distribusi probabilitas menggunakan *softmax*:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) \mathbf{V}. \quad (2.30)$$

Untuk meningkatkan kemampuan representasi, BERT menjalankan mekanisme *attention* ini secara paralel sebanyak h kali pada *subspace* yang berbeda, yang disebut *Multi-Head Attention*. Setiap hasil digabungkan dan diproyeksikan kembali ke

dimensi semula:

$$\text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \mathbf{W}_O. \quad (2.31)$$

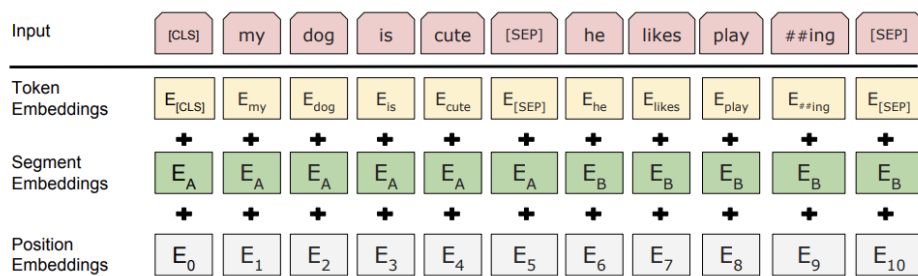
dengan $\text{head}_i = \text{Attention}(\mathbf{QW}_Q^i, \mathbf{KW}_K^i, \mathbf{VW}_V^i)$. Pada BERT-Base, $h = 12$ dan $d_k = d_v = 64$.

2.5.3 Representasi Input BERT

Sebelum teks dapat diproses oleh BERT, setiap token perlu diubah menjadi vektor numerik. BERT merepresentasikan setiap token sebagai penjumlahan dari tiga jenis *embedding* yang masing-masing membawa informasi berbeda:

$$\mathbf{E}_i = \mathbf{E}_{\text{token}}^{(i)} + \mathbf{E}_{\text{segment}}^{(i)} + \mathbf{E}_{\text{position}}^{(i)}. \quad (2.32)$$

Token embedding merepresentasikan makna semantik dari setiap kata atau *sub-word*. BERT menggunakan tokenisasi WordPiece dengan kosakata sebesar 30.000 token, sehingga kata yang tidak dikenal dapat diurai menjadi unit yang lebih kecil. *Segment embedding* menandai asal kalimat dari setiap token, yang berguna ketika *input* terdiri dari dua kalimat. *Position embedding* merepresentasikan posisi setiap token dalam urutan, sebab mekanisme *self-attention* pada dasarnya tidak mengandung informasi urutan. Berbeda dari Transformer asli yang menggunakan rumus sinusoidal, BERT mempelajari *position embedding* langsung dari data dengan panjang urutan maksimum token. Ilustrasi representasi input BERT ditunjukkan pada Gambar 9.



Gambar 9: Representasi *input* BERT yang terdiri dari *token*, *segment*, dan *position embedding* (Devlin *et al.*, 2018)

Selain token biasa, BERT menyisipkan token khusus [CLS] di awal setiap urutan *input*. Token ini dirancang untuk mengakumulasi informasi dari seluruh token dalam kalimat melalui lapisan-lapisan *self-attention*. Pada penelitian ini, representasi token [CLS] pada lapisan terakhir BERT inilah yang digunakan sebagai vektor fitur untuk setiap ulasan.

Pada penelitian ini, BERT digunakan sepenuhnya sebagai ekstraktor fitur dengan bobot yang dibekukan. Setiap ulasan teks dimasukkan ke dalam BERT dengan panjang maksimum token, kemudian representasi *hidden state* pada posisi token [CLS] dari lapisan terakhir diambil sebagai fitur:

$$\mathbf{f} = \mathbf{H}_{[\text{CLS}]}^{(L)} \in \mathbb{R}^d. \quad (2.33)$$

dengan $\mathbf{H}_{[\text{CLS}]}^{(L)}$ adalah vektor *hidden state* token [CLS] pada lapisan ke- L , dan d adalah dimensi *hidden state* yang bergantung pada varian model yang digunakan. Vektor $\mathbf{f}$ ini selanjutnya menjadi representasi numerik dari setiap ulasan dan digunakan sebagai masukan *autoencoder* pada tahap berikutnya.

2.6 Multi-Task Learning

Multi-Task Learning (MTL) merupakan pendekatan dalam *machine learning* yang bertujuan meningkatkan kemampuan generalisasi model dengan memanfaatkan informasi dari beberapa tugas yang saling berkaitan secara bersamaan (Caruana, 1997). Ketika dua tugas memiliki keterkaitan, informasi pembelajaran dari satu tugas dapat membantu tugas lainnya karena keduanya berbagi pola atau struktur yang sama dalam data.

Konsep kunci yang mendasari MTL adalah *inductive transfer*, yaitu kemampuan model untuk memindahkan pengetahuan yang dipelajari dari satu tugas ke tugas lainnya. (Caruana, 1997) menjelaskan bahwa MTL mencapai hal ini dengan cara mempelajari beberapa tugas secara paralel menggunakan representasi bersama. Tugas-tugas lain berfungsi sebagai *inductive bias*, yaitu mekanisme yang mengarahkan model agar mempelajari hipotesis yang dapat menjelaskan lebih dari satu tugas sekaligus (Ruder, 2017).

Pada penelitian ini, MTL digunakan untuk mempelajari analisis sentimen dan pendeteksian topik secara bersamaan. Kedua tugas ini memiliki keterkaitan yang kuat karena sama-sama bergantung pada pemahaman semantis teks ulasan, sehingga representasi yang berguna untuk mendeteksi topik juga cenderung berguna untuk mengklasifikasikan sentimen.

2.6.1 Formulasi MTL

Misalkan terdapat T tugas yang dipelajari secara bersamaan. Untuk setiap tugas $t \in \{1, 2, \dots, T\}$, terdapat dataset $\mathcal{D}_t = \{(\mathbf{x}_i^{(t)}, y_i^{(t)})\}_{i=1}^{n_t}$ dan fungsi *loss* $\mathcal{L}_t$ yang mengukur kesalahan prediksi pada tugas ke- t . Karena MTL mempelajari beberapa

tugas secara bersamaan, proses optimasi dilakukan dengan menggabungkan kontribusi seluruh fungsi *loss* ke dalam satu fungsi objektif:

$$\mathcal{L}_{\text{MTL}} = \sum_{t=1}^T \alpha_t \mathcal{L}_t(\theta_t, \theta_{\text{shared}}). \quad (2.34)$$

dengan α_t adalah skalar bobot yang mengontrol seberapa besar kontribusi tugas ke- t terhadap total *loss*, θ_t adalah parameter yang spesifik untuk tugas ke- t , dan θ_{shared} adalah parameter yang dibagikan bersama oleh semua tugas. Parameter optimal diperoleh dengan meminimalkan persamaan (2.34) secara bersama-sama:

$$\theta_{\text{shared}}^*, \{\theta_t^*\}_{t=1}^T = \arg \min_{\theta_{\text{shared}}, \{\theta_t\}} \mathcal{L}_{\text{MTL}}. \quad (2.35)$$

Pada penelitian ini, fungsi *loss* total terdiri dari dua komponen. Komponen pertama adalah *clustering loss* berbasis *Kullback-Leibler Divergence* (KLD), yang mengoptimalkan jarak antara distribusi penugasan kluster Q dan distribusi target P . Komponen kedua adalah *sentiment loss* berbasis *categorical cross-entropy* untuk klasifikasi sentimen. Keduanya digabungkan sebagai berikut:

$$\mathcal{L}_{\text{MTL}} = \alpha_1 \cdot \mathcal{L}_{\text{cluster}} + \alpha_2 \cdot \mathcal{L}_{\text{sentiment}}. \quad (2.36)$$

dengan α_1 dan α_2 adalah skalar hiperparameter yang mengontrol bobot masing-masing komponen. *Clustering loss* diformulasikan sebagai KLD antara distribusi target P dan distribusi penugasan Q :

$$\mathcal{L}_{\text{cluster}} = D_{KL}(P||Q) = \sum_i \sum_j p_{ij} \log \frac{p_{ij}}{q_{ij}}. \quad (2.37)$$

dengan q_{ij} adalah probabilitas sampel ke- i termasuk ke kluster ke- j , dan p_{ij} adalah distribusi target yang diturunkan dari Q sebagai berikut:

$$p_{ij} = \frac{q_{ij}^2 / \sum_i q_{ij}}{\sum_{j'} q_{ij'}^2 / \sum_i q_{ij'}}. \quad (2.38)$$

Pembagian dengan $\sum_i q_{ij}$ menormalisasi frekuensi kluster agar kluster yang besar tidak mendominasi, sedangkan operasi kuadrat pada q_{ij} membuat distribusi keanggotaan lebih terkonsentrasi pada satu kluster.

2.6.2 Hard Parameter Sharing

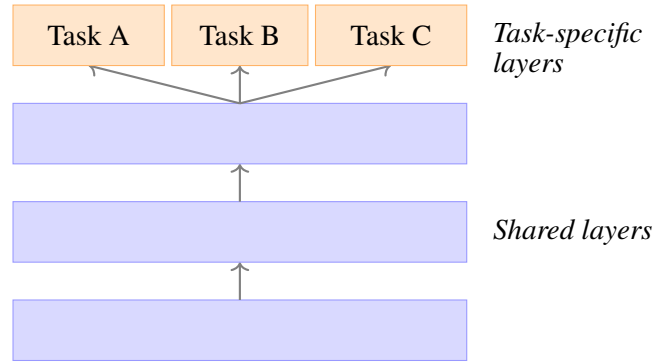
Hard parameter sharing merupakan pendekatan MTL yang paling umum digunakan dalam *deep learning* (Ruder, 2017). Pada pendekatan ini, lapisan tersem-

bunyi bersifat *shared* antar semua tugas, sementara setiap tugas memiliki lapisan *output* tersendiri. Secara formal, untuk input $\mathbf{x}$, proses komputasinya adalah:

$$\mathbf{h}_{\text{shared}} = f_{\text{shared}}(\mathbf{x}; \theta_{\text{shared}}), \quad (2.39)$$

$$\hat{y}_t = f_t(\mathbf{h}_{\text{shared}}; \theta_t), \quad t = 1, \dots, T. \quad (2.40)$$

dengan f_{shared} adalah jaringan bersama yang menghasilkan vektor representasi $\mathbf{h}_{\text{shared}}$, dan f_t adalah jaringan spesifik tugas ke- t . (Baxter, 2000) menunjukkan secara teoritis bahwa mekanisme ini mengurangi risiko *overfitting* karena gradien dari semua tugas mengalir ke lapisan bersama selama *backpropagation*. Arsitektur *hard parameter sharing* diilustrasikan pada Gambar 10.



Gambar 10: Ilustrasi arsitektur *hard parameter sharing* dalam *Multi-Task Learning* (Ruder, 2017, telah diolah kembali)

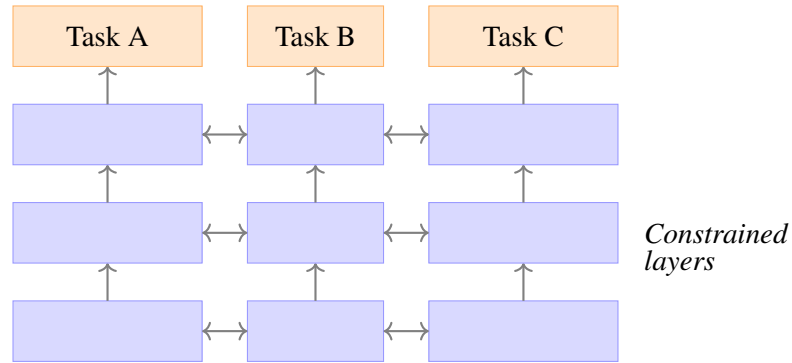
Pada penelitian ini, pendekatan *hard parameter sharing* diterapkan dengan menjadikan *autoencoder* sebagai jaringan bersama. Representasi laten $\mathbf{z}$ yang dihasilkan oleh *encoder* digunakan secara bersamaan oleh dua *head*: satu untuk pendeteksian topik melalui *clustering layer*, dan satu lagi untuk klasifikasi sentimen melalui *sentiment classifier*.

2.6.3 Soft Parameter Sharing

Soft parameter sharing mengambil pendekatan yang berbeda. Setiap tugas memiliki model tersendiri dengan parameter masing-masing, namun parameter antar model diatur agar tidak terlalu jauh berbeda melalui regularisasi (Ruder, 2017). Salah satu cara yang umum digunakan adalah menambahkan penalti ℓ_2 terhadap perbedaan parameter antar tugas:

$$\mathcal{L}_{\text{soft}} = \sum_{t=1}^T \mathcal{L}_t(\theta_t) + \lambda \sum_{t=1}^T \sum_{t'=t+1}^T \|\theta_t - \theta_{t'}\|_2^2. \quad (2.41)$$

dengan λ adalah skalar koefisien regularisasi yang mengontrol seberapa mirip parameter antar tugas harus dijaga, dan $\|\theta_t - \theta_{t'}\|_2^2$ mengukur seberapa jauh kedua set parameter berbeda. Arsitektur *soft parameter sharing* diilustrasikan pada Gambar 11.



Gambar 11: Ilustrasi arsitektur *soft parameter sharing* (Ruder, 2017, telah diolah kembali)

Perbedaan mendasar antara keduanya terletak pada bagaimana pengetahuan dibagikan. Pada *hard parameter sharing*, bobot benar-benar identik sehingga semua tugas menggunakan representasi yang sama persis. Pada *soft parameter sharing*, setiap tugas tetap memiliki ruang belajar sendiri namun didorong untuk tidak terlalu menyimpang, sehingga lebih fleksibel namun membutuhkan lebih banyak parameter dan pengaturan λ yang tepat.

2.7 Clustering

Clustering merupakan teknik *unsupervised learning* yang mengelompokkan data berdasarkan kemiripan tanpa memerlukan label (Bezdek, 1984). Berdasarkan cara pengelompokannya, terdapat dua pendekatan utama. *Hard clustering* membatasi setiap titik data ke satu kluster, sedangkan *soft clustering* memungkinkan satu titik data masuk ke beberapa kluster sekaligus.

Metode *clustering* dapat diklasifikasikan ke dalam beberapa pendekatan, di antaranya *centroid-based*, *distribution-based*, *connectivity-based*, dan *density-based*. Penelitian ini menggunakan pendekatan *centroid-based clustering*, yaitu metode yang memisahkan data berdasarkan jarak setiap titik data terhadap pusat kluster (*centroid*) (Yusdiansyah dan Suryadi, 2019). Untuk mengukur jarak tersebut digunakan jarak Euclidean:

$$\|\mathbf{p} - \mathbf{q}\| = \sqrt{\sum_{d=1}^D (p_d - q_d)^2}. \quad (2.42)$$

dengan $\mathbf{p} = (p_1, \dots, p_D)$ dan $\mathbf{q} = (q_1, \dots, q_D)$ adalah dua vektor dalam ruang berdimensi D .

Penelitian ini menggunakan *Deep Embedded Clustering* (DEC) (Xie, Girshick, dan Farhadi, 2016) sebagai metode pendeteksian topik, yang mempelajari representasi laten dan melakukan *clustering* secara bersamaan. DEC menerapkan *soft clustering* melalui distribusi probabilitas penugasan klaster, sehingga setiap ulasan dapat terhubung ke lebih dari satu topik. Sebagai inisialisasi, DEC menggunakan K-Means untuk menentukan pusat klaster awal di ruang *embedding*.

2.7.1 K-Means Clustering

K-Means merupakan algoritma *centroid-based clustering* yang mengelompokkan data dengan meminimalkan total jarak antara setiap titik data dan pusat klasternya, dan termasuk ke dalam kategori *hard clustering* (MacQueen, 1967).

Secara formal, misalkan terdapat himpunan data $X = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N\}$ dengan $\mathbf{x}_n \in \mathbb{R}^D$ dan jumlah klaster yang diinginkan adalah K . K-Means meminimalkan fungsi objektif berikut:

$$J = \sum_{n=1}^N \sum_{k=1}^K r_{nk} \|\mathbf{x}_n - \boldsymbol{\mu}_k\|^2. \quad (2.43)$$

dengan $\boldsymbol{\mu}_k$ adalah vektor *centroid* klaster ke- k , dan r_{nk} adalah skalar variabel indikator biner yang bernilai 1 jika titik data $\mathbf{x}_n$ termasuk klaster ke- k dan 0 jika tidak. Nilai J mengukur total kuadrat jarak seluruh titik data ke pusat klasternya masing-masing.

Proses optimasi dilakukan secara iteratif melalui dua tahap yang bergantian. Pada tahap pertama (*assignment step*), setiap titik data dialokasikan ke klaster dengan *centroid* terdekat:

$$r_{nk} = \begin{cases} 1, & k = \arg \min_j \|\mathbf{x}_n - \boldsymbol{\mu}_j\|^2 \\ 0, & \text{yang lainnya.} \end{cases} \quad (2.44)$$

Pada tahap kedua (*update step*), *centroid* setiap klaster diperbarui menjadi rata-rata dari seluruh titik yang menjadi anggotanya:

$$\boldsymbol{\mu}_k = \frac{\sum_{n=1}^N r_{nk} \mathbf{x}_n}{\sum_{n=1}^N r_{nk}}. \quad (2.45)$$

Pembilang adalah jumlah posisi seluruh anggota klaster ke- k , sedangkan penyebut adalah jumlah anggotanya. Dua tahap ini diulang sampai nilai J tidak berubah

signifikan atau jumlah iterasi mencapai batas maksimum. Algoritma lengkapnya dapat dilihat pada Tabel 1.

Tabel 1: Algoritma K-Means *Clustering*

Algoritma K-Means <i>Clustering</i>	
INPUT: Himpunan data $\{\mathbf{x}_i\}_{i=1}^N$, banyak kluster K , batas toleransi ϵ , dan maksimal iterasi maxiter	
1. Inisialisasi secara acak himpunan <i>centroid</i> $\{\boldsymbol{\mu}_j\}_{j=1}^K$	
2. Tetapkan $\text{iter} = 0$	
3. REPEAT	
4. Tetapkan $\text{iter} = \text{iter} + 1$	
5. <i>Assignment step:</i> Cari nilai $r_{nk} = \begin{cases} 1, & k = \arg \min_j \ \mathbf{x}_n - \boldsymbol{\mu}_j\ ^2 \\ 0, & \text{yang lainnya} \end{cases}$	
6. <i>Update step:</i> Cari nilai $\boldsymbol{\mu}_k = \frac{\sum_{n=1}^N r_{nk} \mathbf{x}_n}{\sum_{n=1}^N r_{nk}}$	
7. Hitung $J = \sum_{n=1}^N \sum_{k=1}^K r_{nk} \ \mathbf{x}_n - \boldsymbol{\mu}_k\ ^2$	
8. UNTIL $J < \epsilon$ OR $\text{iter} > \text{maxiter}$	
9. Untuk setiap $n = 1, 2, \dots, N$, tetapkan nilai $u_n = \arg \max_k (r_{nk})$, dengan $k = 1, 2, \dots, K$	
10. Bentuk $\{C_1, C_2, \dots, C_K\}$, dengan C_k merupakan himpunan anggota data $\mathbf{x}_i$ di mana $u_i = k$	
11. STOP	
OUTPUT: Nilai keanggotaan $\{u_n\}_{n=1}^N$ dan himpunan kluster $\{C_k\}_{k=1}^K$	

2.8 Interpretabilitas Model *Deep Learning*

Interpretabilitas model *deep learning* merujuk pada kemampuan untuk memahami dan menjelaskan keputusan yang dihasilkan oleh model yang bersifat *black-box* (Sundararajan, Taly, dan Yan, 2017). Dalam konteks pemrosesan bahasa alami, interpretabilitas penting karena model perlu mampu menjelaskan mengapa suatu teks diklasifikasikan ke dalam kategori tertentu, terutama pada aplikasi analisis sentimen dan pengelompokan topik.

Secara umum, metode interpretabilitas dapat dikategorikan menjadi dua kelompok (J. Li *et al.*, 2016). Kelompok pertama adalah metode *intrinsic*, yaitu model yang secara bawaan dapat diinterpretasi seperti pohon keputusan dan regresi linier. Kelompok kedua adalah metode *post-hoc*, yaitu metode yang diterapkan setelah model dilatih untuk menganalisis perilaku model yang kompleks tanpa mengubah arsitekturnya.

Dalam penelitian ini, metode *post-hoc* digunakan untuk menganalisis kontribusi setiap token terhadap prediksi sentimen model, yaitu *occlusion-based token attribution* dan *Integrated Gradients*.

2.8.1 Occlusion-based Token Attribution

Occlusion-based attribution merupakan salah satu metode *post-hoc* yang banyak digunakan untuk menginterpretasi model berbasis teks (Zeiler dan Fergus, 2014; J. Li *et al.*, 2016). Metode ini mengukur pentingnya suatu token dengan cara menggantinya dan mengamati perubahan prediksi model.

Pada model berbasis *transformer* seperti BERT, oklusi dilakukan dengan mengganti token target menggunakan token khusus [MASK], sehingga model tetap menerima input dengan panjang yang sama. Pendekatan ini dikenal sebagai *leave-one-out attribution*.

Secara formal, diberikan teks input $\mathbf{x} = (x_1, x_2, \dots, x_T)$ yang terdiri dari T token, representasi kontekstual diperoleh melalui BERT sebagai:

$$\mathbf{h} = \text{BERT}(\mathbf{x}) \in \mathbb{R}^d. \quad (2.46)$$

dengan $\mathbf{h}$ merupakan vektor representasi [CLS] dari lapisan terakhir BERT. Representasi ini kemudian diteruskan ke lapisan klasifikasi sentimen untuk menghasilkan distribusi probabilitas:

$$f(\mathbf{x}) = \text{softmax}(\mathbf{W} \cdot \mathbf{h} + \mathbf{b}) \in \mathbb{R}^C. \quad (2.47)$$

dengan C adalah jumlah kelas sentimen, $\mathbf{W}$ adalah matriks bobot dan $\mathbf{b}$ adalah vektor bias lapisan klasifikasi. Skor atribusi token x_i terhadap kelas sentimen c didefinisikan sebagai selisih probabilitas prediksi sebelum dan sesudah oklusi:

$$\text{Attr}(x_i, c) = f_c(\mathbf{x}) - f_c(\mathbf{x}_{\setminus i}). \quad (2.48)$$

dengan $f_c(\mathbf{x})$ adalah skalar probabilitas prediksi kelas c pada teks asli, dan $f_c(\mathbf{x}_{\setminus i})$ adalah skalar probabilitas prediksi setelah token x_i diganti dengan [MASK]. Nilai positif berarti token x_i mendukung prediksi kelas c , nilai negatif berarti token menekannya, dan nilai mendekati nol berarti token tidak berpengaruh signifikan.

Dibandingkan dengan metode berbasis gradien, *occlusion-based attribution* lebih mudah diinterpretasi karena tidak bergantung pada komputasi turunan yang kompleks. Namun, metode ini memiliki kompleksitas waktu $O(T)$ terhadap panjang token sehingga lebih mahal secara komputasi untuk teks yang panjang (J. Li *et al.*, 2016).

2.8.2 Integrated Gradients

Integrated Gradients (IG) merupakan metode atribusi berbasis gradien yang diusulkan oleh (Sundararajan, Taly, dan Yan, 2017). Berbeda dengan *occlusion-based attribution* yang bersifat diskrit, IG menghitung kontribusi setiap dimensi input secara kontinu dengan mengintegrasikan gradien prediksi model sepanjang jalur lurus dari *baseline* menuju input asli.

Secara formal, diberikan vektor input $\mathbf{x} \in \mathbb{R}^d$ dan vektor *baseline* $\mathbf{x}' \in \mathbb{R}^d$ yang umumnya berupa vektor nol, atribusi IG untuk dimensi ke- i didefinisikan sebagai:

$$\text{IG}_i(\mathbf{x}) = (x_i - x'_i) \times \int_{\alpha=0}^1 \frac{\partial f(\mathbf{x}' + \alpha(\mathbf{x} - \mathbf{x}'))}{\partial x_i} d\alpha. \quad (2.49)$$

dengan α adalah skalar parameter yang menunjukkan posisi pada jalur dari *baseline* menuju input asli, dan $\frac{\partial f}{\partial x_i}$ adalah skalar gradien prediksi terhadap dimensi ke- i . Dalam praktiknya, integral tersebut diaproksimasi menggunakan *Riemann sum* dengan m langkah interpolasi:

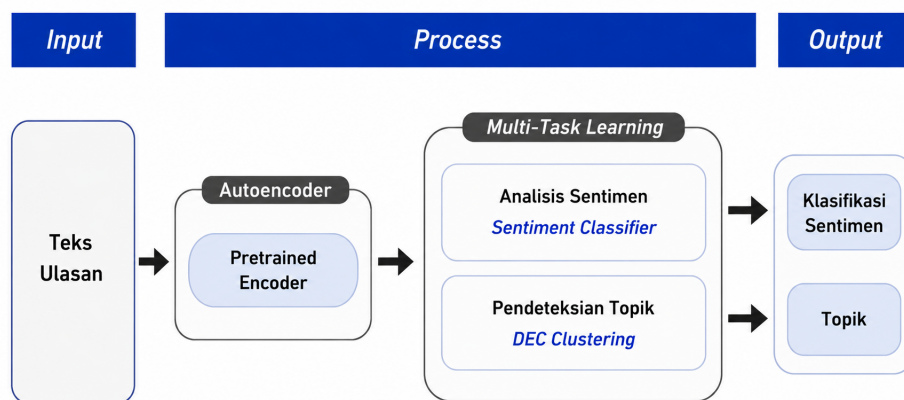
$$\text{IG}_i(\mathbf{x}) \approx (x_i - x'_i) \times \sum_{k=1}^m \frac{\partial f\left(\mathbf{x}' + \frac{k}{m}(\mathbf{x} - \mathbf{x}')\right)}{\partial x_i} \times \frac{1}{m}. \quad (2.50)$$

Metode IG memenuhi dua karakteristik penting, yaitu *sensitivity* dan *implementation invariance* (Sundararajan, Taly, dan Yan, 2017). *Sensitivity* menjamin bahwa apabila suatu fitur mempengaruhi prediksi model, fitur tersebut akan mendapatkan atribusi bernilai tidak nol. *Implementation invariance* menjamin bahwa dua model yang secara fungsional ekuivalen akan menghasilkan atribusi yang sama meskipun memiliki implementasi internal yang berbeda.

BAB 3

METODE MODEL *JOINT DOCUMENT-LEVEL SENTIMENT-TOPIC* (JDST) BERBASIS *DEEP EMBEDDED CLUSTERING* UNTUK ANALISIS SENTIMEN

Bab ini menjelaskan arsitektur dan metode yang digunakan dalam pengembangan model *Joint Document-Level Sentiment-Topic* (JDST-DEC) berbasis *multitask learning*. Sebelum membahas model JDST-DEC, terlebih dahulu dijelaskan komponen representasi teks menggunakan BERT pada Subbab 3.1. Selanjutnya, alur pelatihan model *multitask* dijelaskan pada Subbab 3.2, yang mencakup implementasi *autoencoder*, *Deep Embedded Clustering* (DEC), serta arsitektur JDST-DEC secara keseluruhan. Interpretasi model menggunakan metode atribusi dibahas pada Subbab 3.3, sedangkan evaluasi kualitas topik dijelaskan pada Subbab 3.4. Alur umum proses model dalam melakukan analisis sentimen dan pendeteksian topik secara simultan ditunjukkan pada Gambar 1.



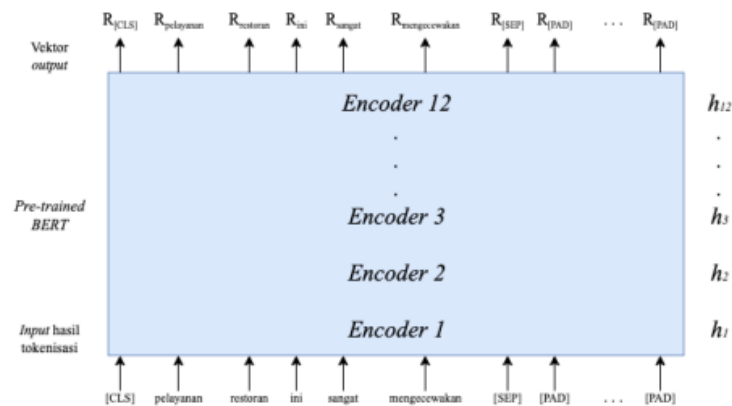
Gambar 1: Alur metode analisis sentimen dan pendeteksian topik

3.1 *Bidirectional Encoder Representations from Transformers* (BERT) Sebagai Model Representasi Teks

Terdapat dua pendekatan utama dalam pemanfaatan model BERT, yaitu *feature-based* dan *fine-tuning*. Pada pendekatan *feature-based*, BERT berfungsi sebagai *feature extractor* tanpa mengubah parameter internalnya. Pada pendekatan *fine-tuning*, lapisan keluaran khusus ditambahkan dan seluruh bobot dilatih ulang secara

end-to-end.

Pada penelitian ini, BERT digunakan melalui pendekatan *feature-based* dengan bobot yang dibekukan sepenuhnya (*frozen*). Representasi yang dihasilkan kemudian digunakan sebagai *input* untuk *autoencoder* dan pengelompokan topik menggunakan DEC. Contoh representasi teks yang dihasilkan BERT ditunjukkan pada Gambar 2.



Gambar 2: Representasi teks menggunakan BERT ((Devlin *et al.*, 2018); telah diolah kembali).

Sebagai ilustrasi, misalkan kalimat *input* yang digunakan adalah “*pelayanan restoran ini sangat mengecewakan*”. Proses pembentukan representasi teks oleh BERT dilakukan melalui beberapa tahapan berikut.

1. **Tokenisasi.** Kalimat dipecah menjadi token menggunakan *WordPiece tokenizer*, lalu ditambahkan token khusus [CLS] di awal dan [SEP] di akhir:

[[CLS], pelayanan, restoran, ini, sangat, mengecewakan,
[SEP]].

2. **Padding.** Token [PAD] ditambahkan agar panjang seluruh urutan seragam, misalnya hingga 10 token:

[[CLS], pelayanan, restoran, ini, sangat, mengecewakan,
[SEP], [PAD], [PAD], [PAD]].

3. **Attention Mask.** Dibuat *attention mask* untuk membedakan token bermakna dengan token *padding*. Token selain [PAD] diberi nilai 1 dan [PAD] diberi nilai 0:

$$attention\ mask = [1, 1, 1, 1, 1, 1, 1, 0, 0, 0].$$

4. **Embedding.** Setiap token diubah menjadi vektor numerik melalui penjumlahan tiga komponen, yaitu *token embedding*, *segment embedding*, dan *position embedding*.

- *Token embedding* merepresentasikan makna setiap token. Setiap token dipetakan ke indeks dalam kosakata model BERT yang digunakan, kemudian diubah menjadi vektor berdimensi 768 menggunakan matriks $\mathbf{E}_{\text{token}} \in \mathbb{R}^{|\mathcal{V}| \times 768}$, dengan $|\mathcal{V}|$ menyatakan ukuran kosakata model. Sebagai contoh, token pelayanan dipetakan ke indeks $v = 2543$:

$$\mathbf{E}_{\text{token}}[2543] = [0,21, -0,54, 0,83, \dots] \in \mathbb{R}^{768}$$

- *Segment embedding* digunakan untuk membedakan asal kalimat suatu token. Karena penelitian ini hanya menggunakan satu kalimat, seluruh token diberi label Segmen A ($s = 0$):

$$\mathbf{E}_{\text{segment}}[0] = [0,01, 0,01, 0,01, \dots] \in \mathbb{R}^{768}$$

- *Position embedding* memberikan informasi posisi token dalam urutan kalimat. Misalnya, token [CLS] berada pada posisi $p = 0$, sedangkan token pelayanan berada pada posisi $p = 1$:

$$\mathbf{E}_{\text{position}}[1] = [-0,12, 0,34, -0,07, \dots] \in \mathbb{R}^{768}$$

Ketiga komponen tersebut dijumlahkan untuk membentuk representasi akhir token. Untuk token pelayanan dengan $v = 2543$, $s = 0$, dan $p = 1$, diperoleh:

$$\begin{aligned} \mathbf{e}_{\text{pelayanan}} &= \mathbf{E}_{\text{token}}[2543] + \mathbf{E}_{\text{segment}}[0] + \mathbf{E}_{\text{position}}[1] \\ &= [0,21, -0,54, 0,83, \dots] \\ &\quad + [0,01, 0,01, 0,01, \dots] \\ &\quad + [-0,12, 0,34, -0,07, \dots] \\ &= [0,10, -0,19, 0,77, \dots] \in \mathbb{R}^{768} \end{aligned} \tag{3.1}$$

Proses yang sama dilakukan pada seluruh token sehingga diperoleh matriks *embedding* awal $\mathbf{E} \in \mathbb{R}^{10 \times 768}$ sebagai masukan ke lapisan *encoder* BERT.

5. **Encoding.** Matriks $\mathbf{E}$ diproses melalui 12 lapisan *encoder*. Pada setiap lapisan, mekanisme *multi-head self-attention* menghitung bobot perhatian antar token:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

dengan $d_k = 64$ untuk setiap *head* ($768/12 = 64$), lalu diproses oleh *feed-forward network* dengan koneksi residu dan normalisasi:

$$\mathbf{H}^{(l)} = \text{LayerNorm}\left(\mathbf{H}^{(l-1)} + \text{FFN}\left(\text{MultiHead}\left(\mathbf{H}^{(l-1)}\right)\right)\right)$$

Setelah 12 lapisan, diambil vektor token [CLS] dari keluaran lapisan terakhir sebagai representasi dokumen:

$$\begin{aligned}\mathbf{f} &= \mathbf{H}_{[0,:]}^{(12)} \\ &= [0,31, -0,42, 0,88, \dots] \in \mathbb{R}^{768}\end{aligned}\tag{3.2}$$

Vektor $\mathbf{f}$ ini digunakan sebagai masukan *autoencoder* pada tahap berikutnya, dengan seluruh bobot BERT dibekukan selama pelatihan.

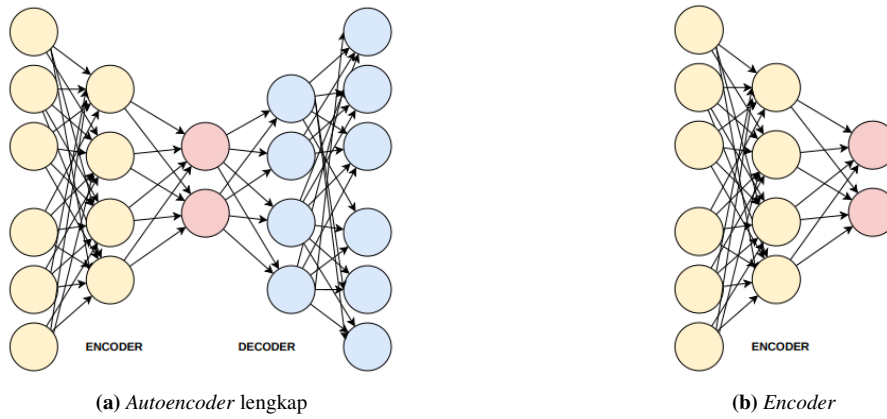
3.2 Alur Pelatihan Model *Multitask*

Pelatihan model *multitask* pada penelitian ini dilakukan dalam tiga tahap yang saling terhubung. Tahap pertama adalah melatih *autoencoder* secara *unsupervised* untuk memperoleh representasi laten yang digunakan sebagai bobot awal model. Tahap kedua, algoritma *Deep Embedded Clustering* (DEC) diterapkan pada representasi laten tersebut untuk menentukan inisialisasi *centroid* kluster. Setelah itu, tahap ketiga dilakukan pelatihan JDST-DEC secara *joint*, dengan menggabungkan informasi dari *clustering head* dan *sentiment head* ke dalam satu fungsi *loss* selama proses optimasi.

3.2.1 Implementasi *Autoencoder*

Pelatihan *autoencoder* secara langsung pada representasi berdimensi tinggi dapat menyebabkan representasi laten yang kurang stabil serta meningkatkan risiko *overfitting*. Oleh karena itu, sebelum pelatihan *multitask*, *autoencoder* terlebih dahulu dilatih secara *unsupervised* melalui proses *pretraining* sebagai inisialisasi bobot awal.

Gambar 3 menunjukkan ilustrasi arsitektur yang digunakan, mulai dari *autoencoder* lengkap hingga komponen *encoder* yang dipertahankan setelah *pretraining*.



Gambar 3: Ilustrasi *autoencoder* lengkap (kiri) dan komponen *encoder* setelah *decoder* dilepas (kanan).

Pada tahap *pretraining*, *autoencoder* dilatih secara *unsupervised* untuk merekonstruksi kembali representasi input melalui representasi laten yang dihasilkan *encoder*. Fungsi *loss* yang digunakan adalah *Mean Squared Error* (MSE) antara vektor *input* asli dan hasil rekonstruksinya:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^N \|\mathbf{f}_i - \hat{\mathbf{f}}_i\|^2 \quad (3.3)$$

dengan N adalah jumlah total sampel, $\mathbf{f}_i$ adalah vektor representasi BERT ke- i , dan $\hat{\mathbf{f}}_i$ adalah hasil rekonstruksinya.

Fungsi aktivasi yang digunakan pada lapisan *encoder* dan *decoder* adalah *Rectified Linear Unit* (ReLU), yang dipilih karena kesederhanaannya, efisiensi komputasi, serta kemampuannya mengurangi permasalahan *vanishing gradient* pada jaringan dalam.

Setelah *pretraining* selesai, bobot *encoder* digunakan sebagai inisialisasi untuk pelatihan *multitask*. Pada tahap ini *encoder* dipertahankan dan dioptimalkan bersama kedua *head*, sedangkan *decoder* tidak lagi digunakan. Representasi laten $\mathbf{z} = f_{\theta}(\mathbf{f})$ yang dihasilkan *encoder* digunakan bersama oleh *sentiment head* dan *clustering head*.

3.2.2 Deep Embedded Clustering (DEC)

DEC bertujuan memperbaiki representasi laten dan menyesuaikan posisi *centroid* agar sesuai dengan struktur distribusi data pada ruang laten (Xie, Girshick, dan Farhadi, 2016). Berbeda dengan *hard assignment* yang menetapkan setiap sampel ke satu kluster, DEC menggunakan distribusi probabilistik sehingga setiap sampel memiliki derajat keanggotaan terhadap seluruh kluster.

Proses DEC dimulai dengan inisialisasi *centroid* menggunakan K-Means pada representasi laten awal hasil *pretraining autoencoder*. Distribusi *soft assignment* q_{ij} untuk setiap data $\mathbf{z}_i$ terhadap *centroid* $\boldsymbol{\mu}_j$ kemudian dihitung menggunakan distribusi Student's t :

$$q_{ij} = \frac{\left(1 + \|\mathbf{z}_i - \boldsymbol{\mu}_j\|^2\right)^{-1}}{\sum_{j'=1}^K \left(1 + \|\mathbf{z}_i - \boldsymbol{\mu}_{j'}\|^2\right)^{-1}} \quad (3.4)$$

dengan q_{ij} menyatakan probabilitas sampel i termasuk kluster j , dan K adalah jumlah kluster.

DEC kemudian membentuk distribusi target p_{ij} berdasarkan distribusi *soft assignment* q_{ij} untuk menyesuaikan proses optimasi kluster. Misalkan $f_j = \sum_i q_{ij}$ adalah frekuensi *soft* kluster j , maka:

$$p_{ij} = \frac{q_{ij}^2 / f_j}{\sum_{j'=1}^K q_{ij'}^2 / f_{j'}} \quad (3.5)$$

Pembagian dengan f_j mencegah kluster berukuran besar mendominasi proses optimasi, sedangkan pengkuadratan nilai q_{ij} membuat distribusi target lebih terkonsentrasi pada kluster yang memiliki tingkat kepercayaan lebih tinggi. Optimasi dilakukan dengan meminimalkan *Kullback-Leibler Divergence* antara distribusi target P dan distribusi *soft assignment* Q :

$$\mathcal{L}_{\text{KLD}} = \sum_i \sum_j p_{ij} \log \frac{p_{ij}}{q_{ij}} \quad (3.6)$$

Pada tahap inisialisasi DEC, fungsi *loss* $\mathcal{L}_{\text{KLD}}$ digunakan untuk memperbarui parameter *encoder* dan posisi *centroid* sehingga representasi laten hasil *pretraining autoencoder* semakin sesuai dengan struktur distribusi data sebelum digunakan pada tahap pelatihan *joint*. Selanjutnya, pada setiap *update interval*, label kluster hasil iterasi sebelumnya dibandingkan dengan label kluster terbaru. Pelatihan dihentikan apabila perubahan label kluster (Δ_{label}) berada di bawah batas toleransi τ .

Tabel 1: Algoritma *Deep Embedded Clustering* (DEC)

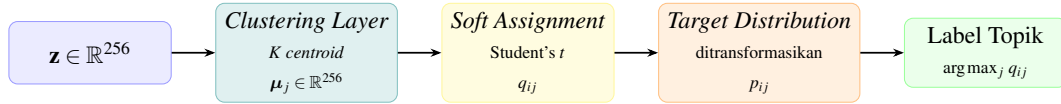
Algoritma <i>Deep Embedded Clustering</i> (DEC)	
INPUT	Representasi laten $\{\mathbf{z}_i\}_{i=1}^N \in \mathbb{R}^{256}$ hasil <i>pretraining autoencoder</i> ; jumlah kluster K ; <i>batch size</i> b ; batas toleransi konvergensi τ ; jumlah iterasi maksimum $T_{\max}$
OUTPUT	Posisi <i>centroid</i> $\{\boldsymbol{\mu}_j\}_{j=1}^K$; label kluster $\{y_{\text{cluster},i}\}$
1.	Inisialisasi <i>centroid</i> $\{\boldsymbol{\mu}_j\}_{j=1}^K$ menggunakan K-Means pada $\{\mathbf{z}_i\}$
2.	FOR iter = 1 hingga $T_{\max}$ 2.1 Simpan label kluster sebelumnya: $y_{\text{prev}} \leftarrow \arg \max_j q_{ij}$ 2.2 Hitung <i>soft assignment</i> q_{ij} menggunakan distribusi Student's t (Persamaan 3.4) 2.3 Hitung distribusi target p_{ij} berdasarkan q_{ij} (Persamaan 3.5) 2.4 Perbarui parameter <i>encoder</i> dan <i>centroid</i> dengan meminimalkan $\mathcal{L}_{\text{KLD}}(P, Q)$ 2.5 Hitung perubahan label: $\Delta_{\text{label}} = \frac{1}{N} \sum_i \mathbf{1}_{[y_{\text{prev},i} \neq \arg \max_j q_{ij}]}$ 2.6 Jika $\Delta_{\text{label}} < \tau$, hentikan pelatihan
3.	END FOR
4.	Kembalikan <i>centroid</i> $\{\boldsymbol{\mu}_j\}$ dan label kluster $\{y_{\text{cluster},i}\}$

3.2.3 Pelatihan *Multitask*: JDST-DEC

Pendekatan *multitask* memungkinkan representasi laten dipelajari secara bersama untuk dua tugas, yaitu klasifikasi sentimen dan klusterisasi topik. Melalui pendekatan ini, proses pembelajaran representasi mempertimbangkan informasi sentimen dan struktur topik secara simultan dalam satu ruang laten, berbeda dengan pendekatan yang mengoptimalkan kedua tugas secara terpisah. Kombinasi DEC dengan *sentiment head* dalam kerangka *multitask* ini selanjutnya disebut JDST-DEC (*Joint Document-level Sentiment-Topic*).

3.2.3.1 *Clustering Head*

Vektor laten $\mathbf{z}$ yang dihasilkan *encoder* diteruskan ke *clustering head* untuk pendeteksian topik. *Clustering head* mengadopsi mekanisme DEC sehingga setiap sampel memiliki derajat keanggotaan probabilistik terhadap seluruh kluster, bukan penetapan ke satu kluster secara langsung. Gambar 4 mengilustrasikan arsitekturnya.

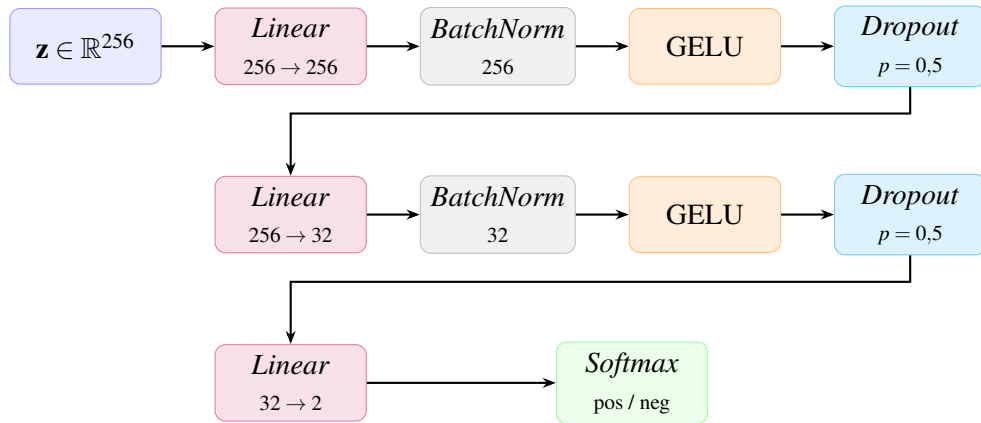


Gambar 4: Arsitektur *clustering head* berbasis DEC.

Clustering head terdiri atas satu lapisan kluster yang menyimpan K *centroid* $\mu_j \in \mathbb{R}^{256}$. Untuk setiap vektor laten $\mathbf{z}_i$, lapisan ini menghitung distribusi *soft assignment* q_{ij} terhadap seluruh *centroid* menggunakan distribusi Student's t sebagaimana didefinisikan pada Persamaan 3.4. Distribusi tersebut kemudian ditransformasikan menjadi distribusi target p_{ij} (Persamaan 3.5), dan label topik akhir diperoleh melalui $\arg \max_j q_{ij}$.

3.2.3.2 Analisis Sentimen Menggunakan *Sentiment Head*

Vektor laten $\mathbf{z} \in \mathbb{R}^{256}$ yang dihasilkan *encoder* diteruskan ke *sentiment head* untuk diklasifikasikan ke salah satu dari dua kelas sentimen. Arsitektur *sentiment head* terdiri atas tiga lapisan linear bertingkat dengan dimensi $256 \rightarrow 256 \rightarrow 32 \rightarrow 2$. Setiap lapisan tersembunyi dilengkapi dengan *batch normalization*, fungsi aktivasi *Gaussian Error Linear Unit* (GELU), dan *dropout* dengan probabilitas $p = 0,5$. Lapisan keluaran berdimensi dua diikuti fungsi *softmax* untuk menghasilkan distribusi probabilitas antarkelas. Gambar 5 mengilustrasikan arsitektur lengkapnya.



Gambar 5: Arsitektur *sentiment head*.

Proses klasifikasi dirumuskan sebagai:

$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{W}_c \mathbf{z} + \mathbf{b}_c) \quad (3.7)$$

dengan $\mathbf{W}_c$ dan $\mathbf{b}_c$ masing-masing adalah bobot dan bias pada lapisan keluaran. Fungsi *softmax* mengubah nilai logit menjadi distribusi probabilitas, di mana setiap elemen $\hat{y}_i \in [0, 1]$ dan $\sum_i \hat{y}_i = 1$.

Selama pelatihan, prediksi $\hat{\mathbf{y}}$ dibandingkan dengan label sebenarnya $\mathbf{y}$ menggunakan fungsi *Categorical Cross-Entropy loss*:

$$\mathcal{L}_{\text{CE}} = - \sum_{i=1}^N \sum_{j=1}^2 y_{ij} \log \hat{y}_{ij} \quad (3.8)$$

dengan N adalah jumlah sampel, y_{ij} adalah label aktual dalam bentuk *one-hot* bernilai 1 jika sampel ke- i termasuk kelas ke- j dan 0 jika tidak, serta $\hat{y}_{ij}$ adalah probabilitas prediksi model.

3.2.3.3 Pelatihan *Joint* dengan *Multitask Loss*

Pada tahap pelatihan *joint*, model dioptimalkan menggunakan kombinasi berbobot antara *clustering loss* dan *sentiment loss*. Fungsi objektif total dirumuskan sebagai:

$$\mathcal{L}_{\text{total}} = \gamma \cdot \mathcal{L}_{\text{KLD}}(P, Q) + \eta \cdot \mathcal{L}_{\text{CE}}(\mathbf{y}, \hat{\mathbf{y}}) \quad (3.9)$$

dengan $\mathcal{L}_{\text{KLD}}$ digunakan untuk mengoptimalkan kualitas struktur kluster dan $\mathcal{L}_{\text{CE}}$ digunakan untuk meningkatkan kemampuan prediksi sentimen. Parameter γ dan η mengontrol kontribusi relatif masing-masing objektif dan ditentukan melalui pencarian *hyperparameter* pada data validasi. Nilai γ yang lebih besar akan lebih menekankan pembentukan struktur kluster, sedangkan nilai η yang lebih besar akan lebih mengarahkan representasi laten terhadap pemisahan kelas sentimen.

Pada proses optimasi, setiap komponen model menerima sinyal gradien yang berbeda sesuai perannya. Pada Persamaan 3.10–3.12, λ menyatakan *learning rate* dan m menyatakan jumlah sampel dalam satu *mini-batch*.

$$\mu_j = \mu_j - \frac{\lambda}{m} \sum_{i=1}^m \frac{\partial \mathcal{L}_{\text{KLD}}}{\partial \mu_j} \quad (3.10)$$

Sementara itu, parameter pada *sentiment head* ($\mathbf{W}_c, \mathbf{b}_c$) hanya diperbarui berdasarkan *sentiment loss*:

$$\mathbf{W}_c = \mathbf{W}_c - \frac{\lambda}{m} \sum_{i=1}^m \frac{\partial \mathcal{L}_{\text{CE}}}{\partial \mathbf{W}_c} \quad (3.11)$$

Berbeda dengan kedua *head*, parameter *encoder* θ menerima gabungan gradien dari kedua fungsi objektif karena representasi laten digunakan secara bersama oleh proses klusterisasi dan klasifikasi. Aturan pembaruan parameter *encoder* dituliskan sebagai:

$$\theta = \theta - \frac{\lambda}{m} \sum_{i=1}^m \left(\gamma \frac{\partial \mathcal{L}_{\text{KLD}}}{\partial \theta} + \eta \frac{\partial \mathcal{L}_{\text{CE}}}{\partial \theta} \right) \quad (3.12)$$

Mekanisme ini serupa dengan IDEC (Guo *et al.*, 2017), namun fungsi rekonstruksi pada IDEC digantikan oleh fungsi *Cross-Entropy loss* sebagai *auxiliary task* untuk analisis sentimen.

3.2.3.4 Algoritma JDST-DEC

Tabel 2 menyajikan prosedur lengkap pelatihan JDST-DEC. Algoritma ini mengasumsikan *encoder* telah diinisialisasi melalui *pretraining autoencoder* dan *centroid* telah diinisialisasi menggunakan K-Means sebagaimana dijelaskan pada Subab 3.2.2.

Tabel 2: Algoritma JDST-DEC (*Joint Document-level Sentiment-Topic*)

Algoritma JDST-DEC	
INPUT	Himpunan vektor BERT $\{\mathbf{f}_i\}_{i=1}^N \in \mathbb{R}^{768}$; label sentimen $\{y_{\text{sentiment},i}\}$; <i>encoder</i> f_θ dan <i>centroid</i> $\{\mu_j\}$ hasil inisialisasi DEC; parameter bobot <i>loss</i> γ, η ; <i>batch size</i> b ; batas toleransi τ ; T_{max}
OUTPUT	Representasi laten $\{\mathbf{z}_i\}$; prediksi label klaster $\{y_{\text{cluster},i}\}$; prediksi probabilitas sentimen
1.	Hitung $q_{ij}^{(0)}$ dari <i>centroid</i> inisialisasi DEC; tetapkan $y_{\text{cluster},i} \leftarrow \arg \max_j q_{ij}^{(0)}$
2.	FOR iter = 1 hingga T_{max} 2.1 Simpan label klaster sebelumnya: $y_{\text{prev}} \leftarrow y_{\text{cluster}}$ 2.2 Proyeksikan data ke ruang laten: $\mathbf{z}_i = f_\theta(\mathbf{f}_i)$ 2.3 Hitung <i>soft assignment</i> q_{ij} menggunakan distribusi Student's t (Persamaan 3.4) 2.4 Perbarui label klaster: $y_{\text{cluster},i} \leftarrow \arg \max_j q_{ij}$ 2.5 Hitung distribusi target p_{ij} berdasarkan q_{ij} (Persamaan 3.5) 2.6 Hitung prediksi sentimen: $\hat{\mathbf{y}}_i = \text{softmax}(\mathbf{W}_c \mathbf{z}_i + \mathbf{b}_c)$ 2.7 Hitung <i>joint loss</i>: $\mathcal{L}_{\text{total}} = \gamma \cdot \mathcal{L}_{\text{KLD}}(P, Q) + \eta \cdot \mathcal{L}_{\text{CE}}(\mathbf{y}, \hat{\mathbf{y}})$ 2.8 Perbarui parameter <i>encoder</i> menggunakan gradien $\mathcal{L}_{\text{total}}$, parameter <i>centroid</i> menggunakan $\mathcal{L}_{\text{KLD}}$, serta parameter sentiment head menggunakan $\mathcal{L}_{\text{CE}}$ 2.9 Hitung perubahan label: $\Delta_{\text{label}} = \frac{1}{N} \sum_i \mathbf{1}_{[y_{\text{prev},i} \neq y_{\text{cluster},i}]}$ 2.10 Jika $\Delta_{\text{label}} < \tau$, hentikan pelatihan
3.	END FOR
4.	Simpan bobot terbaik model berdasarkan metrik validasi

3.3 Interpretasi Model dengan Analisis Atribusi Token

Interpretasi model dilakukan menggunakan metode *post-hoc occlusion-based token attribution* pada level token. Analisis ini diterapkan setelah proses pelatihan selesai menggunakan bobot model terbaik untuk mengidentifikasi kontribusi setiap token terhadap prediksi sentimen serta mengamati pola linguistik pada masing-masing klaster.

Meskipun *Integrated Gradients* dibahas pada tinjauan pustaka sebagai salah satu pendekatan interpretabilitas berbasis gradien, metode tersebut tidak diterapkan pada penelitian ini karena interpretasi difokuskan pada analisis perubahan prediksi akibat manipulasi token secara langsung pada tingkat masukan. Pendekatan ini dipilih untuk mengamati kontribusi masing-masing token terhadap hasil prediksi setelah proses ekstraksi representasi teks selesai dilakukan sehingga merefleksikan pengaruh setiap token terhadap keputusan model pada tingkat masukan. Oleh karena itu, penelitian ini menggunakan metode *occlusion-based token attribution* sebagai pendekatan interpretabilitas.

Teks *input* $\mathbf{x}$ diproses menggunakan BERT untuk menghasilkan representasi dokumen, yang selanjutnya diproyeksikan ke ruang laten melalui *encoder* dan diteruskan ke *sentiment head* guna memperoleh probabilitas *baseline* $f_c(\mathbf{x})$ terhadap kelas c . Selanjutnya, setiap token x_i dalam teks, kecuali token spesial [CLS], [SEP], dan [PAD], digantikan dengan token [MASK] sehingga menghasilkan varian teks $\mathbf{x}_{\setminus i}$. Varian tersebut kemudian diproses kembali melalui tahapan yang sama untuk memperoleh probabilitas prediksi yang diperbarui, yaitu $f_c(\mathbf{x}_{\setminus i})$.

Skor atribusi token x_i terhadap kelas c dihitung sebagai selisih antara probabilitas prediksi sebelum dan sesudah token dimask:

$$\text{Attr}(x_i, c) = f_c(\mathbf{x}) - f_c(\mathbf{x}_{\setminus i}) \quad (3.13)$$

Nilai atribusi positif menunjukkan bahwa token berkontribusi terhadap prediksi kelas c , sedangkan nilai negatif menunjukkan kontribusi yang berlawanan arah. Nilai yang mendekati nol mengindikasikan bahwa token memberikan pengaruh yang relatif kecil terhadap keputusan model.

Untuk memperoleh interpretasi pada level topik, skor atribusi diagregasi berdasarkan hasil klasterisasi. Misalkan $C_k = \{i : y_{\text{cluster}, i} = k\}$ merupakan himpunan sampel yang termasuk ke dalam klaster ke- k . Rata-rata skor atribusi token w pada klaster k didefinisikan sebagai:

$$\overline{\text{Attr}}(w, k, c) = \frac{1}{|C_k|} \sum_{i \in C_k} \text{Attr}(x_i^{(w)}, c) \quad (3.14)$$

dengan $x_i^{(w)}$ menyatakan posisi token w pada sampel ke- i dan $|C_k|$ menyatakan jumlah sampel dalam kluster k .

Untuk meningkatkan reliabilitas interpretasi, hanya token yang muncul minimal pada dua sampel dalam kluster yang sama yang diikutsertakan dalam proses agregasi:

$$\mathcal{V}_k = \left\{ w \mid \sum_{i \in C_k} \mathbf{1}[w \in \mathbf{x}_i] \geq 2 \right\} \quad (3.15)$$

Token dalam $\mathcal{V}_k$ kemudian diurutkan berdasarkan nilai absolut skor atribusi rata-rata untuk memperoleh daftar sebanyak R token paling berpengaruh pada kluster ke- k :

$$\text{TopTokens}(k, c) = \arg \max_{w \in \mathcal{V}_k} \text{top-}R \left| \overline{\text{Attr}}(w, k, c) \right| \quad (3.16)$$

Dalam implementasinya, kosakata yang dievaluasi dibatasi pada *top-N* kata berdasarkan nilai TF-IDF pada masing-masing kluster, sehingga perhitungan atribusi difokuskan pada kata-kata yang paling merepresentasikan karakteristik tiap kluster.

3.4 Evaluasi Topik

Evaluasi kualitas topik dilakukan menggunakan tiga metrik kuantitatif, yaitu *topic coherence*, *topic diversity*, dan *topic coverage*. Ketiga metrik dihitung langsung dari teks asli dan hasil pengelompokan kluster tanpa bergantung pada model eksternal.

3.4.1 Topic Coherence Berbasis NPMI

Topic coherence mengukur sejauh mana kata-kata teratas suatu topik sering muncul bersama dalam korpus. Metrik yang digunakan adalah *Normalized Pointwise Mutual Information* (NPMI) untuk mengukur ko-okurens antar pasangan kata dalam kluster yang sama.

Misalkan $\mathcal{D}_k$ adalah himpunan teks dalam kluster k dan $\{w_1, w_2, \dots, w_N\}$ adalah N kata teratas dari kluster tersebut berdasarkan frekuensi kemunculan setelah pemfilteran *stop words*. Untuk setiap pasangan kata (w_i, w_j) dengan $i < j$, probabilitas kemunculan masing-masing kata dan ko-okurensnya didefinisikan sebagai:

$$p(w_i) = \frac{|\{d \in \mathcal{D}_k : w_i \in d\}|}{|\mathcal{D}_k|}, \quad (3.17)$$

$$p(w_j) = \frac{|\{d \in \mathcal{D}_k : w_j \in d\}|}{|\mathcal{D}_k|}, \quad (3.18)$$

$$p(w_i, w_j) = \frac{|\{d \in \mathcal{D}_k : w_i \in d \text{ dan } w_j \in d\}|}{|\mathcal{D}_k|}. \quad (3.19)$$

Nilai *Pointwise Mutual Information* (PMI) antara dua kata dihitung sebagai:

$$\text{PMI}(w_i, w_j) = \log \frac{p(w_i, w_j) + \epsilon}{p(w_i) \cdot p(w_j) + \epsilon} \quad (3.20)$$

dengan $\epsilon = 10^{-10}$ untuk menghindari pembagian dengan nol. Selanjutnya, PMI dinormalisasi menggunakan probabilitas ko-okurens sebagai pembagi:

$$\text{NPMI}(w_i, w_j) = \frac{\text{PMI}(w_i, w_j)}{-\log(p(w_i, w_j) + \epsilon)} \quad (3.21)$$

Nilai NPMI berada pada rentang $[-1, 1]$: mendekati 1 berarti ko-okurens tinggi, 0 berarti independen, dan negatif berarti kata-kata jarang muncul bersama. Skor *topic coherence* untuk kluster k adalah rata-rata NPMI atas semua pasangan kata teratas:

$$\text{TC}_k = \frac{1}{\binom{N}{2}} \sum_{j=2}^N \sum_{i=1}^{j-1} \text{NPMI}(w_i, w_j) \quad (3.22)$$

Skor koherensi keseluruhan diperoleh dengan merata-ratakan TC_k di seluruh kluster:

$$\overline{\text{TC}} = \frac{1}{K} \sum_{k=1}^K \text{TC}_k \quad (3.23)$$

Metrik ini hanya dihitung untuk kluster dengan setidaknya dua kata teratas yang unik; kluster yang tidak memenuhi syarat diberi nilai koherensi 0.

3.4.2 *Topic Diversity*

Topic diversity mengukur seberapa berbeda kosakata antarkluster. Misalkan $\mathcal{W}_k$ adalah himpunan N kata teratas dari kluster k , himpunan kata unik di seluruh kluster $\mathcal{U} = \bigcup_{k=1}^K \mathcal{W}_k$, dan total kata termasuk duplikat adalah $\sum_{k=1}^K |\mathcal{W}_k|$. *Topic diversity* didefinisikan sebagai:

$$TD = \frac{|\mathcal{U}|}{\sum_{k=1}^K |\mathcal{W}_k|} \quad (3.24)$$

Nilai $TD = 1$ berarti seluruh kata teratas antarkelas unik, sedangkan nilai rendah menunjukkan banyak tumpang tindih kosakata.

BAB 4

HASIL DAN SIMULASI DAN PEMBAHASAN

Bab ini membahas hasil simulasi dan analisis kinerja model *Joint Sentiment-Topic* berbasis *Multi-Task Learning* yang diusulkan. Data yang digunakan berasal dari dataset IMDB dan Yelp yang telah melalui tahap prapengolahan dan pembagian data. Kinerja model dievaluasi dari dua aspek, yaitu analisis sentimen dan pendeteksian topik. Pada analisis sentimen, performa diukur menggunakan metrik *accuracy*, *precision*, *recall*, dan *F1-score*. Pada pendeteksian topik, kualitas topik dinilai menggunakan metrik *Topic Coherence* berbasis NPMI dan *Topic Diversity*. Hasil interpretasi model melalui analisis atribusi token juga dipaparkan untuk menggambarkan karakteristik linguistik tiap kluster topik.

4.1 Spesifikasi Mesin dan Perangkat Lunak

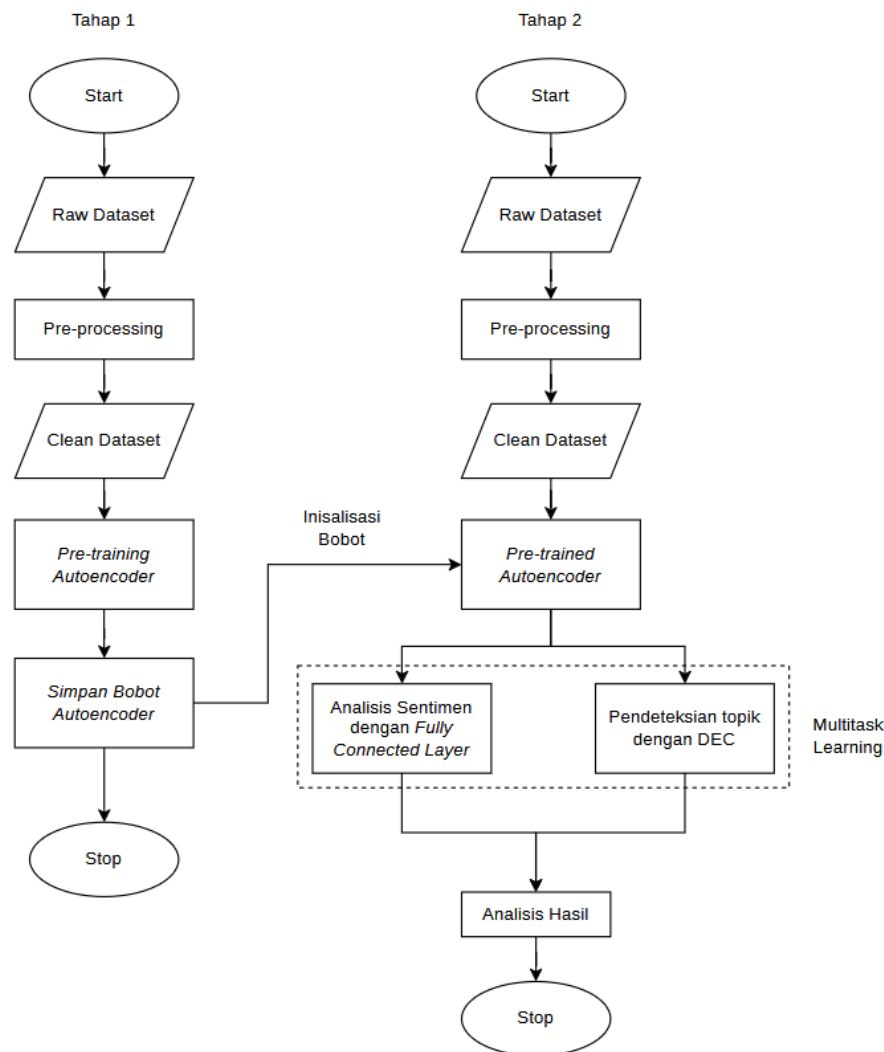
Spesifikasi mesin dan perangkat lunak yang digunakan dalam proses simulasi pada penelitian ini dapat dilihat pada Tabel 1.

Tabel 1: Spesifikasi mesin dan perangkat lunak

Mesin	
<i>Central Processing Unit</i> (CPU)	Apple M1
Memory	8 GB
<i>Graphic Processing Unit</i> (GPU)	Apple M1 GPU (8-core)
Perangkat Lunak	
Perangkat Lunak Pemrograman	Google Colaboratory Kaggle Notebooks Visual Studio Code
Bahasa Pemrograman	Python 3.13.3
Pustaka yang Digunakan	- os, re, csv, json, glob - transformers - numpy, pandas - datasets - PyTorch (torch) - tqdm - scikit-learn, scipy - seaborn

4.2 Tahapan Umum Simulasi

Tahapan umum simulasi pada penelitian ini digambarkan dalam diagram alur pada Gambar 1.



Gambar 1: Diagram alur simulasi penelitian.

Proses dimulai dengan pengumpulan data ulasan yang kemudian melewati tahap prapengolahan untuk menghasilkan data yang bersih. Data tersebut selanjutnya diproses oleh BERT yang digunakan secara *feature-based* dengan bobot yang dibekukan sehingga menghasilkan representasi vektor [CLS] berdimensi 768 untuk setiap dokumen.

Pada tahap pertama, representasi BERT digunakan untuk melatih *autoencoder* guna memperoleh representasi laten berdimensi lebih rendah yang mampu mempertahankan informasi penting dari data masukan. Representasi laten hasil pelatihan ini kemudian digunakan sebagai inisialisasi untuk tahap berikutnya.

Pada tahap kedua, representasi laten dimanfaatkan secara bersamaan oleh dua tugas, yaitu analisis sentimen melalui *sentiment head* dan pendeteksian topik melalui *Deep Embedded Clustering* (DEC) dengan mekanisme pelatihan *joint*. Setelah proses pelatihan selesai, kinerja kedua tugas dievaluasi menggunakan

metrik yang sesuai.

4.3 Data

Subbab ini menjelaskan data yang digunakan dalam penelitian, meliputi sumber data, karakteristik dataset, serta tahapan persiapan data sebelum digunakan dalam pelatihan model. Secara umum, proses pengolahan data dimulai dari pengumpulan dataset ulasan, lalu dilanjutkan dengan prapengolahan teks, pembentukan representasi menggunakan BERT, dan pembagian data.

4.3.1 Pengumpulan Data

Penelitian ini menggunakan dua dataset ulasan berbahasa Inggris, yaitu IMDB Movie Reviews dan Yelp Review Dataset. Kedua dataset dipilih karena umum digunakan sebagai *benchmark* pada penelitian analisis sentimen dan pendeteksian topik, serta keduanya memiliki label sentimen biner.

Dataset IMDB Movie Reviews terdiri dari 50.000 ulasan film yang diunggah oleh (Maas *et al.*, 2011b) dan tersedia secara publik melalui platform Hugging Face dalam format Apache Arrow. Distribusi label sentimen pada dataset ini relatif seimbang antara kelas positif dan negatif, sehingga cocok untuk tugas klasifikasi biner.

Yelp Review Dataset yang digunakan dalam penelitian ini terdiri atas 30.000 ulasan pengguna terhadap berbagai jenis layanan dan bisnis, termasuk restoran dan usaha ritel. Dataset ini tersedia secara publik melalui platform Hugging Face (X. Zhang, J. Zhao, dan LeCun, 2015b) dalam format Apache Arrow dengan distribusi label sentimen yang seimbang. Jumlah data dipilih agar sebanding dengan dataset IMDB yang digunakan pada penelitian ini.

Kedua dataset memiliki label sentimen biner, dengan nilai 1 untuk sentimen positif dan nilai 0 untuk sentimen negatif. Beberapa cuplikan ulasan dari masing-masing dataset ditampilkan pada Tabel 2 dan Tabel 3.

Tabel 2: Cuplikan data ulasan IMDB (Maas *et al.*, 2011b).

No	Text	Label
1	An absolute masterpiece. The storytelling is compelling from start to finish, and the performances are nothing short of extraordinary. Highly recommended to anyone who appreciates great cinema.	1
2	A complete waste of time. The plot makes no sense, the acting is wooden, and the ending is thoroughly unsatisfying. Save yourself two hours and skip this one.	0
⋮	⋮	⋮
50.000	The film captures a quiet kind of grief beautifully. Not much happens on the surface, but every scene carries emotional weight. A slow burn that rewards patient viewers.	1

Tabel 3: Cuplikan data ulasan Yelp (X. Zhang, J. Zhao, dan LeCun, 2015b).

No	Text	Label
1	Hands down the best ramen I have had outside of Japan. The broth is rich and complex, the noodles perfectly chewy. The staff were attentive without being intrusive. Will absolutely return.	1
2	Ordered delivery and the food arrived nearly an hour late and completely cold. When I called to complain, the staff were dismissive. Would not order from here again.	0
⋮	⋮	⋮
30.000	Decent enough for a quick lunch but nothing memorable. The portions are on the smaller side for the price, and the ambiance is a bit too noisy to hold a conversation comfortably.	0

4.3.2 Prapengolahan Data

Tahap prapengolahan bertujuan membersihkan teks dari elemen yang tidak mengandung informasi bahasa sebelum diproses oleh BERT. Karena BERT merupakan model berbasis *transformer* yang menangkap hubungan kontekstual antar-token, penelitian ini menerapkan prapengolahan yang minimalis. Tahapan seperti *stemming*, penghapusan *stop words*, dan normalisasi tanda baca tidak diterapkan, karena kata-kata fungsional seperti *stop words* masih mengandung informasi kontekstual yang relevan bagi model.

Selain itu, BERT menggunakan tokenisasi berbasis WordPiece yang mampu memecah kata di luar kosakata menjadi unit *subword*, sehingga normalisasi kata secara manual tidak diperlukan. Prapengolahan pada penelitian ini difokuskan

pada pembersihan elemen non-linguistik tanpa menghilangkan kandungan informasi dalam teks. Tahapan yang diterapkan adalah sebagai berikut.

1. *Casefolding*, yaitu mengubah seluruh teks menjadi huruf kecil untuk menghindari perbedaan representasi akibat variasi kapitalisasi.
2. Penghapusan tag HTML, yaitu menghapus elemen HTML seperti `<br />` dan `<b>` yang tidak memiliki kandungan bahasa.
3. Penghapusan URL, yaitu menghapus tautan dengan pola `http://...` atau `www....` karena tidak membawa informasi sentimen maupun topik.
4. Penghapusan karakter non-ASCII, yaitu menghapus emoji, simbol unicode, dan karakter khusus yang dapat mengganggu proses tokenisasi.
5. Normalisasi *whitespace*, yaitu menyeragamkan spasi berlebih, karakter tab, dan baris baru menjadi satu spasi tunggal.

Hasil dari tahap ini adalah teks yang bersih dari elemen non-linguistik namun tetap mempertahankan struktur kalimat aslinya. Perbandingan teks sebelum dan sesudah prapengolahan dapat dilihat pada Tabel 4.

Tabel 4: Contoh hasil prapengolahan pada dataset IMDB dan Yelp.

Sebelum Prapengolahan	Sesudah Prapengolahan
<i>IMDB</i>	
An absolute MASTERPIECE. The storytelling is compelling. <code>
</code> Highly recommended to anyone who appreciates great cinema!	an absolute masterpiece. the storytelling is compelling. highly recommended to anyone who appreciates great cinema!
A complete waste of time. <code>
</code> Read the full review at http://www.filmcritic.org/reviews/2024	a complete waste of time. read the full review at
<i>Yelp</i>	
Hands down the BEST ramen outside of Japan :) The broth is incredible! Check the menu at www.ramenhouse.com	hands down the best ramen outside of japan the broth is incredible! check the menu at
Ordered delivery & the food arrived cold after 1hr. Staff were rude when I called :(Would NOT recommend.	ordered delivery the food arrived cold after 1hr. staff were rude when i called would not recommend.

Tabel 4 menunjukkan bahwa perubahan yang dilakukan terbatas pada penghapusan elemen non-linguistik dan penyeragaman format teks, bukan penyederhanaan isi kalimat. Dengan demikian, kandungan kontekstual dalam ulasan tetap terjaga untuk tahap ekstraksi fitur menggunakan BERT.

4.3.3 Representasi Teks

Setelah prapengolahan, teks diubah menjadi representasi numerik melalui dua tahap, yaitu tokenisasi dan ekstraksi fitur menggunakan BERT. Representasi yang dihasilkan menjadi masukan bagi *autoencoder* pada tahap pelatihan berikutnya.

Tokenisasi dilakukan menggunakan tokenizer WordPiece dari model BERT *pre-trained*. Setiap teks dikonversi menjadi urutan *token ID* dengan panjang maksimum $t = 128$ token. Teks yang melebihi batas ini akan dipotong (*truncation*), sedangkan teks yang lebih pendek akan ditambahkan token [PAD] bernilai 0 agar panjang urutan seragam. Contoh hasil tokenisasi ditampilkan pada Tabel 5.

Tabel 5: Contoh proses tokenisasi menggunakan WordPiece BERT.

Teks (hasil prapengolahan)	Hasil tokenisasi (<i>token ID</i>)
an absolute masterpiece the storytelling is compelling	[101, 2019, 7927, 4902, 1996, 4129, 2003, 17075, 102, 0, ..., 0]

Urutan *token ID* tersebut selanjutnya diproses oleh BERT untuk menghasilkan representasi kontekstual. Model yang digunakan adalah BERT *pre-trained* dengan *hidden size* 768 dan bobot yang dibekukan sepenuhnya (*frozen*). Dari keluaran BERT berupa matriks berukuran $t \times 768$, diambil baris pertama yang bersesuaian dengan token [CLS] sebagai representasi dokumen:

$$\mathbf{f} = \mathbf{h}_1 \in \mathbb{R}^{768} \quad (4.1)$$

dengan $\mathbf{h}_1$ adalah vektor keluaran BERT pada posisi token [CLS]. Vektor $\mathbf{f}$ ini merepresentasikan konteks global dokumen dan digunakan sebagai masukan bagi *autoencoder* pada tahap berikutnya. Contoh ilustrasi perubahan representasi dari *token ID* ke vektor BERT ditampilkan pada Tabel 6.

Tabel 6: Contoh representasi vektor [CLS] hasil ekstraksi BERT.

<i>Token ID</i>	Representasi [CLS] $\mathbf{f} \in \mathbb{R}^{768}$
[101, 2019, 7927, 4902, ...]	[0.31, -0.17, 0.08, ..., -0.22]

4.3.4 Pembagian Data

Setelah prapengolahan, data dari kedua dataset dibagi menjadi data latih (*training set*) dan data uji (*test set*) dengan proporsi 80% untuk data latih dan 20% untuk

data uji, mengacu pada skema yang digunakan oleh (Gowandi, Murfi, dan Nurrohmah, 2021). Pembagian ini dilakukan agar proses pelatihan dan evaluasi model dapat dilakukan secara terpisah.

Data latih digunakan sebagai masukan dalam proses optimasi parameter model, baik pada komponen analisis sentimen maupun pendeteksian topik. Data uji digunakan untuk mengukur performa model pada data yang belum pernah dilihat selama pelatihan.

4.4 Implementasi Model

Implementasi pada penelitian ini mencakup tiga model, yaitu JDST-DEC sebagai model yang diusulkan, NN-BERT sebagai model pembanding untuk analisis sentimen, dan BERTopic sebagai model pembanding untuk pendeteksian topik. JDST-DEC dilatih melalui dua tahap, yaitu pra-pelatihan *autoencoder* dan *joint training*. Seluruh model diimplementasikan menggunakan Python dengan pustaka sebagaimana tercantum pada Tabel 1.

4.4.1 Konfigurasi JDST-DEC

Konfigurasi JDST-DEC mencakup parameter pada tahap pra-pelatihan *autoencoder* dan pembentukan kluster menggunakan DEC. Konfigurasi masing-masing komponen disajikan pada Tabel 7 dan Tabel 8.

Tabel 7: Konfigurasi pra-pelatihan *autoencoder*

Parameter	Nilai
Arsitektur	FC, simetris
Dimensi lapisan	768→2022→2022→256
Dimensi laten	256
Fungsi aktivasi	ReLU
Fungsi <i>loss</i>	MSE
<i>Optimizer</i>	Adam
<i>Learning rate</i>	2×10^{-3}
<i>Batch size</i>	128
Epoch	30

Tabel 8: Konfigurasi komponen DEC

Parameter	Nilai
Lapisan klaster	1
Dimensi ruang klaster	256
Jumlah klaster (K)	30, 50, 80
<i>Soft assignment</i>	Student- t
Derajat kebebasan (v)	1
<i>Loss clustering</i>	KL Divergence
Inisialisasi centroid	k -means

Dimensi masukan sebesar 768 mengikuti ukuran vektor [CLS] yang dihasilkan oleh BERT. Representasi tersebut kemudian direduksi ke ruang laten berdimensi 256 untuk menghasilkan fitur yang lebih ringkas sebelum proses klasterisasi. Pada tahap DEC, centroid klaster diinisialisasi menggunakan metode k -means, sedangkan jumlah klaster (K) dievaluasi pada tiga skenario, yaitu 30, 50, dan 80 klaster, guna menganalisis pengaruh jumlah topik terhadap kualitas hasil klasterisasi.

4.4.2 Konfigurasi *Joint Training*

Setelah tahap pra-pelatihan, JDST-DEC dioptimasi melalui proses *joint training* untuk menjalankan dua tugas secara simultan, yaitu klasifikasi sentimen dan pembentukan klaster topik. Fungsi objektif yang digunakan didefinisikan sebagai

$$\mathcal{L} = \eta \mathcal{L}_{\text{sent}} + \gamma \mathcal{L}_{\text{clust}}.$$

Konfigurasi pelatihan yang digunakan pada tahap *joint training* ditunjukkan pada Tabel 9.

Tabel 9: Konfigurasi *joint training* JDST-DEC

Parameter	Nilai
<i>Learning rate</i>	1×10^{-3}
<i>Batch size</i>	256
Iterasi maksimum	100
<i>Update interval</i>	10
Toleransi (τ)	10^{-4}
Bobot sentimen (η)	0,5
Bobot klaster (γ)	0,5
<i>Optimizer</i>	SGD
Jumlah topik (K)	30, 50, dan 80
<i>Seed</i>	42

Nilai η dan γ ditetapkan sebesar 0,5 untuk memberikan kontribusi yang seimbang antara tugas klasifikasi sentimen dan klasterisasi topik selama proses pelatihan. Sementara itu, jumlah topik (K) dievaluasi pada tiga skenario, yaitu 30, 50, dan 80 topik, untuk mengidentifikasi konfigurasi yang memberikan hasil terbaik.

4.4.3 Konfigurasi Model Pembandingan

Penelitian ini menggunakan dua model pembandingan untuk mengevaluasi kinerja JDST-DEC pada masing-masing tugas. NN-BERT digunakan sebagai model pembandingan untuk analisis sentimen karena memanfaatkan representasi BERT yang sama tanpa mekanisme pembelajaran multitugas. Sementara itu, BERTopic digunakan sebagai pembandingan pada pendeteksian topik karena merupakan salah satu pendekatan berbasis *transformer* yang banyak digunakan untuk *topic modeling*.

4.4.3.1 NN-BERT

NN-BERT digunakan sebagai model dasar (*baseline*) untuk tugas analisis sentimen. Representasi [CLS] yang dihasilkan oleh BERT digunakan sebagai masukan bagi lapisan klasifikasi *fully connected* untuk memprediksi kelas sentimen. Konfigurasi model NN-BERT ditunjukkan pada Tabel 10.

Tabel 10: Konfigurasi model NN-BERT

Parameter	Nilai
<i>Encoder</i>	BERT, <i>frozen</i>
<i>Dimensi input</i>	768 ([CLS])
<i>Classifier</i>	FC, 1 lapisan
<i>Dimensi tersembunyi</i>	256
<i>Fungsi aktivasi</i>	ReLU
<i>Dimensi output</i>	2 kelas
<i>Fungsi loss</i>	<i>Cross-Entropy</i>
<i>Optimizer</i>	Adam
<i>Learning rate</i>	5×10^{-5}
<i>Batch size</i>	256
<i>Epoch</i>	10

4.4.3.2 BERTopic

BERTopic digunakan sebagai model pembanding pada tugas pendeteksian topik. Model ini memanfaatkan representasi semantik berbasis BERT untuk memperoleh representasi dokumen, kemudian dilanjutkan dengan reduksi dimensi menggunakan UMAP sebelum proses pembentukan topik dilakukan melalui algoritma klusterisasi. Pada penelitian ini digunakan dua pendekatan klusterisasi, yaitu HDBSCAN dan K-Means, untuk mengevaluasi pengaruh metode pembentukan kluster terhadap kualitas topik yang dihasilkan. Ringkasan konfigurasi yang digunakan ditunjukkan pada Tabel 11.

Tabel 11: Konfigurasi model BERTopic

Komponen	Parameter	Nilai
<i>Embedding</i>	Model	bert-base-uncased
	Dimensi	768
UMAP	n_neighbors	15
	n_components	5
	min_dist	0,0
	Metrik	Kosinus
HDBSCAN	min_cluster_size	10
	min_samples	5
	Metrik	Euclidean
	prediction_data	True
K-Means	n_clusters	30, 50, 80
Topik	Jumlah topik (K)	30, 50, 80
	Metode reduksi	nr_topics
	Representasi topik	c-TF-IDF

Untuk menjaga konsistensi dengan JDST-DEC, jumlah topik yang dievaluasi pada BERTopic ditetapkan sebesar 30, 50, dan 80 topik. Pengaturan ini dilakukan agar perbandingan antar metode dilakukan pada ruang solusi yang setara, sehingga perbedaan hasil yang diperoleh lebih merefleksikan kemampuan model dalam membentuk representasi topik dibandingkan pengaruh jumlah topik yang digunakan.

4.5 Analisis Kinerja Model

Bagian ini menyajikan hasil evaluasi model pada dua tahapan utama, yaitu pra-pelatihan (*pretraining*) *autoencoder* dan proses *joint training*. Evaluasi dilakukan pada dataset IMDB dan Yelp untuk melihat konsistensi performa model pada domain ulasan yang berbeda. Tahap pra-pelatihan digunakan untuk mengevaluasi kemampuan model dalam mempelajari representasi laten sebelum proses pembelajaran multitugas, sedangkan tahap *joint training* digunakan untuk menilai performa akhir model pada tugas analisis sentimen dan pendeteksian topik.

4.5.1 Analisis Kinerja Pra-pelatihan

Tahap pra-pelatihan dilakukan untuk membentuk representasi laten awal melalui mekanisme rekonstruksi pada *autoencoder*. Representasi ini selanjutnya

digunakan sebagai inisialisasi sebelum proses *joint training*, sehingga kualitas hasil rekonstruksi menjadi penting karena dapat memengaruhi proses pembelajaran pada tahap berikutnya.

Evaluasi kinerja pra-pelatihan dilakukan menggunakan *Mean Squared Error* (MSE) pada data latih dan data validasi. Nilai MSE digunakan untuk mengukur selisih antara representasi masukan dan hasil rekonstruksi yang dihasilkan model, di mana nilai yang lebih kecil menunjukkan kemampuan rekonstruksi yang lebih baik. Perbandingan antara MSE data latih dan data validasi juga digunakan untuk mengamati kestabilan proses pelatihan serta indikasi terjadinya *overfitting*.

Hasil pra-pelatihan pada dataset IMDB dan Yelp disajikan pada Tabel 12 dan Tabel 13.

Tabel 12: Perkembangan *loss* pra-pelatihan pada dataset IMDB

Epoch	<i>Train Loss</i>	<i>Validation Loss</i>
1	$2,370 \times 10^{-3}$	$2,544 \times 10^{-3}$
5	$3,300 \times 10^{-4}$	$3,610 \times 10^{-4}$
10	$3,780 \times 10^{-4}$	$4,120 \times 10^{-4}$
15	$1,570 \times 10^{-4}$	$1,760 \times 10^{-4}$
20	$1,370 \times 10^{-4}$	$1,550 \times 10^{-4}$
25	$1,440 \times 10^{-4}$	$1,620 \times 10^{-4}$
30	$1,410 \times 10^{-4}$	$1,590 \times 10^{-4}$

Pra-pelatihan *autoencoder* pada dataset IMDB menunjukkan penurunan MSE yang signifikan dari epoch pertama hingga epoch ke-20, kemudian cenderung stabil pada kisaran $1,4 \times 10^{-4}$. *Validation loss* konsisten berada sedikit di atas *train loss*, mengindikasikan bahwa model tidak mengalami *overfitting*.

Tabel 13: Perkembangan *loss* pra-pelatihan pada dataset Yelp

Epoch	<i>Train Loss</i>	<i>Validation Loss</i>
1	$3,790 \times 10^{-3}$	$4,062 \times 10^{-3}$
5	$4,180 \times 10^{-4}$	$4,540 \times 10^{-4}$
10	$2,780 \times 10^{-4}$	$3,040 \times 10^{-4}$
15	$1,990 \times 10^{-4}$	$2,200 \times 10^{-4}$
20	$1,670 \times 10^{-4}$	$1,860 \times 10^{-4}$
25	$1,830 \times 10^{-4}$	$2,030 \times 10^{-4}$
30	$1,550 \times 10^{-4}$	$1,730 \times 10^{-4}$

Pola serupa terjadi pada dataset Yelp, dengan MSE turun tajam pada epoch-

epoch awal dan mulai konvergen setelah epoch ke-20. Nilai akhir *train loss* sebesar $1,550 \times 10^{-4}$ menunjukkan bahwa *autoencoder* mampu merekonstruksi representasi BERT dengan baik pada kedua dataset.

4.5.2 Analisis Kinerja *Joint Training*

Kinerja *joint training* dievaluasi pada dua aspek utama, yaitu klasifikasi sentimen dan pemodelan topik. Evaluasi membandingkan JDST-DEC dengan beberapa model pembanding, yaitu NN-BERT, MTL GUI (Gui *et al.*, 2022), BERTopic-HDBSCAN, BERTopic-KMeans, dan MambaQuin. Khusus pada dataset Yelp, hasil MambaQuin tidak sepenuhnya dapat dibandingkan secara langsung karena menggunakan proses *sampling* data yang berbeda.

4.5.2.1 Klasifikasi Sentimen

Performa klasifikasi sentimen dievaluasi menggunakan *accuracy* dan F1-score. Model yang tidak memiliki komponen klasifikasi sentimen tidak disertakan dalam evaluasi.

Tabel 14: Perbandingan hasil *Sentiment Classification*

Method	IMDB						Yelp					
	Acc		F1				Acc		F1			
NN-BERT	0.8581		0.8612				0.8664		0.8715			
	$K = 30$		$K = 50$		$K = 80$		$K = 30$		$K = 50$		$K = 80$	
	Acc	F1	Acc	F1	Acc	F1	Acc	F1	Acc	F1	Acc	F1
JDST-DEC	0.8957	0.8963	0.8967	0.8986	0.8941	0.8930	0.8965	0.8944	0.8983	0.9020	0.8935	0.8953
MTL GUI	0.8327	0.8326	0.8317	0.8317	0.8334	0.8334	0.8385	0.8385	0.8380	0.8380	0.8402	0.8401
MambaQuin	0.9085	0.9085	0.9083	0.9083	0.9086	0.9086	0.9433	0.9433	0.9416	0.9416	0.9428	0.9428

Berdasarkan Tabel 14, MambaQuin memperoleh nilai *accuracy* dan F1-score tertinggi pada seluruh konfigurasi. Namun, hasil MambaQuin pada dataset Yelp tidak sepenuhnya dapat dibandingkan secara langsung karena menggunakan proses *sampling* data yang berbeda.

Di antara model yang dievaluasi pada pengaturan eksperimen yang sama, JDST-DEC secara konsisten mengungguli NN-BERT dan MTL GUI. Pada dataset IMDB, JDST-DEC memperoleh F1-score terbaik sebesar 0.8986 pada $K = 50$, lebih tinggi dibandingkan NN-BERT (0.8612) dan MTL GUI (0.8317). Sementara itu, pada dataset Yelp, JDST-DEC mencapai F1-score sebesar 0.9020 pada $K = 50$, melampaui NN-BERT (0.8715) dan MTL GUI (0.8401). Perubahan jumlah topik dari

$K = 30$ hingga $K = 80$ juga hanya memberikan perbedaan performa yang kecil, sehingga menunjukkan bahwa kinerja JDST-DEC relatif stabil terhadap variasi jumlah topik.

4.5.2.2 Pemodelan Topik

Kualitas pemodelan topik dievaluasi menggunakan dua metrik, yaitu *Topic Coherence* (TC-NPMI) dan *Topic Diversity* (TD). TC-NPMI mengukur seberapa bermakna hubungan semantik antar kata dalam satu topik berdasarkan ko-munculan pada korpus, sedangkan TD mengukur proporsi kata unik yang muncul lintas topik. Nilai yang lebih tinggi pada kedua metrik menunjukkan topik yang lebih koheren sekaligus lebih beragam. NN-BERT tidak diikutsertakan karena tidak memiliki komponen pemodelan topik.

Tabel 15: Perbandingan hasil *Topic Modelling*

Method	IMDB						Yelp					
	$K = 30$		$K = 50$		$K = 80$		$K = 30$		$K = 50$		$K = 80$	
	TC-NPMI	TD	TC-NPMI	TD	TC-NPMI	TD	TC-NPMI	TD	TC-NPMI	TD	TC-NPMI	TD
JDST-DEC	0.0803	0.2256	0.0551	0.2100	0.0675	0.2875	0.0788	0.3267	0.0369	0.2280	0.0563	0.3150
BERTopic-HDBSCAN	0.1714	0.7966	0.1823	0.7680	0.1751	0.7625	0.1092	0.7233	0.1666	0.7041	0.1646	0.6671
BERTopic-KMeans	0.1566	0.9567	0.1596	0.9000	0.1874	0.8413	0.1077	0.7800	0.1120	0.6740	0.1465	0.6438
MTL GUI	0.0200	0.5833	0.0108	0.5800	0.0060	0.4913	0.1578	0.6600	0.1086	0.6720	0.1123	0.5000
MambaQuin	0.1589	0.2400	0.1561	0.2040	0.1606	0.1575	0.2557	0.2533	0.2727	0.2060	0.2699	0.1588

Berdasarkan Tabel 15, BERTopic memperoleh nilai *Topic Diversity* (TD) tertinggi pada sebagian besar konfigurasi. Pada dataset IMDB, BERTopic-KMeans menghasilkan TD tertinggi sebesar 0.9567 ($K = 30$), 0.9000 ($K = 50$), dan 0.8413 ($K = 80$). Pada dataset Yelp, BERTopic-KMeans memperoleh TD tertinggi pada $K = 30$ sebesar 0.7800, sedangkan BERTopic-HDBSCAN unggul pada $K = 50$ dan $K = 80$ dengan nilai masing-masing 0.7041 dan 0.6671.

Untuk metrik TC-NPMI, MambaQuin memperoleh nilai tertinggi pada IMDB $K = 30$ sebesar 0.1589, sedangkan BERTopic-HDBSCAN dan BERTopic-KMeans masing-masing unggul pada IMDB $K = 50$ (0.1823) dan $K = 80$ (0.1874). Pada dataset Yelp, MambaQuin memperoleh TC-NPMI tertinggi pada seluruh konfigurasi, yaitu 0.2557, 0.2727, dan 0.2699. Namun, hasil MambaQuin pada dataset Yelp tidak sepenuhnya dapat dibandingkan secara langsung karena menggunakan proses *sampling* data yang berbeda.

4.5.3 Analisis Multiloss

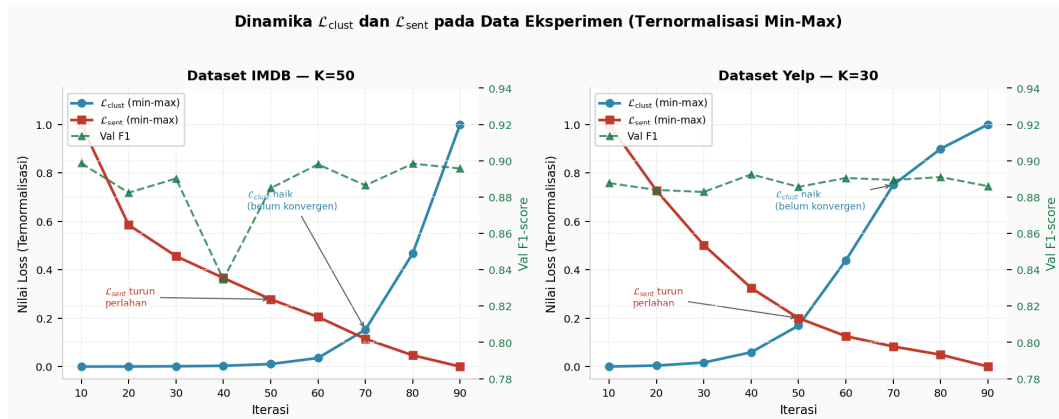
Proses *joint training* pada JDST-DEC mengoptimasi dua komponen *loss* secara simultan, yaitu *clustering loss* berbasis KL Divergence ($\mathcal{L}_{\text{clust}}$) dan *sentiment loss* berbasis *cross-entropy* ($\mathcal{L}_{\text{sent}}$), dengan fungsi objektif:

$$\mathcal{L} = \gamma \mathcal{L}_{\text{clust}} + \eta \mathcal{L}_{\text{sent}}$$

Karena kedua komponen memiliki rentang nilai yang berbeda, $\mathcal{L}_{\text{sent}}$ cenderung memiliki nilai lebih besar dibandingkan $\mathcal{L}_{\text{clust}}$, sehingga analisis dilakukan menggunakan normalisasi *min-max*. Normalisasi ini digunakan agar perubahan kedua komponen dapat diamati pada skala yang sama.

4.5.3.1 Dinamika pada Data Eksperimen

Gambar 2 menunjukkan perkembangan $\mathcal{L}_{\text{clust}}$ dan $\mathcal{L}_{\text{sent}}$ selama proses *joint training* pada salah satu contoh eksperimen yang dijalankan, yaitu IMDB ($K = 50$) dan Yelp ($K = 30$).



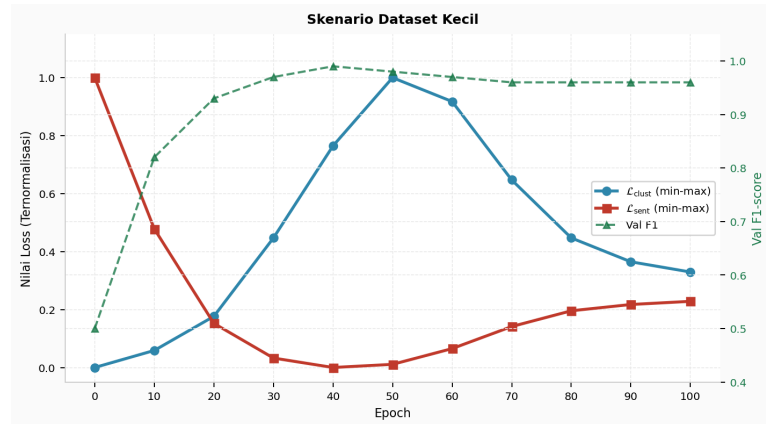
Gambar 2: Dinamika $\mathcal{L}_{\text{clust}}$ dan $\mathcal{L}_{\text{sent}}$ (ternormalisasi *min-max*) selama *joint training* pada dataset IMDB ($K = 50$) dan Yelp ($K = 30$).

Berdasarkan Gambar 2, $\mathcal{L}_{\text{sent}}$ menunjukkan tren menurun sejak iterasi awal. Sebaliknya, $\mathcal{L}_{\text{clust}}$ cenderung meningkat selama proses pelatihan dan belum menunjukkan kecenderungan konvergen hingga iterasi ke-100.

Pola tersebut dapat dikaitkan dengan mekanisme DEC. Pada tahap awal, distribusi *soft assignment* (q) masih mendekati distribusi target (p), sehingga nilai KL Divergence relatif kecil. Seiring proses pelatihan, perubahan distribusi target menyebabkan nilai KL Divergence meningkat. Hasil ini mengindikasikan bahwa proses pembentukan kluster masih berlangsung.

4.5.3.2 Eksperimen Dataset Kecil

Untuk memberikan ilustrasi tambahan mengenai dinamika *multiloss*, dilakukan percobaan pada dataset dengan ukuran lebih kecil (1000 data). Hasilnya ditunjukkan pada Gambar 3.



Gambar 3: Skenario dataset kecil: $\mathcal{L}_{\text{sent}}$ mencapai kondisi stabil lebih dahulu, kemudian $\mathcal{L}_{\text{clust}}$ mulai menurun dan kedua komponen menunjukkan pola yang lebih seimbang.

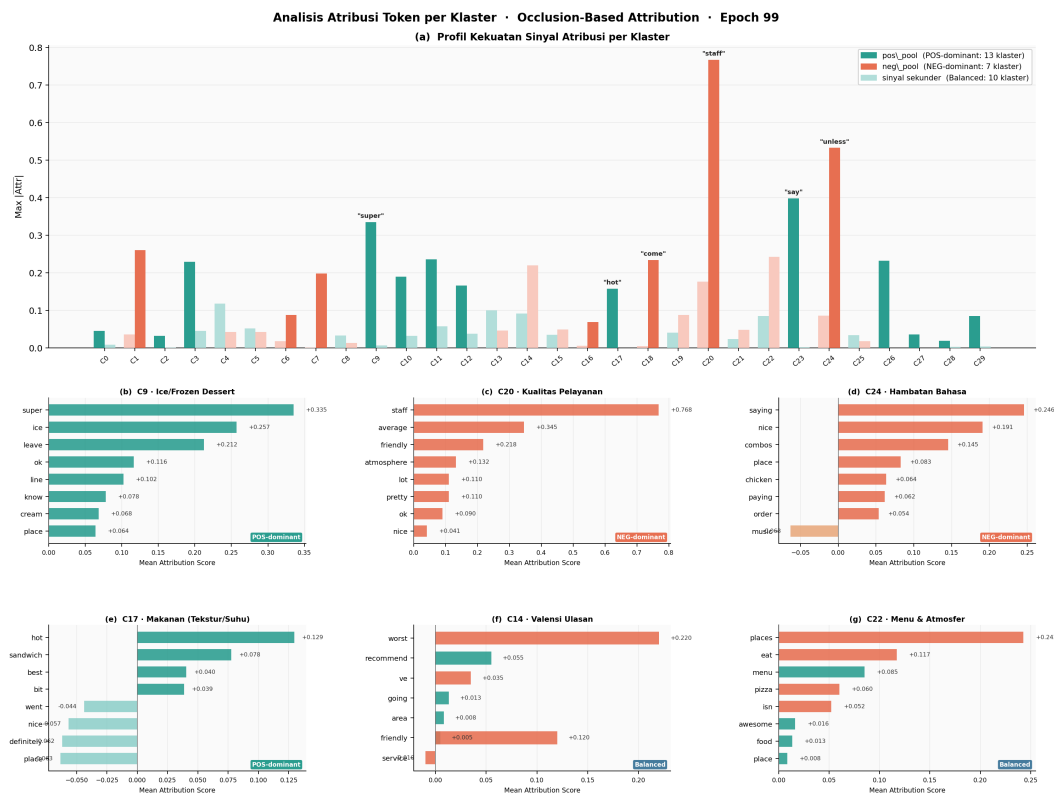
Pada Gambar 3, penurunan $\mathcal{L}_{\text{sent}}$ terjadi lebih cepat dibandingkan $\mathcal{L}_{\text{clust}}$. Setelah laju penurunan $\mathcal{L}_{\text{sent}}$ mulai melambat, perubahan pada komponen klusterisasi menjadi lebih terlihat dan $\mathcal{L}_{\text{clust}}$ mulai mengalami penurunan. Pola ini menunjukkan bahwa ketika proses klasifikasi sentimen telah cukup terbentuk, optimasi mulai diikuti oleh perbaikan pada struktur klaster.

4.6 Analisis Interpretabilitas Model

Analisis interpretabilitas dilakukan menggunakan metode *occlusion-based token attribution* sebagaimana dirumuskan pada Persamaan 3.13–3.16. Evaluasi dilakukan menggunakan model hasil *joint training* pada epoch terakhir (epoch ke-99), bukan model dengan performa evaluasi terbaik. Pemilihan epoch terakhir bertujuan untuk mengamati representasi dan pola atribusi yang terbentuk setelah seluruh proses pelatihan selesai.

Analisis dilakukan pada konfigurasi jumlah klaster sebesar 30 ($K = 30$) menggunakan dataset Yelp. Konfigurasi ini dipilih dari beberapa percobaan jumlah klaster yang digunakan pada penelitian, yaitu $K = 30$, $K = 50$, dan $K = 80$. Analisis interpretabilitas dibatasi pada konfigurasi ini sebagai studi kasus untuk memberikan gambaran perilaku model secara kualitatif. Untuk setiap klaster, dihitung skor atribusi rata-rata token terhadap masing-masing kelas sentimen, yaitu *pos_pool*

dan *neg_pool*. Gambar 4 memperlihatkan ringkasan atribusi token pada seluruh 30 kluster.



Gambar 4: Ringkasan hasil atribusi token pada 30 kluster. Panel (a) menunjukkan nilai atribusi tertinggi pada setiap kluster, sedangkan panel (b)–(g) menampilkan token dengan kontribusi terbesar pada beberapa kluster terpilih berdasarkan skor atribusi rata-rata.

Pada kelompok kluster dengan dominansi positif, beberapa token yang sering muncul sebagai kontributor utama antara lain “*super*”, “*hot*”, “*sandwich*”, dan “*amazing*”. Token-token tersebut menunjukkan bahwa model cenderung memanfaatkan kata yang bersifat evaluatif serta kata yang berkaitan dengan konteks makanan dalam menghasilkan prediksi positif. Namun, temuan ini hanya menunjukkan pola kontribusi token dan belum cukup untuk digunakan sebagai dasar dalam menentukan tema kluster secara pasti.

Pada kluster dengan dominansi negatif, token seperti “*staff*”, “*average*”, dan “*atmosphere*” memiliki kontribusi yang cukup besar terhadap prediksi negatif. Selain itu, ditemukan beberapa token dengan skor negatif pada *neg_pool*, seperti “*unless*” dan “*english*”, yang menunjukkan bahwa pengaruh token terhadap prediksi model tidak selalu sesuai dengan polaritas kata secara umum. Hal ini menunjukkan bahwa model juga mempertimbangkan konteks kemunculan token, bukan hanya keberadaan kata secara terpisah.

Pada kluster dengan distribusi sentimen yang lebih seimbang, ditemukan to-

ken positif dan negatif yang muncul secara bersamaan, seperti “*recommend*” dan “*worst*”. Kondisi ini menunjukkan bahwa beberapa klaster masih memiliki variasi sentimen yang cukup tinggi dan belum sepenuhnya terpisah berdasarkan kecenderungan sentimennya.

Secara keseluruhan, hasil atribusi menunjukkan bahwa model telah menangkap pola penggunaan kata yang relevan terhadap tugas analisis sentimen. Namun, interpretasi pada tingkat klaster masih memiliki keterbatasan karena kontribusi token yang ditemukan belum selalu menghasilkan pemisahan topik maupun sentimen secara konsisten.

BAB 5

PENUTUP

5.1 Kesimpulan

Berdasarkan hasil penelitian yang telah dilakukan, dapat ditarik kesimpulan sebagai berikut.

1. Model berbasis *Fully Neural Network* JST-DEC berhasil dikembangkan untuk menangani klasifikasi sentimen dan pemodelan topik secara simultan dalam satu *framework* terpadu. Arsitektur yang dibangun mengintegrasikan *autoencoder* sebagai komponen reduksi dimensi dan *Deep Embedded Clustering* (DEC) sebagai komponen pemodelan topik, dengan *sentiment head* berbasis *fully connected* untuk tugas klasifikasi sentimen.
2. Pendekatan *Multi-Task Learning* (MTL) pada JDST-DEC mampu meningkatkan performa klasifikasi sentimen dibandingkan pendekatan *single-task*. Model memperoleh F1-score terbaik sebesar 0,8986 pada dataset IMDB dan 0,9020 pada dataset Yelp, lebih tinggi dibandingkan NN-BERT yang memperoleh F1-score sebesar 0,8612 dan 0,8715.
3. Penerapan *autoencoder* berhasil mereduksi dimensi representasi BERT dari 768 menjadi 256 dimensi. Proses pra-pelatihan menunjukkan konvergensi yang stabil pada kedua dataset, dengan nilai MSE akhir sebesar $1,41 \times 10^{-4}$ pada IMDB dan $1,55 \times 10^{-4}$ pada Yelp, tanpa indikasi *overfitting*.
4. JST-DEC mengungguli NN-BERT pada tugas analisis sentimen, namun menghasilkan kualitas topik yang lebih rendah dibandingkan BERTopic. Nilai TC-NPMI tertinggi JST-DEC mencapai 0,0803 pada IMDB dan 0,0788 pada Yelp, sedangkan BERTopic memperoleh hingga 0,1874 dan 0,1666. Demikian pula pada metrik *Topic Diversity*, JDST-DEC masih tertinggal dibandingkan BERTopic. Meskipun demikian, JDST-DEC tetap mampu menghasilkan topik yang memadai sambil melakukan klasifikasi sentimen secara simultan dalam satu model.

5.2 Saran

Berdasarkan hasil dan keterbatasan pada penelitian ini, beberapa saran untuk penelitian selanjutnya adalah sebagai berikut.

1. Mengeksplorasi metode inisialisasi kluster selain *K-Means*, seperti HDB-SCAN, untuk memperoleh struktur kluster yang lebih sesuai terhadap karakteristik data. Saran ini didasarkan pada hasil perbandingan yang menunjukkan adanya perbedaan kualitas kluster pada beberapa konfigurasi yang diuji.
2. Mengeksplorasi pengembangan arsitektur *sentiment head* untuk mendukung keseimbangan proses optimisasi antara tugas analisis sentimen dan klusterisasi. Berdasarkan hasil analisis *multiloss*, perubahan pada komponen ini berpotensi membantu model mencapai keseimbangan yang lebih baik antara kualitas prediksi sentimen dan kualitas kluster.
3. Mengeksplorasi strategi pembobotan *loss* yang adaptif antara $\mathcal{L}_{\text{sent}}$ dan $\mathcal{L}_{\text{clust}}$ guna mengurangi *trade-off* antara performa analisis sentimen dan kualitas kluster yang dihasilkan.

DAFTAR PUSTAKA

- [1] Charu C. Aggarwal. *Neural Networks and Deep Learning: A Textbook*. Cham: Springer, 2018. ISBN: 978-3-319-94462-3. DOI: 10.1007/978-3-319-94463-0.
- [2] James Allan. *Topic Detection and Tracking: Event-Based Information Organization*. Vol. 12. The Information Retrieval Series. New York: Springer, 2002. ISBN: 978-0-7923-7664-4.
- [3] Dima Angelov. “Top2Vec: Distributed Representations of Topics”. *arXiv preprint arXiv:2008.09470* (2020).
- [4] Jonathan Baxter. “A Model of Inductive Bias Learning”. *Journal of Artificial Intelligence Research* 12 (2000), pp. 149–198. DOI: 10.1613/jair.731.
- [5] James C. Bezdek. *Pattern Recognition with Fuzzy Objective Function Algorithms*. New York: Springer Science & Business Media, 1984. ISBN: 978-1-4757-0450-1. DOI: 10.1007/978-1-4757-0450-1.
- [6] Christopher M. Bishop. *Pattern Recognition and Machine Learning*. New York: Springer, 2006. ISBN: 978-0-387-31073-2.
- [7] David M. Blei. “Probabilistic topic models”. *Communications of the ACM* 55.4 (2012), pp. 77–84.
- [8] David M. Blei, Andrew Y. Ng, dan Michael I. Jordan. “Latent Dirichlet allocation”. *Journal of Machine Learning Research* 3 (2003), pp. 993–1022.
- [9] Jordan Boyd-Graber, Yuening Hu, dan David Mimno. “Applications of topic models”. *Foundations and Trends in Information Retrieval* 11.2–3 (2017), pp. 143–296.
- [10] Ricardo J. G. B. Campello, Davoud Moulavi, dan Jrg Sander. “Density-Based Clustering Based on Hierarchical Density Estimates”. *Advances in Knowledge Discovery and Data Mining* (2013), pp. 160–172.
- [11] Dallas Card, Chenhao Tan, dan Noah A. Smith. “A Neural Framework for Generalized Topic Models”. *arXiv preprint arXiv:1705.09296* (2017).
- [12] Rich Caruana. “Multitask Learning”. *Machine Learning* 28 (1997), pp. 41–75. DOI: 10.1023/A:1007379606734.

- [13] Huimin Chen *et al.* “Neural Sentiment Classification with User and Product Attention”. *Proceedings of the Conference on Empirical Methods in Natural Language Processing*. 2016, pp. 1650–1859.
- [14] Kyunghyun Cho *et al.* “Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation”. *Proceedings of the Conference on Empirical Methods in Natural Language Processing*. 2014, pp. 1724–1734.
- [15] Junyoung Chung *et al.* “Empirical Evaluation of Gated Recurrent Neural Networks on Sequence Modeling”. *arXiv preprint arXiv:1412.3555* (2014).
- [16] Ronan Collobert *et al.* “Natural Language Processing (Almost) from Scratch”. *Journal of Machine Learning Research* 12 (2011), pp. 2493–2537.
- [17] Jacob Devlin *et al.* “BERT: Pre-training of deep bidirectional transformers for language understanding”. *arXiv preprint arXiv:1810.04805* (2018).
- [18] Adji B. Dieng, Francisco J. R. Ruiz, dan David M. Blei. “Topic Modeling in Embedding Spaces”. *Transactions of the Association for Computational Linguistics* 8 (2020), pp. 439–453.
- [19] Long Duong *et al.* “Low Resource Dependency Parsing: Cross-lingual Parameter Sharing in a Neural Network Parser” (2015), pp. 845–850.
- [20] Martin Ester *et al.* “A Density-based Algorithm for Discovering Clusters in Large Spatial Databases with Noise”. *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining (KDD)*. 1996, pp. 226–231.
- [21] Ian Goodfellow, Yoshua Bengio, dan Aaron Courville. *Deep Learning*. Cambridge, MA: MIT Press, 2016. ISBN: 978-0-262-03561-3. URL: <http://www.deeplearningbook.org>.
- [22] Theresia Gowandi, Hendri Murfi, dan Siti Nurrohmah. “Performance Analysis of Hybrid Architectures of Deep Learning for Indonesian Sentiment Analysis”. *Soft Computing in Data Science: 6th International Conference, SCDS 2021, Proceedings*. Ed. by Azlinah Mohamed *et al.* Vol. 1489. Communications in Computer and Information Science. Springer, 2021, pp. 18–27. DOI: 10.1007/978-981-16-7334-4_2.
- [23] Alex Graves. “Generating Sequences with Recurrent Neural Networks”. *arXiv preprint arXiv:1308.0850* (2013).
- [24] Maarten Grootendorst. “BERTopic: Neural topic modeling with a class-based TF-IDF procedure”. *arXiv preprint arXiv:2203.05794* (2022).

- [25] Lin Gui *et al.* “Multi-Task Mutual Learning for Joint Sentiment Classification and Topic Detection”. *IEEE Transactions on Knowledge and Data Engineering* 34.4 (2022), pp. 1915–1927.
- [26] Xifeng Guo *et al.* “Improved Deep Embedded Clustering with Local Structure Preservation”. *Proceedings of the 26th International Joint Conference on Artificial Intelligence (IJCAI)*. 2017, pp. 1753–1759. DOI: 10.24963/ijcai.2017/243. URL: <https://www.ijcai.org/proceedings/2017/0243.pdf>.
- [27] Dan Hendrycks dan Kevin Gimpel. “Gaussian Error Linear Units (GELUs)”. *arXiv preprint arXiv:1606.08415* (2016). URL: <https://arxiv.org/abs/1606.08415>.
- [28] Geoffrey E. Hinton dan Ruslan R. Salakhutdinov. “Reducing the Dimensionality of Data with Neural Networks”. *Science* 313.5786 (2006), pp. 504–507. DOI: 10.1126/science.1127647.
- [29] Yohan Jo dan Alice H. Oh. “Aspect and Sentiment Unification Model for Online Review Analysis”. *Proceedings of the Fourth ACM International Conference on Web Search and Data Mining*. 2011, pp. 815–824.
- [30] Yoon Kim. “Convolutional Neural Networks for Sentence Classification”. *Proceedings of the Conference on Empirical Methods in Natural Language Processing*. 2014, pp. 1746–1751.
- [31] Gary King dan Langche Zeng. “Logistic Regression in Rare Events Data”. *Political Analysis* 9.2 (2001), pp. 137–163.
- [32] Diederik P. Kingma dan Jimmy Ba. “Adam: A Method for Stochastic Optimization”. *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*. Published as a conference paper at ICLR 2015. 2015. URL: <https://arxiv.org/abs/1412.6980>.
- [33] Diederik P. Kingma dan Max Welling. “Auto-Encoding Variational Bayes”. *arXiv preprint arXiv:1312.6114* (2013).
- [34] Daniel D. Lee dan H. Sebastian Seung. “Learning the Parts of Objects by Non-Negative Matrix Factorization”. *Nature* 401.6755 (1999), pp. 788–791. DOI: 10.1038/44565.
- [35] Jiwei Li *et al.* “Visualizing and Understanding Neural Models in NLP”. *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*. Association for Computational Linguistics, 2016, pp. 681–691.

- [36] Qianqian Liang *et al.* “JST-RR model: Joint modeling of ratings and reviews in sentiment-topic prediction”. *arXiv preprint arXiv:2102.11048* (2021).
- [37] Chong Lin dan Yulan He. “Joint sentiment/topic model for sentiment analysis”. *Proceedings of the 18th ACM Conference on Information and Knowledge Management (CIKM)*. 2009, pp. 375–384.
- [38] Bing Liu. *Sentiment Analysis and Opinion Mining*. Synthesis Lectures on Human Language Technologies. Morgan & Claypool Publishers, 2012. ISBN: 978-1-60845-884-4.
- [39] Pengfei Liu, Xipeng Qiu, dan Xuanjing Huang. “Adversarial Multi-task Learning for Text Classification”. *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics*. 2017, pp. 1–10.
- [40] Pengfei Liu, Xipeng Qiu, dan Xuanjing Huang. “Recurrent Neural Network for Text Classification with Multi-Task Learning”. *Proceedings of the 25th International Joint Conference on Artificial Intelligence*. 2016, pp. 2873–2879.
- [41] Andrew L. Maas *et al.* “Learning Word Vectors for Sentiment Analysis”. *Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies*. 2011, pp. 142–150.
- [42] Andrew L. Maas *et al.* “Learning Word Vectors for Sentiment Analysis”. *Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, 2011, pp. 142–150.
- [43] James MacQueen. “Some Methods for Classification and Analysis of Multivariate Observations”. *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*. Vol. 1. Berkeley, CA: University of California Press, 1967, pp. 281–297.
- [44] Pradipta Maji dan Sankar K. Pal. “Rough Set Based Generalized Fuzzy C-Means Algorithm and Quantitative Indices”. *IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics)* 37.6 (2007). Cited as Maji *et al.*, 2002 for soft/hard clustering concepts, pp. 1529–1540. DOI: 10.1109/TSMCB.2007.906578.
- [45] Julian McAuley *et al.* “Image-based Recommendations on Styles and Substitutes”. *Proceedings of the 38th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2015, pp. 43–52.
- [46] Jon D. McAuliffe dan David M. Blei. “Supervised topic models”. *Advances in Neural Information Processing Systems*. 2008, pp. 121–128.

- [47] Qiaozhu Mei *et al.* “Topic Sentiment Mixture: Modeling Facets and Opinions in Weblogs”. *Proceedings of the 16th International Conference on World Wide Web*. 2007, pp. 171–180.
- [48] Yishu Miao, Lei Yu, dan Phil Blunsom. “Neural Variational Inference for Text Processing”. *Proceedings of the International Conference on Machine Learning*. 2016, pp. 1727–1736.
- [49] Tomas Mikolov, Kai Chen *et al.* “Efficient estimation of word representations in vector space”. *arXiv preprint arXiv:1301.3781* (2013).
- [50] Tomas Mikolov, Ilya Sutskever *et al.* “Distributed Representations of Words and Phrases and Their Compositionality”. *Proceedings of Advances in Neural Information Processing Systems*. 2013, pp. 3111–3119.
- [51] Shruti Pandit dan Shikha Gupta. “A Comparative Study on Distance Measuring Approaches for Clustering”. *International Journal of Research in Computer Science* 2.1 (2011), pp. 29–31. DOI: 10.7815/ijorcs.21.2011.011.
- [52] Bo Pang, Lillian Lee, dan Shivakumar Vaithyanathan. “Thumbs up? Sentiment Classification using Machine Learning Techniques”. *Proceedings of the Conference on Empirical Methods in Natural Language Processing*. 2002, pp. 79–86.
- [53] Michael Rder, Andreas Both, dan Alexander Hinneburg. “Exploring the Space of Topic Coherence Measures”. *Proceedings of the 8th ACM International Conference on Web Search and Data Mining*. 2015, pp. 399–408.
- [54] Sebastian Ruder. “An Overview of Multi-Task Learning in Deep Neural Networks”. *arXiv preprint arXiv:1706.05098* (2017). URL: <https://arxiv.org/abs/1706.05098>.
- [55] David E. Rumelhart, Geoffrey E. Hinton, dan Ronald J. Williams. “Learning representations by back-propagating errors”. *Nature* 323 (1986), pp. 533–536.
- [56] Gerard Salton dan Christopher Buckley. “Term-weighting approaches in automatic text retrieval”. *Information Processing & Management* 24.5 (1988), pp. 513–523.
- [57] Ivan Nunes da Silva *et al.* *Artificial Neural Networks: A Practical Course*. Cham: Springer, 2017. ISBN: 978-3-319-43162-8. DOI: 10.1007/978-3-319-43162-8.

- [58] Akash Srivastava dan Charles Sutton. “Autoencoding Variational Inference for Topic Models”. *Proceedings of the International Conference on Learning Representations*. 2017.
- [59] Chang Sun *et al.* “Fine-tuning BERT for joint entity and relation extraction in Chinese medical text”. *Proceedings of the 12th International Conference on Knowledge Science, Engineering and Management*. 2019, pp. 197–207.
- [60] Mukund Sundararajan, Ankur Taly, dan Qiqi Yan. “Axiomatic Attribution for Deep Networks”. *Proceedings of the 34th International Conference on Machine Learning (ICML)*. PMLR, 2017, pp. 3319–3328.
- [61] Duyu Tang, Bing Qin, dan Ting Liu. “Aspect Level Sentiment Classification with Deep Memory Network”. *Proceedings of the Conference on Empirical Methods in Natural Language Processing*. 2016, pp. 214–224.
- [62] Duyu Tang, Bing Qin, dan Ting Liu. “Document Modeling with Gated Recurrent Neural Network for Sentiment Classification”. *Proceedings of the Conference on Empirical Methods in Natural Language Processing*. 2015, pp. 1422–1432.
- [63] Ashish Vaswani *et al.* “Attention Is All You Need”. *Advances in Neural Information Processing Systems*. Vol. 30. 2017.
- [64] Pascal Vincent *et al.* “Stacked denoising autoencoders: Learning useful representations in a deep network with a local denoising criterion”. *Journal of Machine Learning Research* 11 (2010), pp. 3371–3408.
- [65] Sida Wang dan Christopher D. Manning. “Baselines and Bigrams: Simple, Good Sentiment and Topic Classification”. *Proceedings of the 50th Annual Meeting of the Association for Computational Linguistics*. 2012, pp. 90–94.
- [66] Junyuan Xie, Ross Girshick, dan Ali Farhadi. “Unsupervised deep embedding for clustering analysis”. *International Conference on Machine Learning (ICML)*. 2016.
- [67] Zichao Yang *et al.* “Hierarchical Attention Networks for Document Classification”. *Proceedings of the Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. 2016, pp. 1480–1489.
- [68] Hendi Yusdiansyah dan Kuspriyanto Suryadi. “Centroid-Based Clustering for Text Classification”. *Journal of Physics: Conference Series* 1192.1 (2019), p. 012022. DOI: 10.1088/1742-6596/1192/1/012022.

- [69] Matthew D. Zeiler dan Rob Fergus. “Visualizing and Understanding Convolutional Networks”. *European Conference on Computer Vision (ECCV)*. Springer, 2014, pp. 818–833.
- [70] Aston Zhang *et al.* *Dive into Deep Learning*. Cambridge University Press, 2023. ISBN: 978-1-009-38943-3. URL: <https://d2l.ai>.
- [71] Xiang Zhang, Junbo Zhao, dan Yann LeCun. “Character-level Convolutional Networks for Text Classification”. *Advances in Neural Information Processing Systems*. 2015, pp. 649–657.
- [72] Xiang Zhang, Junbo Zhao, dan Yann LeCun. “Character-level Convolutional Networks for Text Classification”. *Advances in Neural Information Processing Systems*. Vol. 28. Curran Associates, Inc., 2015.
- [73] Yinhan Zhang, Hao Zhao, dan Ming Li. “Neural topic models: A survey”. *arXiv preprint arXiv:2004.11906* (2020).
- [74] Yongliang Zhang *et al.* “Multi-task mutual learning for joint sentiment classification and topic detection”. *IEEE/ACM Transactions on Audio, Speech, and Language Processing* 28 (2020), pp. 2407–2417.
- [75] Yu Zhang dan Qiang Yang. “A Survey on Multi-Task Learning”. *arXiv preprint arXiv:1707.08114* (2017).
- [76] Zhibin Zhou *et al.* “A comprehensive survey on clustering: From traditional methods to modern deep learning algorithms”. *arXiv preprint arXiv:2102.08012* (2021).

LAMPIRAN

LAMPIRAN 1

KODE PROGRAM

Transformasi Data Text -> BERT Embeddings -> Cached Data [CDT]

```
1 import torch
2 from torch.utils.data import Dataset
3 from transformers import AutoTokenizer, AutoModel
4
5 class CachedBERTDataset(Dataset):
6     def __init__(
7         self,
8         texts,
9         labels=None,
10         bert_model="bert-base-uncased",
11         max_length=128,
12         cuda=True,
13         testing_mode=False,
14         return_texts=True # CRITICAL: Enable for TF-IDF plots
15     ):
16         """
17         Dataset that caches BERT embeddings for text data
18
19         Args:
20             texts: List of text strings to encode
21             labels: Optional list of labels corresponding to the texts
22             bert_model: Pre-trained BERT model name to use
23             max_length: Maximum sequence length for BERT tokenizer
24             cuda: Whether to use GPU acceleration
25             testing_mode: If True, only use a small subset of data
26             return_texts: If True, return (embedding, label, text) tuple for TF-IDF plots
27         """
28         self.texts = texts
29         self.labels = labels
30         self.cuda = cuda
31         self.testing_mode = testing_mode
32         self.return_texts = return_texts #
33         self._cache = {}
34
35         print(f"Loading BERT model: {bert_model}")
36         self.tokenizer = AutoTokenizer.from_pretrained(bert_model)
37         self.model = AutoModel.from_pretrained(bert_model)
38
39         if cuda and torch.cuda.is_available():
40             self.model = self.model.cuda()
41
42         self.max_length = max_length
43         self.model.eval()
44
45         print(f"Dataset initialized: {len(self)} samples")
46         print(f"Return format: {'(embedding, label, text)' if return_texts and labels is not None else '(embedding, label)' if labels is not None else 'embedding'}")
47
48     def _get_bert_embedding(self, text):
49         """Generate BERT embedding for a single text"""
50         inputs = self.tokenizer(
51             text,
52             return_tensors="pt",
53             max_length=self.max_length,
54             padding="max_length",
55             truncation=True
56         )
57
58         if self.cuda and torch.cuda.is_available():
```

```

59         inputs = {k: v.cuda() for k, v in inputs.items()}
60
61     with torch.no_grad():
62         outputs = self.model(**inputs)
63         # [CLS] token embedding
64         embeddings = outputs.last_hidden_state[:, 0, :]
65
66     return embeddings.squeeze(0).cpu() # Move to CPU for caching
67
68     def __getitem__(self, index: int):
69         """
70         Get embedding and (optional) label and text for index
71
72         Returns:
73         - If labels=None: embedding (Tensor)
74         - If labels provided & return_texts=False: (embedding, label)
75         - If labels provided & return_texts=True: (embedding, label, text)
76         """
77         # Handle testing mode wrapping
78         if self.testing_mode and index >= 128:
79             index = index % 128
80
81         # Check cache
82         if index not in self._cache:
83             text = self.texts[index]
84             embedding = self._get_bert_embedding(text)
85
86             if self.labels is not None:
87                 label = self.labels[index]
88
89                 # Handle both int labels and one-hot/array labels
90                 if isinstance(label, (int, float)):
91                     label_tensor = torch.tensor(label, dtype=torch.long)
92                 elif isinstance(label, (list, tuple)):
93                     label_tensor = torch.tensor(label, dtype=torch.float)
94                 elif isinstance(label, torch.Tensor):
95                     label_tensor = label.clone().detach()
96                 else:
97                     label_tensor = torch.tensor(label, dtype=torch.long)
98
99                 # Return with text for TF-IDF plots
100                 if self.return_texts:
101                     self._cache[index] = (embedding, label_tensor, text)
102                 else:
103                     self._cache[index] = (embedding, label_tensor)
104             else:
105                 # No labels - just embedding
106                 self._cache[index] = embedding
107
108     return self._cache[index]
109
110     def __len__(self):
111         """Return dataset length, limited in testing mode"""
112         return min(128, len(self.texts)) if self.testing_mode else len(self.texts)
113
114     def clear_cache(self):
115         """Clear cached embeddings to free memory"""
116         self._cache.clear()
117         print("Cache cleared")
118
119     def get_embedding_dim(self):
120         """Get the dimensionality of BERT embeddings"""
121         sample = self[0]
122         if isinstance(sample, tuple):
123             return sample[0].shape[0]
124         else:
125             return sample.shape[0]

```

Listing 1: Full implementation of Cached Data Transformation (CDT).

Arsitektur Model

```

1  from __future__ import annotations
2
3  import os
4  import csv
5  import glob
6  import re
7  import warnings
8  import json
9  import copy
10 from collections import Counter, defaultdict
11 from typing import Iterable, List, Dict, Tuple, Optional, Union
12 from pathlib import Path
13
14 import numpy as np
15 import pandas as pd
16 import torch
17 import torch.nn as nn
18 import torch.optim as optim
19 from torch.utils.data import DataLoader, TensorDataset, Subset
20 import matplotlib.gridspec as gridspec
21 import matplotlib.patches as mpatches
22 import matplotlib.ticker as mticker
23 from matplotlib.colors import LinearSegmentedColormap
24 from matplotlib.lines import Line2D
25
26 from tqdm import tqdm
27 from sklearn.cluster import KMeans
28 from sklearn.metrics import (
29     precision_score, recall_score, f1_score, silhouette_score,
30     normalized_mutual_info_score, adjusted_rand_score,
31     homogeneity_score, completeness_score, v_measure_score
32 )
33 from sklearn.decomposition import PCA
34 from sklearn.feature_extraction.text import TfidfVectorizer
35 from scipy.optimize import linear_sum_assignment as linear_assignment
36 from scipy.stats import entropy
37 from transformers import AutoTokenizer, AutoModel
38
39 import matplotlib.pyplot as plt
40 import seaborn as sns
41
42
43 # -----
44 # Professional Plot Style
45 # -----
46 def _apply_professional_style():
47     plt.rcParams.update({
48         "figure.facecolor":      "#FAFAFA",
49         "axes.facecolor":        "#FFFFFF",
50         "axes.edgecolor":        "#CCCCCC",
51         "axes.linewidth":        0.8,
52         "axes.grid":             True,
53         "axes.grid.axis":        "x",
54         "grid.color":            "#E8E8E8",
55         "grid.linewidth":        0.6,
56         "grid.linestyle":        "--",
57         "axes.spines.top":        False,
58         "axes.spines.right":      False,
59         "axes.spines.left":       False,
60         "xtick.color":            "#555555",
61         "ytick.color":            "#555555",
62         "xtick.labelsize":        8,
63         "ytick.labelsize":        8,
64         "axes.labelsize":         9,
65         "axes.titlesize":         11,
66         "axes.titleweight":       "bold",
67         "axes.titlepad":          10,
68         "figure.titlesize":        13,
69         "figure.titleweight":     "bold",
70         "font.family":            "DejaVu Sans",
71         "legend.framealpha":       0.92,
72         "legend.edgecolor":       "#CCCCCC",
73         "legend.fontsize":        8,
74     })
75
76 _RWG = LinearSegmentedColormap.from_list(

```

```

77     "rwg_pro", ["#C0392B", "#FAFAFA", "#1A7A4A"], N=512
78 )
79
80 _COL_POS = "#1A7A4A"
81 _COL_NEG = "#C0392B"
82 _COL_NEU = "#7F8C8D"
83
84 _CLUSTER_PALETTE = [
85     "#2E86AB", "#A23B72", "#F18F01", "#C73E1D", "#3B1F2B",
86     "#44BBA4", "#E94F37", "#393E41", "#F5A623", "#7B2D8B",
87     "#0D7A5F", "#D62246", "#4F5D75", "#EF8C2A", "#1B4332",
88     "#6A0572", "#C9A84C", "#2C7BB6", "#B5451B", "#3D6B35",
89     "#9C4A1A", "#2980B9", "#8E44AD", "#16A085", "#D35400",
90     "#1ABC9C", "#E74C3C", "#34495E", "#F39C12", "#27AE60",
91 ]
92
93 warnings.filterwarnings("ignore", category=UserWarning)
94
95
96 # -----
97 # Device
98 # -----
99 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
100
101
102 # -----
103 # Utils
104 # -----
105 def cluster_acc(y_true: np.ndarray, y_pred: np.ndarray) -> float:
106     y_true = y_true.astype(np.int64)
107     assert y_pred.size == y_true.size
108     D = max(y_pred.max(), y_true.max()) + 1
109     w = np.zeros((D, D), dtype=np.int64)
110     for i in range(y_pred.size):
111         w[y_pred[i], y_true[i]] += 1
112     row_ind, col_ind = linear_assignment(w.max() - w)
113     return float(sum(w[i, j] for i, j in zip(row_ind, col_ind)) / y_pred.size)
114
115
116 # -----
117 # Model Components
118 # -----
119 class ClusteringLayer(nn.Module):
120     def __init__(self, n_clusters: int, input_dim: int, alpha: float = 1.0) -> None:
121         super().__init__()
122         self.n_clusters = n_clusters
123         self.alpha = alpha
124         self.clusters = nn.Parameter(torch.empty(n_clusters, input_dim))
125         nn.init.xavier_uniform_(self.clusters)
126
127     def forward(self, x: torch.Tensor) -> torch.Tensor:
128         dist = torch.sum((x.unsqueeze(1) - self.clusters.unsqueeze(0)) ** 2, dim=2)
129         q = 1.0 / (1.0 + dist / self.alpha)
130         q = q ** ((self.alpha + 1.0) / 2.0)
131         q = q / torch.sum(q, dim=1, keepdim=True)
132         return q
133
134
135 class Autoencoder(nn.Module):
136     def __init__(self, dims, act='relu'):
137         super().__init__()
138         assert len(dims) >= 2
139         self.dims = list(dims)
140         act_map = {'relu': nn.ReLU(), 'sigmoid': nn.Sigmoid(), 'tanh': nn.Tanh()}
141         self.activation = act_map.get(act, nn.ReLU())
142
143         enc = []
144         for i in range(len(dims) - 2):
145             enc += [nn.Linear(dims[i], dims[i + 1]), self.activation]
146         enc += [nn.Linear(dims[-2], dims[-1])]
147         self.encoder = nn.Sequential(*enc)
148
149         dec = []
150         for j in range(len(dims) - 1, 0, -1):
151             dec += [nn.Linear(dims[j], dims[j - 1])]
152             if j != 1:
153                 dec += [self.activation]

```

```

154         self.decoder = nn.Sequential(*dec)
155
156         for m in self.modules():
157             if isinstance(m, nn.Linear):
158                 nn.init.xavier_uniform_(m.weight)
159                 if m.bias is not None:
160                     nn.init.zeros_(m.bias)
161
162         def encode(self, x): return self.encoder(x)
163         def decode(self, h): return self.decoder(h)
164         def forward(self, x): h = self.encode(x); return h, self.decode(h)
165
166
167 # -----
168 # SEMTGPU
169 # -----
170 class SEMTGPU(nn.Module):
171     """
172     Joint Sentiment + Topic Clustering (DEC-style) with Autoencoder features.
173
174     v3.7 Changes (vs v3.6.2):
175     Reconstruction loss REMOVED from fit().
176     The encoder already receives gradient signal from the clustering loss
177     (gamma) and the sentiment loss (eta) during multi-task joint training,
178     so reconstruction is redundant here.
179     Reconstruction loss is still used in pretrain_autoencoder() where it
180     is the sole training signal and remains essential.
181     alpha parameter removed from fit() signature entirely.
182     fit() log CSV: 'Lr' column removed (reconstruction loss no longer tracked).
183
184     v3.6.2 Fixes retained:
185     FIX 1 compute_integrated_gradients uses eval() to prevent BatchNorm
186           running stat corruption from synthetic interpolated inputs.
187     FIX 2 fit() computes final metrics BEFORE IG / token-attribution calls.
188     FIX 3 (removed was the alpha=0 warning, no longer applicable).
189     FIX 4 Early-stop guard: delta < tol only checked after ite >= update_interval.
190     """
191
192     def __init__(
193         self,
194         dims: List[int],
195         n_clusters: int = 10,
196         alpha_clustering: float = 1.0,
197     ) -> None:
198         super().__init__()
199         assert len(dims) >= 2
200         self.dims = dims
201         self.n_clusters = int(n_clusters)
202         self.alpha_clustering = float(alpha_clustering)
203
204         self.autoencoder = Autoencoder(dims)
205         self.clustering = ClusteringLayer(n_clusters, dims[-1], alpha_clustering)
206
207         self.sentiment = nn.Sequential(
208             nn.Linear(dims[-1], 256), nn.BatchNorm1d(256), nn.GELU(), nn.Dropout(0.5),
209             nn.Linear(256, 32), nn.BatchNorm1d(32), nn.GELU(), nn.Dropout(0.5),
210             nn.Linear(32, 2),
211         )
212
213         self.class_labels: Dict[int, str] = {0: "negative", 1: "positive"}
214         self.topic_mapping: Dict[int, str] = {}
215         self.stop_words: set = set()
216
217         self._best_val_score: float = -1.0
218         self._best_val_iter: int = -1
219         self._best_state_dict: Optional[dict] = None
220
221         for m in self.sentiment.modules():
222             if isinstance(m, nn.Linear):
223                 nn.init.xavier_uniform_(m.weight)
224                 if m.bias is not None:
225                     nn.init.zeros_(m.bias)
226
227 # -- Token Cleaning -----
228 @staticmethod
229 def _clean_token(tok: str) -> str:
230     tok = tok.lstrip('\u0120')

```

```

231     tok = tok.lstrip('\u2581')
232     tok = tok.replace('##', '')
233     tok = tok.lstrip('\u2047')
234     tok = tok.strip()
235     return tok
236
237 # -- Forward / inference -----
238 def forward(self, x):
239     z = self.autoencoder.encode(x)
240     return self.clustering(z), torch.softmax(self.sentiment(z), dim=1)
241
242 def extract_feature(self, x):
243     self.eval()
244     with torch.no_grad():
245         xt = torch.as_tensor(x, dtype=torch.float32,
246                               device=next(self.parameters()).device)
247     return self.autoencoder.encode(xt)
248
249 def predict_clusters(self, x):
250     self.eval()
251     with torch.no_grad():
252         q, _ = self(torch.as_tensor(x, dtype=torch.float32,
253                                     device=next(self.parameters()).device))
254     return q.argmax(1).cpu().numpy()
255
256 def predict_sentiment(self, x):
257     self.eval()
258     with torch.no_grad():
259         _, s = self(torch.as_tensor(x, dtype=torch.float32,
260                                     device=next(self.parameters()).device))
261     return s.argmax(1).cpu().numpy()
262
263 def get_cluster_assignments(self, x):
264     return self.predict_clusters(x)
265
266 # -- Weight I/O -----
267 def save_weights(self, path: str) -> None:
268     os.makedirs(os.path.dirname(os.path.abspath(path)), exist_ok=True)
269     torch.save({"model_state_dict": self.state_dict()}, path)
270
271 def load_weights(self, path: str) -> None:
272     loc = None if torch.cuda.is_available() else {"cuda:0": "cpu"}
273     state = torch.load(path, map_location=loc)
274     self.load_state_dict(
275         state["model_state_dict"] if "model_state_dict" in state else state
276     )
277     print(f"Loaded weights from {path}")
278
279 def load_best_weights(self) -> None:
280     if self._best_state_dict is None:
281         print("No best checkpoint available; keeping current weights.")
282         return
283     self.load_state_dict(self._best_state_dict)
284     print(f"Restored best model (iter={self._best_val_iter}, "
285           f"val_score={self._best_val_score:.4f})")
286
287 # -- Validation evaluation -----
288 def _evaluate_val(
289     self,
290     X_val: torch.Tensor,
291     Y_val: Optional[torch.Tensor],
292     texts_val: Optional[List[str]],
293     batch_size: int,
294 ) -> Dict[str, float]:
295     self.eval()
296
297     q_list, s_list = [], []
298     with torch.no_grad():
299         for i in range(0, X_val.size(0), batch_size):
300             qb, sb = self(X_val[i: i + batch_size])
301             q_list.append(qb); s_list.append(sb)
302
303     q_val = torch.cat(q_list, 0)
304     s_val = torch.cat(s_list, 0)
305     y_pred_cluster = q_val.argmax(1).cpu().numpy()
306     y_pred_sent = s_val.argmax(1).cpu().numpy()
307

```

```

308 metrics: Dict[str, float] = {}
309
310 if Y_val is not None:
311     y_true = Y_val.cpu().numpy()
312     if y_true.ndim == 2 and y_true.shape[1] > 1:
313         y_true = y_true.argmax(1)
314     metrics["val_acc_sentiment"] = float((y_pred_sent == y_true).mean())
315     metrics["val_f1"] = float(f1_score(y_true, y_pred_sent,
316                                     average="binary", zero_division=0))
317     metrics["val_precision"] = float(precision_score(y_true, y_pred_sent,
318                                     average="binary", zero_division=0))
319     metrics["val_recall"] = float(recall_score(y_true, y_pred_sent,
320                                     average="binary", zero_division=0))
321     if len(np.unique(y_true)) > 1 and len(np.unique(y_pred_cluster)) > 1:
322         metrics["val_nmi"] = float(normalized_mutual_info_score(y_true, y_pred_cluster))
323         metrics["val_ari"] = float(adjusted_rand_score(y_true, y_pred_cluster))
324         metrics["val_acc_cluster"] = cluster_acc(y_true, y_pred_cluster)
325     else:
326         metrics.update({"val_nmi": 0.0, "val_ari": 0.0, "val_acc_cluster": 0.0})
327 else:
328     metrics.update({
329         "val_acc_sentiment": 0.0, "val_f1": 0.0,
330         "val_precision": 0.0, "val_recall": 0.0,
331         "val_nmi": 0.0, "val_ari": 0.0,
332         "val_acc_cluster": 0.0,
333     })
334
335 if texts_val and len(texts_val) > 0:
336     coh_scores = self.compute_topic_coherence(texts_val, y_pred_cluster)
337     metrics["val_coherence"] = float(np.mean(list(coh_scores.values()))) or 0.0
338     metrics["val_diversity"] = self.compute_topic_diversity(texts_val, y_pred_cluster)
339 else:
340     feats_val = self.extract_feature(X_val).cpu().numpy()
341     if len(np.unique(y_pred_cluster)) >= 2:
342         try:
343             metrics["val_coherence"] = float(silhouette_score(feats_val, y_pred_cluster))
344         except Exception:
345             metrics["val_coherence"] = 0.0
346     else:
347         metrics["val_coherence"] = 0.0
348     metrics["val_diversity"] = 0.0
349
350 metrics["val_cluster_score"] = (
351     metrics["val_coherence"] + metrics["val_diversity"]
352 ) / 2.0
353
354 metrics["val_primary_score"] = (
355     metrics["val_f1"] if Y_val is not None else metrics["val_cluster_score"]
356 )
357 return metrics
358
359 # -- Pretraining (reconstruction loss lives here unchanged) -----
360 def pretrain_autoencoder(
361     self,
362     dataset,
363     batch_size=256, epochs=200, lr=1e-3,
364     save_dir="./results/ae", weights_name="pretrained_ae.weights.pth",
365 ) -> str:
366     print("=" * 60)
367     print("Pretraining Autoencoder")
368     print("=" * 60)
369     save_path = Path(save_dir); save_path.mkdir(parents=True, exist_ok=True)
370     weights_path = save_path / weights_name
371
372     embs = []
373     for i in range(len(dataset)):
374         item = dataset[i]
375         emb = item[0] if isinstance(item, tuple) else item
376         embs.append((emb if isinstance(emb, torch.Tensor) else
377                     torch.tensor(emb, dtype=torch.float32)).detach().cpu())
378     X = torch.stack(embs)
379     if X.shape[1] != self.dims[0]:
380         raise ValueError(f"Input dim mismatch: {X.shape[1]} vs dims[0]={self.dims[0]}")
381
382     dev = next(self.parameters()).device
383     loader = DataLoader(TensorDataset(X.to(dev)), batch_size=batch_size, shuffle=True)
384     self.autoencoder.to(dev).train()

```

```

385     opt = optim.Adam(self.autoencoder.parameters(), lr=lr)
386     crit = nn.MSELoss()
387
388     for ep in range(epochs):
389         total = 0.0
390         with tqdm(loader, desc=f"AE Epoch {ep+1}/{epochs}") as pbar:
391             for (xb,) in pbar:
392                 _, rec = self.autoencoder(xb)
393                 loss = crit(rec, xb)
394                 opt.zero_grad(); loss.backward(); opt.step()
395                 total += float(loss.item())
396                 pbar.set_postfix({"mse": total / (pbar.n + 1)})
397
398     torch.save({"autoencoder_state_dict": self.autoencoder.state_dict(),
399               "dims": self.dims, "str(weights_path)"})
400     print(f" AE pretrain complete {weights_path}")
401     return str(weights_path)
402
403 # -- Metrics helpers -----
404 @staticmethod
405 def target_distribution(q):
406     weight = (q ** 2) / torch.clamp(torch.sum(q, 0), min=1e-12)
407     return (weight.t() / torch.clamp(torch.sum(weight, 1), min=1e-12)).t()
408
409 def compute_class_weights(self, y):
410     if isinstance(y, torch.Tensor): y = y.detach().cpu().numpy()
411     if y.ndim == 2 and y.shape[1] > 1: y = y.argmax(1)
412     cls, cnt = np.unique(y, return_counts=True)
413     total = len(y); k = len(cls)
414     return {int(c): total / (k * int(n)) for c, n in zip(cls, cnt)}
415
416 def compute_clustering_metrics(self, y_true, y_pred):
417     if len(np.unique(y_true)) < 2 or len(np.unique(y_pred)) < 2:
418         return {k: 0.0 for k in
419               ['ACC', 'NMI', 'ARI', 'Homogeneity', 'Completeness', 'V-measure', 'Topic_Coverage']}
420     uc, cc = np.unique(y_pred, return_counts=True)
421     cp = cc / len(y_pred)
422     tc = 1.0 - (entropy(cp) / np.log(len(uc))) if len(uc) > 1 else 0.0
423     return {
424         'ACC': cluster_acc(y_true, y_pred),
425         'NMI': float(normalized_mutual_info_score(y_true, y_pred)),
426         'ARI': float(adjusted_rand_score(y_true, y_pred)),
427         'Homogeneity': float(homogeneity_score(y_true, y_pred)),
428         'Completeness': float(completeness_score(y_true, y_pred)),
429         'V-measure': float(v_measure_score(y_true, y_pred)),
430         'Topic_Coverage': float(tc),
431     }
432
433 def compute_silhouette_score(self, embeddings, cluster_assignments):
434     emb = embeddings.detach().cpu().numpy() if isinstance(embeddings, torch.Tensor) else embeddings
435     if len(np.unique(cluster_assignments)) < 2: return 0.0
436     try: return float(silhouette_score(emb, cluster_assignments))
437     except: return 0.0
438
439 def extract_tfidf_keywords(self, texts, cluster_assignments, top_n=10, max_features=5000):
440     clusters: Dict[int, List[str]] = {}
441     for i, cid in enumerate(cluster_assignments):
442         clusters.setdefault(int(cid), []).append(texts[i])
443     result = {}
444     all_docs = [" ".join(t) for t in clusters.values()]
445     for cid, c_texts in clusters.items():
446         vect = TfidfVectorizer(max_features=max_features,
447                               stop_words=list(self.stop_words) or None,
448                               min_df=1, ngram_range=(1, 2))
449         try:
450             mat = vect.fit_transform(all_docs)
451             fn = vect.get_feature_names_out()
452             idx = list(clusters.keys()).index(cid)
453             sc = mat[idx].toarray()[0]
454             ti = sc.argsort()[::-top_n][::-1]
455             result[cid] = [(fn[i], float(sc[i])) for i in ti]
456         except Exception:
457             words = [w for w in " ".join(c_texts).lower().split()
458                     if w not in self.stop_words and len(w) > 2]
459             result[cid] = [(w, float(c)) for w, c in Counter(words).most_common(top_n)]
460     return result
461

```

```

462 def compute_topic_coherence(self, texts, cluster_assignments, top_n=10):
463     from itertools import combinations
464     clusters: Dict[int, List[str]] = {}
465     for i, cid in enumerate(cluster_assignments):
466         clusters.setdefault(int(cid), []).append(texts[i])
467     scores = {}
468     for cid, c_texts in clusters.items():
469         words = [w for w in " ".join(c_texts).lower().split()
470                  if w not in self.stop_words and len(w) > 2]
471         top = [w for w, _ in Counter(words).most_common(top_n)]
472         if len(top) < 2: scores[cid] = 0.0; continue
473         df = {w: sum(1 for t in c_texts if w in t.lower()) for w in top}
474         npmi = []
475         for w1, w2 in combinations(top, 2):
476             co = sum(1 for t in c_texts if w1 in t.lower() and w2 in t.lower())
477             if co > 0 and df[w1] > 0 and df[w2] > 0:
478                 p12 = co / len(c_texts); p1 = df[w1]/len(c_texts); p2 = df[w2]/len(c_texts)
479                 pmi = np.log((p12 + 1e-10) / (p1 * p2 + 1e-10))
480                 npmi.append(pmi / (-np.log(p12 + 1e-10)))
481         scores[cid] = float(np.mean(npmi)) if npmi else 0.0
482     return scores
483
484 def compute_topic_diversity(self, texts, cluster_assignments, top_n=10):
485     clusters: Dict[int, List[str]] = {}
486     for i, cid in enumerate(cluster_assignments):
487         clusters.setdefault(int(cid), []).append(texts[i])
488
489     cluster_ids = sorted(clusters.keys())
490     cluster_docs = [" ".join(clusters[c]) for c in cluster_ids]
491
492     if len(cluster_docs) < 2:
493         return 0.0
494
495     try:
496         vect = TfidfVectorizer(
497             max_features=5000,
498             stop_words=list(self.stop_words) or None,
499             min_df=1, ngram_range=(1, 1),
500         )
501         mat = vect.fit_transform(cluster_docs)
502         feat_names = vect.get_feature_names_out()
503         all_sets = []
504         for row_idx in range(mat.shape[0]):
505             row = mat[row_idx].toarray()[0]
506             top_indices = row.argsort()[::-1][-top_n:]
507             top_words = set(feat_names[i] for i in top_indices if row[i] > 0)
508             all_sets.append(top_words)
509     except Exception:
510         all_sets = []
511     for c in cluster_ids:
512         words = [w for w in " ".join(clusters[c]).lower().split()
513                  if w not in self.stop_words and len(w) > 2]
514         all_sets.append(set(w for w, _ in Counter(words).most_common(top_n)))
515
516     if not all_sets:
517         return 0.0
518
519     unique_words = set().union(*all_sets)
520     total_words = sum(len(s) for s in all_sets)
521     return float(len(unique_words) / total_words) if total_words else 0.0
522
523 def compute_topic_coverage(self, texts, cluster_assignments):
524     counts = Counter(cluster_assignments)
525     total = len(cluster_assignments)
526     sizes = [c / total for c in counts.values()]
527     ent = -sum(p * np.log(p + 1e-10) for p in sizes)
528     maxent = np.log(len(counts))
529     return {
530         'cluster_balance': float(ent / maxent) if maxent > 0 else 0.0,
531         'min_cluster_size': float(min(sizes)),
532         'max_cluster_size': float(max(sizes)),
533         'n_clusters': len(counts),
534     }
535
536 def set_stop_words(self, stop_words):
537     self.stop_words = set(stop_words); return self
538

```

```

539 def set_topic(self, cluster_id, topic_name):
540     self.topic_mapping[cluster_id] = topic_name; return self
541
542 def reset_topics(self):
543     self.topic_mapping = {}; return self
544
545 def get_topic_assignments(self):
546     return self.topic_mapping.copy()
547
548 # -- Integrated Gradients -----
549 def compute_integrated_gradients(self, x, target_class=1, n_steps=50, batch_size=64):
550     """Uses eval() to prevent BatchNorm running stat corruption (FIX 1)."""
551     dev = next(self.parameters()).device
552     xt = torch.as_tensor(x, dtype=torch.float32, device=dev)
553     N, D = xt.shape
554     base = torch.zeros_like(xt)
555     all_attr = []
556
557     was_training = self.training
558     self.eval()
559     try:
560         for start in range(0, N, batch_size):
561             end = min(start + batch_size, N)
562             xb = xt[start:end]; bb = base[start:end]; b = xb.shape[0]
563             alphas = torch.linspace(0, 1, n_steps, device=dev)
564             interp = (bb.unsqueeze(0) +
565                     alphas.view(-1, 1, 1) * (xb - bb).unsqueeze(0)
566                     ).view(n_steps * b, D).requires_grad_(True)
567             z = self.autoencoder.encode(interp)
568             s = self.sentiment(z)
569             grads = torch.autograd.grad(s[:, target_class].sum(), interp,
570                                         create_graph=False)[0]
571             grads = grads.view(n_steps, b, D)
572             avg = (grads[:-1] + grads[1:]).mean(0) / 2.0
573             all_attr.append(((xb - bb) * avg.detach()).detach().cpu().numpy())
574         finally:
575             if was_training:
576                 self.train()
577
578     return np.concatenate(all_attr, 0)
579
580 def plot_integrated_gradients(self, attributions, cluster_assignments, epoch,
581                             save_dir="./results/fnnjst",
582                             figsize=(16, 7), top_dims=20,
583                             save_plot=True, show_plot=False):
584     _apply_professional_style()
585
586     abs_attr = np.abs(attributions)
587     global_imp = abs_attr.mean(0)
588     top_idx = np.argsort(global_imp)[-top_dims:]
589     uniq = np.unique(cluster_assignments)
590     cl_ig = np.array([
591         abs_attr[cluster_assignments == c][:, top_idx].mean(0)
592         if (cluster_assignments == c).sum() > 0 else np.zeros(top_dims)
593         for c in uniq
594     ])
595     top_vals = global_imp[top_idx]
596     vmax_hm = cl_ig.max() if cl_ig.max() > 0 else 1.0
597
598     fig = plt.figure(figsize=figsize, facecolor="#FAFAFA")
599     fig.suptitle(
600         f"Integrated Gradients Attribution Epoch {epoch}",
601         fontsize=13, fontweight="bold", color="#1C1C1C", y=1.01
602     )
603
604     gs = gridspec.GridSpec(1, 2, figure=fig, width_ratios=[1, 2.0], wspace=0.35)
605     ax1 = fig.add_subplot(gs[0])
606     norm_vals = top_vals / (top_vals.max() + 1e-8)
607     bar_colors = [plt.cm.YlOrRd(0.25 + 0.70 * v) for v in norm_vals]
608     ypos = np.arange(top_dims)
609     bars = ax1.barh(ypos, top_vals, color=bar_colors, edgecolor="#FFFFFF",
610                    linewidth=0.6, height=0.72)
611     ax1.set_yticks(ypos)
612     ax1.set_yticklabels([f"dim {i}" for i in top_idx],
613                        fontsize=7.5, fontfamily="monospace")
614     ax1.invert_yaxis()
615     ax1.set_xlabel("Mean |IG Attribution|", fontsize=9, color="#4444")

```

```

616 ax1.set_title("Top Embedding Dimensions\n(Global Sentiment Influence)",
617               fontsize=10, fontweight="bold", pad=8)
618 ax1.axvline(top_vals.mean(), color="#888", lw=0.8,
619             linestyle=":", label=f"mean=(top_vals.mean():.4f)")
620 ax1.legend(fontsize=7, loc="lower right")
621 for bar, val in zip(bars, top_vals):
622     ax1.text(val + top_vals.max() * 0.015, bar.get_y() + bar.get_height() / 2,
623            f"{val:.4f}", va="center", ha="left", fontsize=6.5, color="#333")
624 ax1.set_facecolor("#FFFFFF")
625
626 ax2 = fig.add_subplot(gs[1])
627 im = ax2.imshow(cl_ig, aspect="auto", cmap="YlOrRd", vmin=0, vmax=vmax_hm,
628                interpolation="nearest")
629 ax2.set_xticks(range(top_dims))
630 ax2.set_xticklabels([f"d{i}" for i in top_idx], rotation=90,
631                    fontsize=7, fontfamily="monospace")
632 ax2.set_yticks(range(len(uniq)))
633 ax2.set_yticklabels([f"C{c}" for c in uniq])
634 ax2.set_title("Per-Cluster Dimension Attribution Heatmap",
635               fontsize=10, fontweight="bold", pad=8)
636 ax2.set_xlabel("Embedding Dimension", fontsize=9, color="#444")
637 ax2.set_ylabel("Cluster", fontsize=9, color="#444")
638 ax2.tick_params(axis="both", which="both", length=0)
639 ax2.set_xticks(np.arange(-0.5, top_dims, 1), minor=True)
640 ax2.set_yticks(np.arange(-0.5, len(uniq), 1), minor=True)
641 ax2.grid(which="minor", color="#E0E0E0", linewidth=0.5)
642
643 if top_dims <= 30 and len(uniq) <= 40:
644     for i in range(len(uniq)):
645         for j in range(top_dims):
646             v = cl_ig[i, j]
647             txt_col = "white" if v > vmax_hm * 0.65 else "#2C2C2C"
648             ax2.text(j, i, f"{v:.3f}", ha="center", va="center",
649                    fontsize=5.5, color=txt_col)
650
651 cb = plt.colorbar(im, ax=ax2, fraction=0.025, pad=0.02)
652 cb.set_label("Mean |IG|", fontsize=8, color="#444")
653 cb.ax.tick_params(labelsize=7)
654
655 plt.tight_layout()
656
657 if save_plot:
658     os.makedirs(save_dir, exist_ok=True)
659     out = os.path.join(save_dir, f"integrated_gradients_epoch_{epoch}.png")
660     plt.savefig(out, dpi=180, bbox_inches="tight", facecolor="#FAFAFA")
661     print(f"IG plot saved: {out}")
662 if show_plot: plt.show()
663 else: plt.close()
664
665 # -- Frequency-based representativeness scoring -----
666
667 def _rank_texts_by_vocab_coverage(
668     self, texts: List[str], vocab: List[str], max_samples: int,
669 ) -> List[int]:
670     if not vocab:
671         return list(range(min(max_samples, len(texts))))
672     vocab_set = set(w.lower() for w in vocab)
673     scores = []
674     for i, text in enumerate(texts):
675         words = set(re.findall(r'\b\w+\b', text.lower()))
676         coverage = len(words & vocab_set)
677         scores.append((i, coverage))
678     scores.sort(key=lambda x: (-x[1], x[0]))
679     return [idx for idx, _ in scores[:max_samples]]
680
681 def _get_cluster_tfidf_vocab(
682     self, all_texts: List[str], cluster_mask: np.ndarray,
683     top_n: int = 30, max_features: int = 5000,
684 ) -> List[str]:
685     c_texts = [all_texts[i] for i in np.where(cluster_mask)[0]]
686     if not c_texts:
687         return []
688     try:
689         vect = TfidfVectorizer(
690             max_features=max_features,
691             stop_words=list(self.stop_words) or None,
692             min_df=1, ngram_range=(1, 1),

```

```

693         )
694         mat = vect.fit_transform(c_texts)
695         fn = vect.get_feature_names_out()
696         mean_scores = mat.toarray().mean(axis=0)
697         top_idx = mean_scores.argsort()[::-1]
698         return [fn[i] for i in top_idx if mean_scores[i] > 0]
699     except Exception:
700         words = [w for w in " ".join(c_texts).lower().split()
701                  if w not in self.stop_words and len(w) > 2]
702         return [w for w, _ in Counter(words).most_common(top_n)]
703
704 # -- Longformer CLS helpers -----
705 def _longformer_cls_embedding(self, text, model, tokenizer, max_length=4096):
706     dev = next(self.parameters()).device
707     enc = tokenizer(text, return_tensors="pt", padding=True,
708                    truncation=True, max_length=max_length).to(dev)
709     global_attention_mask = torch.zeros_like(enc["input_ids"])
710     global_attention_mask[:, 0] = 1
711     with torch.no_grad():
712         out = model(**enc, global_attention_mask=global_attention_mask)
713     return out.last_hidden_state[:, 0, :], None, enc, global_attention_mask
714
715 def _get_model_cls_embedding(self, text, model, tokenizer, max_length,
716                             is_longformer: bool):
717     if is_longformer:
718         return self._longformer_cls_embedding(text, model, tokenizer, max_length)
719     else:
720         dev = next(self.parameters()).device
721         enc = tokenizer(text, return_tensors="pt", padding=True,
722                        truncation=True, max_length=max_length).to(dev)
723         with torch.no_grad():
724             out = model(**enc)
725         return out.last_hidden_state[:, 0, :], None, enc, None
726
727 # -- Bidirectional Occlusion Scoring -----
728 def _occlusion_scores_bidirectional(
729     self, text, model, tokenizer, max_length=4096, is_longformer: bool = True
730 ) -> Tuple[List[str], np.ndarray, np.ndarray, np.ndarray]:
731     dev = next(self.parameters()).device
732     cls_emb, _, enc, global_attn_mask = self._get_model_cls_embedding(
733         text, model, tokenizer, max_length, is_longformer
734     )
735     self.eval()
736     with torch.no_grad():
737         _, s = self(cls_emb)
738     base_prob = s.squeeze(0).cpu().numpy()
739
740     tokens = tokenizer.convert_ids_to_tokens(enc["input_ids"][0].cpu().tolist())
741     ids = enc["input_ids"][0].cpu().tolist()
742     n_tok = len(ids)
743     mask_id = tokenizer.mask_token_id
744     if mask_id is None:
745         mask_id = tokenizer.pad_token_id
746
747     scores_pos = np.zeros(n_tok)
748     scores_neg = np.zeros(n_tok)
749
750     _SKIP_TOKENS = {"[CLS]", "[SEP]", "[PAD]", "<s>", "</s>", "<pad>", "<mask>"}
751
752     for i, tok in enumerate(tokens):
753         if tok in _SKIP_TOKENS:
754             continue
755         masked_ids = ids.copy()
756         masked_ids[i] = mask_id
757         inp = {k: enc[k].clone() for k in enc}
758         inp["input_ids"] = torch.tensor([masked_ids], device=dev)
759         with torch.no_grad():
760             if is_longformer:
761                 out = model(**inp, global_attention_mask=global_attn_mask)
762             else:
763                 out = model(**inp)
764             _, sm = self(out.last_hidden_state[:, 0, :])
765         masked_prob = sm.squeeze(0).cpu().numpy()
766         scores_pos[i] = float(base_prob[1] - masked_prob[1])
767         scores_neg[i] = float(base_prob[0] - masked_prob[0])
768
769     return tokens, scores_pos, scores_neg, base_prob

```

```

770
771 # -- compute_token_attribution_per_cluster_with_embeddings (v3.6) -----
772 def compute_token_attribution_per_cluster_with_embeddings(
773     self,
774     texts: List[str],
775     embeddings: np.ndarray,
776     cluster_assignments: np.ndarray,
777     bert_model_name: str = "allenai/longformer-base-4096",
778     max_length: int = 4096,
779     max_samples_per_cluster: int = 30,
780     top_k: int = 15,
781     tfidf_vocab_size: int = 30,
782 ) -> Dict[int, Dict[str, Dict[str, float]]]:
783     is_longformer = "longformer" in bert_model_name.lower()
784     pred_sent = self.predict_sentiment(embeddings)
785
786     print(f"\n Predicted sentiment distribution:")
787     print(f"    Negative (0): {(pred_sent == 0).sum()}")
788     print(f"    Positive (1): {(pred_sent == 1).sum()}")
789
790     cl_idx_neg: Dict[int, List[int]] = defaultdict(list)
791     cl_idx_pos: Dict[int, List[int]] = defaultdict(list)
792     for i, (cid, ps) in enumerate(zip(cluster_assignments, pred_sent)):
793         if ps == 1:
794             cl_idx_pos[int(cid)].append(i)
795         else:
796             cl_idx_neg[int(cid)].append(i)
797
798     all_clusters = sorted(set(cluster_assignments.tolist()))
799     n_neg_quota = max_samples_per_cluster // 2
800     n_pos_quota = max_samples_per_cluster - n_neg_quota
801
802     print(f"\n Building sentiment-specific TF-IDF vocab "
803           f"(top {tfidf_vocab_size} per pool) per cluster")
804
805     vocab_neg: Dict[int, List[str]] = {}
806     vocab_pos: Dict[int, List[str]] = {}
807     vocab_neg_set: Dict[int, set] = {}
808     vocab_pos_set: Dict[int, set] = {}
809
810     for cid in all_clusters:
811         neg_indices = cl_idx_neg[cid]
812         pos_indices = cl_idx_pos[cid]
813         neg_mask = np.zeros(len(texts), dtype=bool)
814         pos_mask = np.zeros(len(texts), dtype=bool)
815         if neg_indices: neg_mask[neg_indices] = True
816         if pos_indices: pos_mask[pos_indices] = True
817
818         v_neg = self._get_cluster_tfidf_vocab(texts, neg_mask, top_n=tfidf_vocab_size)
819         v_pos = self._get_cluster_tfidf_vocab(texts, pos_mask, top_n=tfidf_vocab_size)
820
821         vocab_neg[cid] = v_neg
822         vocab_pos[cid] = v_pos
823         vocab_neg_set[cid] = set(w.lower() for w in v_neg)
824         vocab_pos_set[cid] = set(w.lower() for w in v_pos)
825
826         print(f"    Cluster {cid:3d} "
827               f"neg_vocab ({len(v_neg)}): {v_neg[:6]} | "
828               f"pos_vocab ({len(v_pos)}): {v_pos[:6]}")
829
830     stratified: Dict[int, Dict[str, List[int]]] = {}
831     for cid in all_clusters:
832         pool_neg = cl_idx_neg[cid]
833         pool_pos = cl_idx_pos[cid]
834
835         if pool_neg:
836             ranked_neg = self._rank_texts_by_vocab_coverage(
837                 [texts[i] for i in pool_neg], vocab_neg[cid], n_neg_quota)
838             samp_neg = [pool_neg[r] for r in ranked_neg]
839         else:
840             samp_neg = []
841
842         if pool_pos:
843             ranked_pos = self._rank_texts_by_vocab_coverage(
844                 [texts[i] for i in pool_pos], vocab_pos[cid], n_pos_quota)
845             samp_pos = [pool_pos[r] for r in ranked_pos]
846         else:

```

```

847         samp_pos = []
848
849         stratified[cid] = {"neg": samp_neg, "pos": samp_pos}
850
851     dev = next(self.parameters()).device
852     print(f"\n Loading model: {bert_model_name}")
853     print(f" Mode: {'Longformer (global CLS attention)' if is_longformer else 'Standard BERT'}")
854     tokenizer = AutoTokenizer.from_pretrained(bert_model_name)
855     lang_model = AutoModel.from_pretrained(bert_model_name).to(dev).eval()
856
857     cl_neg_pool: Dict[int, Dict[str, List[float]]] = defaultdict(lambda: defaultdict(list))
858     cl_pos_pool: Dict[int, Dict[str, List[float]]] = defaultdict(lambda: defaultdict(list))
859
860     _SKIP_TOKENS = {"[CLS]", "[SEP]", "[PAD]", "<s>", "</s>", "<pad>", "<mask>"}
861
862     self.eval()
863     for cid, pools in sorted(stratified.items()):
864         neg_indices = pools["neg"]
865         pos_indices = pools["pos"]
866         total = len(neg_indices) + len(pos_indices)
867         vset_neg = vocab_neg_set[cid]
868         vset_pos = vocab_pos_set[cid]
869
870         print(f" Cluster {cid:3d} "
871               f"({len(neg_indices)} neg + {len(pos_indices)} pos | "
872               f"neg_vocab={len(vset_neg)} pos_vocab={len(vset_pos)})...",
873               end=" ", flush=True)
874         ok = 0
875
876         for idx in neg_indices:
877             try:
878                 toks, _sc_pos, _sc_neg, _ = self._occlusion_scores_bidirectional(
879                     texts[idx], lang_model, tokenizer, max_length, is_longformer)
880                 for tok, sn in zip(toks, _sc_neg):
881                     if tok in _SKIP_TOKENS: continue
882                     clean = self._clean_token(tok)
883                     if clean.lower() not in vset_neg: continue
884                     cl_neg_pool[cid][clean].append(sn)
885                 ok += 1
886             except Exception:
887                 continue
888
889         for idx in pos_indices:
890             try:
891                 toks, _sc_pos, _sc_neg, _ = self._occlusion_scores_bidirectional(
892                     texts[idx], lang_model, tokenizer, max_length, is_longformer)
893                 for tok, sp in zip(toks, _sc_pos):
894                     if tok in _SKIP_TOKENS: continue
895                     clean = self._clean_token(tok)
896                     if clean.lower() not in vset_pos: continue
897                     cl_pos_pool[cid][clean].append(sp)
898                 ok += 1
899             except Exception:
900                 continue
901
902         print(f" ({ok}/{total} succeeded) "
903               f"neg_tokens={len(cl_neg_pool[cid])} "
904               f"pos_tokens={len(cl_pos_pool[cid])}")
905
906     del lang_model
907
908     def _top(d: Dict[str, List[float]]) -> Dict[str, float]:
909         means = {t: float(np.mean(sc)) for t, sc in d.items() if len(sc) >= 1}
910         return dict(sorted(means.items(), key=lambda x: abs(x[1]), reverse=True)[:top_k])
911
912     result: Dict[int, Dict[str, Dict[str, float]]] = {}
913     for cid in all_clusters:
914         neg_agg = _top(cl_neg_pool[cid])
915         pos_agg = _top(cl_pos_pool[cid])
916         result[cid] = {"neg_pool": neg_agg, "pos_pool": pos_agg}
917         print(f" Cluster {cid:3d} "
918               f"neg_pool ({len(neg_agg)}): {list(neg_agg.keys())[:6]} | "
919               f"pos_pool ({len(pos_agg)}): {list(pos_agg.keys())[:6]}")
920
921     return result
922
923 # -- compute_token_attribution_per_cluster (no embeddings) -----

```

```

924 def compute_token_attribution_per_cluster(
925     self,
926     texts,
927     cluster_assignments,
928     bert_model_name: str = "allenai/longformer-base-4096",
929     max_length: int = 4096,
930     max_samples_per_cluster: int = 15,
931     top_k: int = 10,
932     tfidf_vocab_size: int = 30,
933 ) -> Dict[int, Dict[str, Dict[str, float]]]:
934     is_longformer = "longformer" in bert_model_name.lower()
935     dev = next(self.parameters()).device
936
937     print(f" Loading model: {bert_model_name}")
938     print(f" Mode: {'Longformer (global CLS attention)' if is_longformer else 'Standard BERT'}")
939     tokenizer = AutoTokenizer.from_pretrained(bert_model_name)
940     lang_model = AutoModel.from_pretrained(bert_model_name).to(dev).eval()
941
942     cl_idx: Dict[int, List[int]] = defaultdict(list)
943     for i, cid in enumerate(cluster_assignments):
944         cl_idx[int(cid)].append(i)
945
946     print(f"\n Precomputing TF-IDF vocab (top {tfidf_vocab_size}) per cluster")
947     cluster_vocab: Dict[int, List[str]] = {}
948     for cid, indices in cl_idx.items():
949         mask = np.zeros(len(texts), dtype=bool)
950         mask[indices] = True
951         cluster_vocab[cid] = self._get_cluster_tfidf_vocab(
952             texts, mask, top_n=tfidf_vocab_size)
953
954     cl_tok_pos: Dict[int, Dict[str, List[float]]] = defaultdict(lambda: defaultdict(list))
955     cl_tok_neg: Dict[int, Dict[str, List[float]]] = defaultdict(lambda: defaultdict(list))
956     _SKIP_TOKENS = {"[CLS]", "[SEP]", "[PAD]", "<s>", "</s>", "<pad>", "<mask>"}
957
958     self.eval()
959     for cid, indices in sorted(cl_idx.items()):
960         vocab = cluster_vocab[cid]
961         ranked = self._rank_texts_by_vocab_coverage(
962             [texts[i] for i in indices], vocab, max_samples_per_cluster)
963         sampled = [indices[r] for r in ranked]
964
965         print(f" Cluster {cid:3d} ({len(sampled)} freq-selected samples)...",
966               end=" ", flush=True)
967         ok = 0
968         for idx in sampled:
969             try:
970                 toks, sc_pos, sc_neg, _ = self._occlusion_scores_bidirectional(
971                     texts[idx], lang_model, tokenizer, max_length, is_longformer)
972                 for tok, sp, sn in zip(toks, sc_pos, sc_neg):
973                     if tok in _SKIP_TOKENS: continue
974                     clean = self._clean_token(tok)
975                     if len(clean) < 2: continue
976                     cl_tok_pos[cid][clean].append(sp)
977                     cl_tok_neg[cid][clean].append(sn)
978                 ok += 1
979             except Exception:
980                 continue
981         print(f" ({ok}/{len(sampled)} succeeded)")
982
983     del lang_model
984
985     def _top(d: Dict[str, List[float]]) -> Dict[str, float]:
986         means = {t: float(np.mean(sc)) for t, sc in d.items() if len(sc) >= 2}
987         return dict(sorted(means.items(), key=lambda x: abs(x[1]), reverse=True)[:top_k])
988
989     result: Dict[int, Dict[str, Dict[str, float]]] = {}
990     for cid in sorted(set(list(cl_tok_pos.keys()) + list(cl_tok_neg.keys()))):
991         result[cid] = {
992             "pos_pool": _top(cl_tok_pos[cid]),
993             "neg_pool": _top(cl_tok_neg[cid]),
994         }
995     return result
996
997 # -- _get_pos_neg_pools -----
998 @staticmethod
999 def _get_pos_neg_pools(
1000     entry: Dict

```

```

1001 ) -> Tuple[Dict[str, float], Dict[str, float]]:
1002     if "pos_pool" in entry or "neg_pool" in entry:
1003         return entry.get("pos_pool", {}), entry.get("neg_pool", {})
1004     if "pos" in entry or "neg" in entry:
1005         return entry.get("pos", {}), entry.get("neg", {})
1006     pos = {t: v for t, v in entry.items() if isinstance(v, float) and v > 0}
1007     neg = {t: abs(v) for t, v in entry.items() if isinstance(v, float) and v < 0}
1008     return pos, neg
1009
1010 # -- Plot: POSITIVE pool grid -----
1011 def plot_token_attribution_pos_grid(
1012     self,
1013     cluster_token_scores: Dict[int, Dict[str, Dict[str, float]]],
1014     epoch: int = 0,
1015     save_dir: str = "./results/fnnjst",
1016     figsize_per_cluster: Tuple[float, float] = (4.5, 4.0),
1017     top_k: int = 10,
1018     max_clusters_per_row: int = 4,
1019     save_plot: bool = True,
1020     show_plot: bool = False,
1021 ):
1022     _apply_professional_style()
1023     cids = sorted(cluster_token_scores.keys())
1024     if not cids: return None
1025
1026     ncols = min(max_clusters_per_row, len(cids))
1027     nrows = (len(cids) + ncols - 1) // ncols
1028     fig, axes = plt.subplots(nrows, ncols,
1029                             figsize=(figsize_per_cluster[0] * ncols,
1030                                     figsize_per_cluster[1] * nrows),
1031                             facecolor="#FAFAFA")
1032     axes = np.array(axes).reshape(-1)
1033
1034     for i, cid in enumerate(cids):
1035         ax = axes[i]
1036         pos_d, _ = self._get_pos_neg_pools(cluster_token_scores[cid])
1037
1038         if not pos_d:
1039             ax.text(0.5, 0.5, "No positive data", ha="center", va="center",
1040                    transform=ax.transAxes, color="#AAAAAA", fontsize=9)
1041             ax.axis("off"); continue
1042
1043         tokens = list(reversed(
1044             sorted(pos_d.keys(), key=lambda t: abs(pos_d[t]), reverse=True)[:top_k]))
1045         vals = np.array([pos_d[t] for t in tokens])
1046         ypos = np.arange(len(tokens))
1047         max_abs = max(np.abs(vals).max(), 1e-8)
1048         bar_colors = [plt.cm.Greens(0.35 + 0.55 * (abs(v) / max_abs)) for v in vals]
1049
1050         bars = ax.barh(ypos, vals, color=bar_colors,
1051                       edgecolor="FFFFFF", linewidth=0.5, height=0.72)
1052         ax.set_yticks(ypos)
1053         ax.set_yticklabels(tokens, fontsize=8, fontfamily="monospace")
1054         ax.axvline(0, color="#888888", linewidth=0.7, linestyle="--")
1055         ax.set_xlabel("Mean Attribution Positive", fontsize=8, color="#555")
1056         ax.spines["left"].set_visible(False)
1057         ax.spines["bottom"].set_color("#CCCCCC")
1058         ax.tick_params(axis="y", which="both", length=0)
1059         ax.set_facecolor("FFFFFF")
1060
1061         for bar, v in zip(bars, vals):
1062             if abs(v) > 1e-6:
1063                 ax.text(v + max_abs * 0.02, bar.get_y() + bar.get_height() / 2,
1064                        f"{v:.4f}", va="center", ha="left",
1065                        fontsize=6.5, color="#1A7A4A")
1066
1067         cluster_label = self.topic_mapping.get(cid, f"Cluster {cid}")
1068         ax.set_title(f"{cluster_label}\n({len(pos_d)} tokens)",
1069                    fontsize=9, fontweight="bold", pad=5, color="#1C1C1C")
1070
1071     for j in range(i + 1, len(axes)):
1072         axes[j].axis("off")
1073
1074     fig.suptitle(
1075         f"Token Attribution POSITIVE Pool Epoch {epoch}\n"
1076         f"Tokens driving POSITIVE sentiment frequency-selected samples best model",
1077         fontsize=12, fontweight="bold", color="#1C1C1C", y=1.02

```

```

1078     )
1079     plt.tight_layout(rect=[0, 0, 1, 0.97])
1080
1081     if save_plot:
1082         os.makedirs(save_dir, exist_ok=True)
1083         out = os.path.join(save_dir, f"token_attr_POS_grid_epoch_{epoch}.png")
1084         plt.savefig(out, dpi=200, bbox_inches="tight", facecolor="#FAFAFA")
1085         print(f"Pos-grid saved: {out}")
1086     if show_plot: plt.show()
1087     else:         plt.close()
1088
1089 # -- Plot: NEGATIVE pool grid -----
1090 def plot_token_attribution_neg_grid(
1091     self,
1092     cluster_token_scores: Dict[int, Dict[str, Dict[str, float]]],
1093     epoch: int = 0,
1094     save_dir: str = "./results/fnnjst",
1095     figsize_per_cluster: Tuple[float, float] = (4.5, 4.0),
1096     top_k: int = 10,
1097     max_clusters_per_row: int = 4,
1098     save_plot: bool = True,
1099     show_plot: bool = False,
1100 ):
1101     _apply_professional_style()
1102     cids = sorted(cluster_token_scores.keys())
1103     if not cids: return None
1104
1105     ncols = min(max_clusters_per_row, len(cids))
1106     nrows = (len(cids) + ncols - 1) // ncols
1107     fig, axes = plt.subplots(nrows, ncols,
1108                             figsize=(figsize_per_cluster[0] * ncols,
1109                                     figsize_per_cluster[1] * nrows),
1110                             facecolor="#FAFAFA")
1111     axes = np.array(axes).reshape(-1)
1112
1113     for i, cid in enumerate(cids):
1114         ax = axes[i]
1115         _, neg_d = self._get_pos_neg_pools(cluster_token_scores[cid])
1116
1117         if not neg_d:
1118             ax.text(0.5, 0.5, "No negative data", ha="center", va="center",
1119                    transform=ax.transAxes, color="AAAAAA", fontsize=9)
1120             ax.axis("off"); continue
1121
1122         tokens = list(reversed(
1123             sorted(neg_d.keys(), key=lambda t: abs(neg_d[t]), reverse=True)[:top_k]))
1124         vals = np.array([neg_d[t] for t in tokens])
1125         ypos = np.arange(len(tokens))
1126         max_abs = max(np.abs(vals).max(), 1e-8)
1127         bar_colors = [plt.cm.Reds(0.35 + 0.55 * (abs(v) / max_abs)) for v in vals]
1128
1129         bars = ax.barh(ypos, vals, color=bar_colors,
1130                        edgecolor="FFFFFF", linewidth=0.5, height=0.72)
1131         ax.set_yticks(ypos)
1132         ax.set_yticklabels(tokens, fontsize=8, fontfamily="monospace")
1133         ax.axvline(0, color="#888888", linewidth=0.7, linestyle="--")
1134         ax.set_xlabel("Mean Attribution Negative", fontsize=8, color="#5555")
1135         ax.spines["left"].set_visible(False)
1136         ax.spines["bottom"].set_color("#CCCCC")
1137         ax.tick_params(axis="y", which="both", length=0)
1138         ax.set_facecolor("FFFFFF")
1139
1140         for bar, v in zip(bars, vals):
1141             if abs(v) > 1e-6:
1142                 ax.text(v + max_abs * 0.02, bar.get_y() + bar.get_height() / 2,
1143                        f"{v:.4f}", va="center", ha="left",
1144                        fontsize=6.5, color="#C0392B")
1145
1146         cluster_label = self.topic_mapping.get(cid, f"Cluster {cid}")
1147         ax.set_title(f"{cluster_label}\n({len(neg_d)} tokens)",
1148                    fontsize=9, fontweight="bold", pad=5, color="#1C1C1C")
1149
1150     for j in range(i + 1, len(axes)):
1151         axes[j].axis("off")
1152
1153     fig.suptitle(
1154         f"Token Attribution NEGATIVE Pool Epoch {epoch}\n"

```

```

1155         f"Tokens driving NEGATIVE sentiment    frequency-selected samples    best model",
1156         fontsize=12, fontweight="bold", color="#1C1C1C", y=1.02
1157     )
1158     plt.tight_layout(rect=[0, 0, 1, 0.97])
1159
1160     if save_plot:
1161         os.makedirs(save_dir, exist_ok=True)
1162         out = os.path.join(save_dir, f"token_attr_NEG_grid_epoch_{epoch}.png")
1163         plt.savefig(out, dpi=200, bbox_inches="tight", facecolor="#FAFAFA")
1164         print(f"    Neg-grid saved: {out}")
1165     if show_plot: plt.show()
1166     else:         plt.close()
1167
1168     # -- Plot: Combined bidirectional -----
1169     def plot_token_attribution_per_cluster(
1170         self,
1171         cluster_token_scores: Dict[int, Dict[str, Dict[str, float]]],
1172         epoch: int = 0,
1173         save_dir: str = "./results/fnnjst",
1174         figsize: tuple = (18, 11),
1175         top_k: int = 10,
1176         max_clusters_per_row: int = 4,
1177         save_plot: bool = True,
1178         show_plot: bool = False,
1179     ):
1180         _apply_professional_style()
1181         cids = sorted(cluster_token_scores.keys())
1182         if not cids: return None
1183
1184         ncols = min(max_clusters_per_row, len(cids))
1185         nrows = (len(cids) + ncols - 1) // ncols
1186         fig_h = max(figsize[1], nrows * 4.5)
1187         fig, axes = plt.subplots(nrows, ncols, figsize=(figsize[0], fig_h),
1188                                 facecolor="#FAFAFA")
1189         axes = np.array(axes).reshape(-1)
1190
1191         for i, cid in enumerate(cids):
1192             ax = axes[i]
1193             pos_d, neg_d = self._get_pos_neg_pools(cluster_token_scores[cid])
1194
1195             if not pos_d and not neg_d:
1196                 ax.text(0.5, 0.5, "No attribution data", ha="center", va="center",
1197                         transform=ax.transAxes, color="#AAAAAA", fontsize=9)
1198                 ax.axis("off"); continue
1199
1200             neg_tokens = sorted(neg_d.keys(), key=lambda t: abs(neg_d[t]), reverse=True)[:top_k]
1201             pos_tokens = sorted(pos_d.keys(), key=lambda t: abs(pos_d[t]), reverse=True)[:top_k]
1202             all_tokens = list(reversed(list(dict.fromkeys(neg_tokens + pos_tokens))))
1203             ypos = np.arange(len(all_tokens))
1204             width = 0.35
1205
1206             pos_vals = np.array([pos_d.get(t, 0.0) for t in all_tokens])
1207             neg_vals = np.array([neg_d.get(t, 0.0) for t in all_tokens])
1208
1209             ax.barh(ypos + width / 2, pos_vals, height=width,
1210                    color=COL_POS, alpha=0.85, label=" Positive (pos_pool)",
1211                    edgecolor="FFFFFF", linewidth=0.4)
1212             ax.barh(ypos - width / 2, neg_vals, height=width,
1213                    color=COL_NEG, alpha=0.85, label=" Negative (neg_pool)",
1214                    edgecolor="FFFFFF", linewidth=0.4)
1215
1216             ax.set_yticks(ypos)
1217             ax.set_yticklabels(all_tokens, fontsize=8, fontfamily="monospace")
1218             ax.axvline(0, color="#888888", linewidth=0.8, linestyle="--")
1219             ax.set_xlabel("Mean Occlusion Attribution", fontsize=8, color="#5555")
1220             ax.spines["left"].set_visible(False)
1221             ax.spines["bottom"].set_color("CCCCC")
1222             cluster_label = self.topic_mapping.get(cid, f"Cluster {cid}")
1223             ax.set_title(f"{cluster_label}\n"
1224                          f"neg: {len(neg_tokens)} tok | pos: {len(pos_tokens)} tok",
1225                          fontsize=8, fontweight="bold", pad=5, color="#1C1C1C")
1226             ax.tick_params(axis="y", which="both", length=0)
1227             ax.set_facecolor("FFFFFF")
1228
1229         for j in range(i + 1, len(axes)):
1230             axes[j].axis("off")
1231

```

```

1232     legend_handles = [
1233         mpatches.Patch(facecolor=_COL_POS, alpha=0.85,
1234             label="pos_pool Supports Positive Sentiment"),
1235         mpatches.Patch(facecolor=_COL_NEG, alpha=0.85,
1236             label="neg_pool Supports Negative Sentiment"),
1237     ]
1238     fig.legend(handles=legend_handles, loc="upper center",
1239         bbox_to_anchor=(0.5, 1.01), ncol=2,
1240         fontsize=9, framealpha=0.92, edgecolor="#CCCCCC")
1241     fig.suptitle(
1242         f"Bidirectional Token Sentiment Attribution Epoch {epoch}\n"
1243         f"neg_pool from negative samples pos_pool from positive samples",
1244         fontsize=12, fontweight="bold", color="#1C1C1C", y=1.05
1245     )
1246     plt.tight_layout(rect=[0, 0, 1, 0.97])
1247
1248     if save_plot:
1249         os.makedirs(save_dir, exist_ok=True)
1250         out = os.path.join(save_dir, f"token_attr_per_cluster_epoch_{epoch}.png")
1251         plt.savefig(out, dpi=200, bbox_inches="tight", facecolor="#FAFAFA")
1252         print(f" Combined plot saved: {out}")
1253     if show_plot: plt.show()
1254     else: plt.close()
1255
1256 # -- Plot: Attribution Heatmap -----
1257 def plot_token_attribution_heatmap(
1258     self,
1259     cluster_token_scores: Dict[int, Dict[str, Dict[str, float]]],
1260     epoch: int = 0,
1261     save_dir: str = "./results/fnnjst",
1262     top_k_global: int = 20,
1263     figsize: tuple = (18, 9),
1264     save_plot: bool = True,
1265     show_plot: bool = False,
1266 ):
1267     _apply_professional_style()
1268     cids = sorted(cluster_token_scores.keys())
1269     if not cids: return None
1270
1271     global_pos_sc: Dict[str, List[float]] = defaultdict(list)
1272     global_neg_sc: Dict[str, List[float]] = defaultdict(list)
1273     for cid in cids:
1274         pos_d, neg_d = self._get_pos_neg_pools(cluster_token_scores[cid])
1275         for t, v in pos_d.items(): global_pos_sc[t].append(v)
1276         for t, v in neg_d.items(): global_neg_sc[t].append(v)
1277
1278     top_pos_tokens = [
1279         t for t, _ in sorted(
1280             (t: float(np.mean(np.abs(s))) for t, s in global_pos_sc.items()).items(),
1281             key=lambda x: x[1], reverse=True)[:top_k_global]
1282     ]
1283     top_neg_tokens = [
1284         t for t, _ in sorted(
1285             (t: float(np.mean(np.abs(s))) for t, s in global_neg_sc.items()).items(),
1286             key=lambda x: x[1], reverse=True)[:top_k_global]
1287     ]
1288
1289     mat_pos = np.array([
1290         [self._get_pos_neg_pools(cluster_token_scores[c])[0].get(t, 0.0)
1291          for t in top_pos_tokens] for c in cids
1292     ])
1293     mat_neg = np.array([
1294         [self._get_pos_neg_pools(cluster_token_scores[c])[1].get(t, 0.0)
1295          for t in top_neg_tokens] for c in cids
1296     ])
1297
1298     vmax_pos = max(np.abs(mat_pos).max(), 1e-6)
1299     vmax_neg = max(np.abs(mat_neg).max(), 1e-6)
1300     cluster_labels = [self.topic_mapping.get(c, f"C{c}") for c in cids]
1301
1302     fig, (ax_pos, ax_neg) = plt.subplots(
1303         1, 2, figsize=figsize, facecolor="#FAFAFA",
1304         gridspec_kw={"wspace": 0.08}
1305     )
1306     fig.suptitle(
1307         f"Per-Cluster Token Attribution Heatmap Epoch {epoch}\n"
1308         f"pos_pool (freq-selected positive samples) "

```

```

1309         f"neg_pool (freq-selected negative samples)",
1310         fontsize=12, fontweight="bold", color="#1C1C1C", y=1.02
1311     )
1312
1313     def _draw_panel(ax, matrix, top_tokens, title, vmax_, cmap):
1314         im = ax.imshow(matrix, aspect="auto", cmap=cmap, vmin=0, vmax=vmax_,
1315             interpolation="nearest")
1316         ax.set_xticks(range(len(top_tokens)))
1317         ax.set_xticklabels(top_tokens, rotation=45, ha="right",
1318             fontsize=8, fontfamily="monospace")
1319         ax.set_yticks(range(len(cids)))
1320         ax.set_yticklabels(cluster_labels, fontsize=8)
1321         ax.set_title(title, fontsize=10, fontweight="bold", pad=8, color="#1C1C1C")
1322         ax.set_xlabel("Token", fontsize=9, color="#4444")
1323         ax.set_ylabel("Cluster", fontsize=9, color="#4444")
1324         ax.tick_params(axis="both", length=0)
1325         ax.set_xticks(np.arange(-0.5, len(top_tokens), 1), minor=True)
1326         ax.set_yticks(np.arange(-0.5, len(cids), 1), minor=True)
1327         ax.grid(which="minor", color="#EEEEEE", linewidth=0.8)
1328         if len(top_tokens) <= 25 and len(cids) <= 30:
1329             for i in range(len(cids)):
1330                 for j in range(len(top_tokens)):
1331                     v = matrix[i, j]
1332                     if abs(v) < 1e-6: continue
1333                     txt_col = "white" if v > vmax_ * 0.65 else "#2C2C2C"
1334                     ax.text(j, i, f"{v:.3f}", ha="center", va="center",
1335                         fontsize=5.5, color=txt_col, fontweight="bold")
1336                 cb = plt.colorbar(im, ax=ax, fraction=0.025, pad=0.015, shrink=0.85)
1337                 cb.set_label("Mean Attribution", fontsize=8, color="#4444")
1338                 cb.ax.tick_params(labelsize=7)
1339
1340     _draw_panel(ax_pos, mat_pos, top_pos_tokens,
1341         "pos_pool Positive Sentiment Attribution\n"
1342         "(tokens from freq-selected positive samples)",
1343         vmax_pos, cmap="YlGn")
1344     _draw_panel(ax_neg, mat_neg, top_neg_tokens,
1345         "neg_pool Negative Sentiment Attribution\n"
1346         "(tokens from freq-selected negative samples)",
1347         vmax_neg, cmap="YlOrRd")
1348     ax_neg.set_ylabel("")
1349     ax_neg.set_yticklabels([])
1350
1351     plt.tight_layout()
1352     if save_plot:
1353         os.makedirs(save_dir, exist_ok=True)
1354         out = os.path.join(save_dir, f"token_attr_heatmap_epoch_{epoch}.png")
1355         plt.savefig(out, dpi=200, bbox_inches="tight", facecolor="#FAFAFA")
1356         print(f" Heatmap saved: {out}")
1357     if show_plot: plt.show()
1358     else: plt.close()
1359
1360     # -- Plot: Explain single text -----
1361     def explain_single_text(
1362         self, text,
1363         bert_model_name="allenai/longformer-base-4096",
1364         max_length=4096, figsize=(13, 5),
1365         save_path=None, show=False
1366     ):
1367         _apply_professional_style()
1368         is_longformer = "longformer" in bert_model_name.lower()
1369         dev = next(self.parameters()).device
1370         tokenizer = AutoTokenizer.from_pretrained(bert_model_name)
1371         lang_model = AutoModel.from_pretrained(bert_model_name).to(dev).eval()
1372
1373         tokens, scores_pos, scores_neg, base_prob = \
1374             self._occlusion_scores_bidirectional(
1375                 text, lang_model, tokenizer, max_length, is_longformer)
1376         del lang_model
1377
1378         pred_class = int(base_prob.argmax())
1379         pred_label = {0: "Negative", 1: "Positive"}.get(pred_class, str(pred_class))
1380         pred_conf = float(base_prob.max())
1381
1382         _SKIP_TOKENS = {"[CLS]", "[SEP]", "[PAD]", "<s>", "</s>", "<pad>", "<mask>"}
1383         dt, ds_pos, ds_neg = [], [], []
1384         for tok, sp, sn in zip(tokens, scores_pos, scores_neg):
1385             if tok in _SKIP_TOKENS: continue

```

```

1386         clean = self._clean_token(tok)
1387         if len(clean) < 1: continue
1388         dt.append(clean); ds_pos.append(sp); ds_neg.append(sn)
1389
1390     dt = np.array(dt)
1391     ds_pos = np.array(ds_pos)
1392     ds_neg = np.array(ds_neg)
1393     order = np.argsort(np.abs(ds_pos) + np.abs(ds_neg))[:, -1][:20][::-1]
1394     dt = dt[order]; ds_pos = ds_pos[order]; ds_neg = ds_neg[order]
1395
1396     ypos = np.arange(len(dt))
1397     width = 0.35
1398     fig, ax = plt.subplots(figsize=figsize, facecolor="#FAFAFA")
1399     ax.barh(ypos + width / 2, ds_pos, height=width,
1400            color=_COL_POS, alpha=0.85, label=" Positive Sentiment",
1401            edgecolor="#FFFFFF", linewidth=0.4)
1402     ax.barh(ypos - width / 2, ds_neg, height=width,
1403            color=_COL_NEG, alpha=0.85, label=" Negative Sentiment",
1404            edgecolor="#FFFFFF", linewidth=0.4)
1405     ax.set_yticks(ypos)
1406     ax.set_yticklabels(dt, fontsize=9, fontfamily="monospace")
1407     ax.axvline(0, color="#888888", linewidth=0.8, linestyle="--")
1408     ax.set_xlabel("Occlusion Attribution Score", fontsize=9, color="#555")
1409     ax.set_facecolor("#FFFFFF")
1410     ax.spines["left"].set_visible(False)
1411
1412     badge_col = _COL_POS if pred_class == 1 else _COL_NEG
1413     ax.text(1.02, 0.5, f"Prediction\n{pred_label}\n{pred_conf:.1%}",
1414            transform=ax.transAxes, fontsize=9, va="center", ha="left",
1415            color="white", fontweight="bold",
1416            bbox=dict(boxstyle="round,pad=0.5", facecolor=badge_col, alpha=0.9))
1417     ax.legend(loc="lower right", fontsize=8)
1418     ax.set_title(
1419         f"Token Attribution Analysis      Bidirectional Occlusion\n"
1420         f"Model: {bert_model_name}",
1421         fontsize=11, fontweight="bold", pad=8)
1422     fig.suptitle(f'"{text[:100]}"{"..." if len(text) > 100 else ""}',
1423                fontsize=8, style="italic", color="#666666", y=1.01)
1424     plt.tight_layout()
1425     if save_path:
1426         os.makedirs(os.path.dirname(save_path) if os.path.dirname(save_path) else ".",
1427                    exist_ok=True)
1428         fig.savefig(save_path, dpi=200, bbox_inches="tight", facecolor="#FAFAFA")
1429         print(f" Saved: {save_path}")
1430     if show: plt.show()
1431     else: plt.close()
1432     return fig
1433
1434 # -- Cluster evolution plot -----
1435 def plot_cluster_evolution(
1436     self, embeddings, cluster_assignments, epoch,
1437     texts=None, save_dir="./results/fnnjst", method="tsne",
1438     figsize=(7, 7), point_size=22, alpha=0.7,
1439     save_plot=True, show_plot=False,
1440     plot_tfidf_version=True, max_keywords_per_cluster=3, keyword_min_score=0.3,
1441 ):
1442     _apply_professional_style()
1443     emb = embeddings.detach().cpu().numpy() if isinstance(embeddings, torch.Tensor) else embeddings
1444     n = emb.shape[0]
1445     if n < 3: return None
1446
1447     if method.lower() == 'pca':
1448         r = PCA(n_components=2, random_state=42)
1449         emb_2d = r.fit_transform(emb)
1450         axis_label = f"PCA ({r.explained_variance_ratio_.sum():.1%} variance explained)"
1451     elif method.lower() == "tsne":
1452         from sklearn.manifold import TSNE
1453         r = TSNE(n_components=2, perplexity=max(2, min(30, n - 1)),
1454                 random_state=42, init="pca", learning_rate="auto")
1455         emb_2d = r.fit_transform(emb)
1456         axis_label = "t-SNE"
1457     else:
1458         try:
1459             import umap
1460             r = umap.UMAP(n_components=2, random_state=42,
1461                           n_neighbors=max(2, min(15, n - 1)), min_dist=0.1)
1462             emb_2d = r.fit_transform(emb)

```

```

1463         axis_label = "UMAP"
1464     except Exception:
1465         from sklearn.manifold import TSNE
1466         r = TSNE(n_components=2, perplexity=max(2, min(30, n - 1)),
1467                 random_state=42, init="pca", learning_rate="auto")
1468         emb_2d = r.fit_transform(emb)
1469         axis_label = "t-SNE (fallback)"
1470
1471     uniq = np.unique(cluster_assignments)
1472     k = len(uniq)
1473     colors = [_CLUSTER_PALETTE[i % len(_CLUSTER_PALETTE)] for i in range(k)]
1474
1475     fig, ax = plt.subplots(figsize=figsize, facecolor="#FAFAFA")
1476     for i, cid in enumerate(uniq):
1477         mask = cluster_assignments == cid
1478         label = self.topic_mapping.get(int(cid), f"C{cid}") + f" (n={mask.sum()})"
1479         ax.scatter(emb_2d[mask, 0], emb_2d[mask, 1],
1480                  c=[colors[i]], marker="o", s=point_size, alpha=alpha,
1481                  label=label, edgecolors="white", linewidths=0.3)
1482
1483     ax.set_title(f"Cluster Distribution Epoch {epoch} {axis_label}",
1484                fontsize=11, fontweight="bold", pad=10)
1485     ax.set_xlabel(f"{axis_label} Dim 1", fontsize=8, color="#666")
1486     ax.set_ylabel(f"{axis_label} Dim 2", fontsize=8, color="#666")
1487     ax.set_facecolor("#FFFFFF")
1488     ax.tick_params(colors="#AAAAAA")
1489     ax.grid(True, color="#F0F0F0", linewidth=0.5)
1490     if k <= 15:
1491         ax.legend(fontsize=7, loc="best", framealpha=0.9, edgecolor="#CCCCCC",
1492                 markerscale=1.3, ncol=max(1, k // 8))
1493
1494     plt.tight_layout()
1495     if save_plot:
1496         os.makedirs(save_dir, exist_ok=True)
1497         plt.savefig(os.path.join(save_dir, f"cluster_evolution_epoch_{epoch}.png"),
1498                   dpi=160, bbox_inches="tight", facecolor="#FAFAFA")
1499     if show_plot: plt.show()
1500     else: plt.close()
1501     return fig
1502
1503 # -- K-means init -----
1504 def _init_clusters_with_kmeans(self, all_embeddings, n_init=20, random_state=42):
1505     dev = next(self.parameters()).device
1506     if all_embeddings.size(0) < self.n_clusters:
1507         raise ValueError("n_samples < n_clusters")
1508     self.eval()
1509     with torch.no_grad():
1510         feats = self.extract_feature(all_embeddings).cpu().numpy()
1511         km = KMeans(self.n_clusters, n_init=n_init, random_state=random_state)
1512         y_pred = km.fit_predict(feats)
1513         self.clustering.clusters.data = torch.tensor(
1514             km.cluster_centers_, dtype=torch.float32, device=dev)
1515     return y_pred
1516
1517 # -----
1518 # fit()
1519 # -----
1520 def fit(
1521     self,
1522     dataset,
1523     gamma: float = 1.0, # clustering KL-divergence weight
1524     eta: float = 0.1, # sentiment cross-entropy weight
1525     optimizer_type: str = "adam",
1526     learning_rate: float = 1e-3,
1527     momentum: float = 0.9,
1528     tol: float = 1e-3,
1529     update_interval: int = 140,
1530     batch_size: int = 128,
1531     maxiter: int = int(2e4),
1532     save_dir: str = "./results/fnnjst",
1533     val_ratio: float = 0.1,
1534     val_metric: str = "auto",
1535     plot_evolution: bool = True,
1536     plot_interval: Optional[int] = None,
1537     plot_method: str = "tsne",
1538     compute_metrics: bool = True,
1539     plot_integrated_gradients: bool = True,

```

```

1540     ig_target_class: int = 1,
1541     ig_n_steps:      int = 50,
1542     ig_top_dims:     int = 20,
1543     ig_max_samples:  int = 512,
1544     plot_token_attribution: bool = False,
1545     token_attr_bert_name: str = "allenai/longformer-base-4096",
1546     token_attr_top_k: int = 15,
1547     token_attr_max_samples: int = 30,
1548     token_attr_max_length: int = 4096,
1549     token_attr_tfidf_vocab: int = 30,
1550 ):
1551     """
1552     Joint DEC + Sentiment training (v3.7 no reconstruction loss).
1553
1554     The encoder is updated purely by:
1555         L_cluster (KL divergence to target distribution, DEC-style)
1556         L_sentiment (cross-entropy, when labels are available)
1557
1558     Reconstruction loss belongs in pretrain_autoencoder(), where it is the
1559     sole signal. During multi-task joint training it is redundant and has
1560     been removed.
1561
1562     Fixes retained from v3.6.2:
1563         FIX 1 compute_integrated_gradients uses eval() no BN stat corruption.
1564         FIX 2 final metrics computed BEFORE IG/token-attribution calls.
1565         FIX 4 early stop only after ite >= update_interval (no spurious stop at ite=0).
1566     """
1567     print("=" * 60)
1568     print("SEMTGPU v3.7 Joint Training: Clustering + Sentiment")
1569     print(" (reconstruction loss removed lives in pretrain_autoencoder)")
1570     print(f"Loss weights: (cluster)={gamma}, (sentiment)={eta}")
1571     print(f"Update interval: {update_interval} | val_ratio: {val_ratio:.0%}")
1572     print(f"Best-model metric: {val_metric}")
1573
1574     if plot_token_attribution:
1575         print(f"Token attribution model : {token_attr_bert_name}")
1576         print(f"Token attribution timing: FINAL BEST MODEL ONLY")
1577         print(f" sample budget      = {token_attr_max_samples} total "
1578               f"({(token_attr_max_samples//2)} neg + "
1579               f"{(token_attr_max_samples - token_attr_max_samples//2)} pos per cluster)")
1580         print(f" selection method = frequency-based (TF-IDF vocab top-{token_attr_tfidf_vocab})")
1581         print(f" token filter      = VOCAB-CONSTRAINED")
1582         print(f" top_k per pool      = {token_attr_top_k}")
1583     print("=" * 60)
1584
1585     dev = next(self.parameters()).device
1586     maxiter = int(maxiter)
1587     pinterv = int(plot_interval) if plot_interval is not None else update_interval
1588     os.makedirs(save_dir, exist_ok=True)
1589     plot_dir = os.path.join(save_dir, "evolution_plots")
1590     os.makedirs(plot_dir, exist_ok=True)
1591
1592     # -- Collect dataset -----
1593     embs, lbls, texts_list = [], [], []
1594     for i in range(len(dataset)):
1595         item = dataset[i]
1596         if isinstance(item, tuple):
1597             if len(item) >= 2:
1598                 embs.append(item[0].detach().cpu())
1599                 lbls.append(item[1].detach().cpu())
1600             if len(item) >= 3:
1601                 texts_list.append(item[2])
1602         else:
1603             t = item.detach() if isinstance(item, torch.Tensor) else \
1604                 torch.tensor(item, dtype=torch.float32)
1605             embs.append(t.cpu())
1606
1607     X_all = torch.stack(embs)
1608     N, D = X_all.shape
1609     if D != self.dims[0]: raise ValueError(f"Input dim={D}, dims[0]={self.dims[0]}")
1610     if N < self.n_clusters: raise ValueError(f"n_samples={N} < n_clusters={self.n_clusters}")
1611
1612     has_labels = len(lbls) > 0
1613     has_texts = len(texts_list) > 0
1614
1615     # -- Validation split -----
1616     n_val = max(1, int(N * val_ratio))

```

```

1617     n_train = N - n_val
1618     idx_all = np.random.permutation(N)
1619     idx_train = idx_all[:n_train]
1620     idx_val = idx_all[n_train:]
1621
1622     X_train = X_all[idx_train].to(dev)
1623     X_val = X_all[idx_val].to(dev)
1624     Y_train = torch.stack(lbls[idx_train]).to(dev) if has_labels else None
1625     Y_val = torch.stack(lbls[idx_val]).to(dev) if has_labels else None
1626     texts_train = [texts_list[i] for i in idx_train] if has_texts else []
1627     texts_val = [texts_list[i] for i in idx_val] if has_texts else []
1628
1629     print(f"Dataset split train: {n_train}, val: {n_val}")
1630
1631     if val_metric == "auto":
1632         _primary = "val_f1" if has_labels else "val_cluster_score"
1633     elif val_metric == "f1":
1634         _primary = "val_f1"
1635     elif val_metric in ("cluster_score", "silhouette"):
1636         _primary = "val_cluster_score"
1637     else:
1638         raise ValueError(f"val_metric must be 'auto' | 'f1' | 'cluster_score', got {val_metric}")
1639     print(f"Primary validation metric: {_primary}")
1640
1641     class_w_t = None
1642     if has_labels:
1643         y_tr_np = Y_train.cpu().numpy()
1644         if y_tr_np.ndim == 2 and y_tr_np.shape[1] > 1: y_tr_np = y_tr_np.argmax(1)
1645         cw = self.compute_class_weights(y_tr_np)
1646         class_w_t = torch.tensor([cw.get(i, 1.0) for i in range(2)],
1647                                 dtype=torch.float32, device=dev)
1648
1649     opt_map = {
1650         "adam": lambda: optim.Adam(self.parameters(), lr=learning_rate),
1651         "adamw": lambda: optim.AdamW(self.parameters(), lr=learning_rate),
1652         "sgd": lambda: optim.SGD(self.parameters(), lr=learning_rate, momentum=momentum),
1653         "rmsprop": lambda: optim.RMSprop(self.parameters(), lr=learning_rate),
1654         "adamax": lambda: optim.Adamax(self.parameters(), lr=learning_rate),
1655         "nadam": lambda: optim.NAdam(self.parameters(), lr=learning_rate),
1656         "adagrad": lambda: optim.Adagrad(self.parameters(), lr=learning_rate),
1657         "adadelta": lambda: optim.Adadelta(self.parameters(), lr=learning_rate),
1658         "asgd": lambda: optim.ASGD(self.parameters(), lr=learning_rate),
1659     }
1660     if optimizer_type.lower() not in opt_map:
1661         raise ValueError(f"Unknown optimizer: {optimizer_type}")
1662     optimizer = opt_map[optimizer_type.lower()]()
1663
1664     # Reconstruction loss (MSE) intentionally omitted see pretrain_autoencoder()
1665     kld_loss = nn.KLDivLoss(reduction="batchmean")
1666     ce_loss = nn.CrossEntropyLoss(weight=class_w_t) if class_w_t is not None else \
1667         nn.CrossEntropyLoss()
1668
1669     print("Initialising cluster centres with k-means (train split).")
1670     y_pred_last = self._init_clusters_with_kmeans(X_train)
1671
1672     log_path = os.path.join(save_dir, "idec_sentiment_log.csv")
1673     log_fields = [
1674         "iter", "split",
1675         "train_acc", "train_f1", "train_precision", "train_recall",
1676         "l", "Lc", "Ls", # Lr removed
1677         "ACC", "NMI", "ARI", "Homogeneity", "Completeness", "V-measure",
1678         "Train_Coherence", "Train_Diversity", "Train_Cluster_Score",
1679         "Cluster_Balance", "Min_Cluster_Size", "Max_Cluster_Size",
1680         "val_acc", "val_f1", "val_precision", "val_recall",
1681         "val_nmi", "val_ari", "val_acc_cluster",
1682         "val_coherence", "val_diversity", "val_cluster_score",
1683         "val_primary_score", "is_best",
1684         "IG_TopDim", "IG_TopDim_Score", "IG_Mean_Attribution",
1685     ]
1686
1687     with open(log_path, "w", newline="") as logfile:
1688         writer = csv.DictWriter(logfile, fieldnames=log_fields)
1689         writer.writeheader()
1690
1691         save_interval = max(1, (max(1, n_train // batch_size)) * 5)
1692         train_loader: Optional[DataLoader] = None
1693         self.train()

```

```

1694 iter_count = 0
1695 tot_L = Lc = Ls = 0.0          # Lr removed
1696 last_ite = 0
1697
1698 for ite in range(maxiter):
1699     last_ite = ite
1700
1701     if ite % update_interval == 0:
1702         self.eval()
1703         with torch.no_grad():
1704             q_list, s_list = [], []
1705             for i in range(0, n_train, batch_size):
1706                 qb, sb = self(X_train[i:i+batch_size])
1707                 q_list.append(qb); s_list.append(sb)
1708             q_all = torch.cat(q_list, 0)
1709             s_all = torch.cat(s_list, 0)
1710             p_all = self.target_distribution(q_all)
1711
1712             y_pred = q_all.argmax(1).cpu().numpy()
1713             delta = float((y_pred != y_pred_last).sum() / len(y_pred))
1714             y_pred_last = y_pred.copy()
1715
1716             train_acc = train_f1 = train_prec = train_rec = 0.0
1717             if has_labels:
1718                 s_lab = s_all.argmax(1).cpu().numpy()
1719                 y_true = Y_train.cpu().numpy()
1720                 if y_true.ndim == 2 and y_true.shape[1] > 1:
1721                     y_true = y_true.argmax(1)
1722                 train_acc = float((s_lab == y_true).mean())
1723                 train_f1 = float(f1_score(y_true, s_lab, average="binary", zero_division=0))
1724                 train_prec = float(precision_score(y_true, s_lab, average="binary", zero_division=0))
1725                 train_rec = float(recall_score(y_true, s_lab, average="binary", zero_division=0))
1726
1727             feats_tr = self.extract_feature(X_train).cpu().numpy()
1728             cl_sup_tr = {}
1729             if has_labels:
1730                 yt = Y_train.cpu().numpy()
1731                 if yt.ndim == 2 and yt.shape[1] > 1: yt = yt.argmax(1)
1732                 cl_sup_tr = self.compute_clustering_metrics(yt, y_pred)
1733
1734             train_coh = train_div = 0.0
1735             cov_tr = {'cluster_balance': 0.0, 'min_cluster_size': 0.0,
1736                      'max_cluster_size': 0.0}
1737             if has_texts:
1738                 coh_scores = self.compute_topic_coherence(texts_train, y_pred)
1739                 train_coh = float(np.mean(list(coh_scores.values()))) or 0.0
1740                 train_div = self.compute_topic_diversity(texts_train, y_pred)
1741                 cov_tr = self.compute_topic_coverage(texts_train, y_pred)
1742             train_cluster_score = (train_coh + train_div) / 2.0
1743
1744             avg_L = tot_L / update_interval if iter_count > 0 else 0.0
1745             avg_Lc = Lc / update_interval if iter_count > 0 else 0.0
1746             avg_Ls = Ls / update_interval if iter_count > 0 else 0.0
1747
1748             val_metrics = self._evaluate_val(X_val, Y_val, texts_val, batch_size)
1749             vs = val_metrics["val_primary_score"]
1750             is_best = vs > self._best_val_score
1751             if is_best:
1752                 self._best_val_score = vs
1753                 self._best_val_iter = ite
1754                 self._best_state_dict = copy.deepcopy(self.state_dict())
1755                 print(f" New best model at iter {ite}: {_primary}={vs:.4f}")
1756                 self.save_weights(os.path.join(save_dir, "SEMTGPU_best.weights.pth"))
1757
1758             print(f"\nIter {ite:5d} | Lc={avg_Lc:.5f} Ls={avg_Ls:.5f} L={avg_L:.5f}")
1759             print(f" Sentiment Train Acc={train_acc:.4f} F1={train_f1:.4f} "
1760                   f"P={train_prec:.4f} R={train_rec:.4f}")
1761             print(f" Sentiment Val Acc={val_metrics.get('val_acc_sentiment',0):.4f} "
1762                   f"F1={val_metrics.get('val_f1',0):.4f} "
1763                   f"P={val_metrics.get('val_precision',0):.4f} "
1764                   f"R={val_metrics.get('val_recall',0):.4f}")
1765             print(f" Cluster Train Coh={train_coh:.4f} Div={train_div:.4f} "
1766                   f"Score={train_cluster_score:.4f}")
1767             print(f" Cluster Val Coh={val_metrics.get('val_coherence',0):.4f} "
1768                   f"Div={val_metrics.get('val_diversity',0):.4f} "
1769                   f"Score={val_metrics.get('val_cluster_score',0):.4f} "
1770                   f"{'BEST' if is_best else ''}")

```

```

1771         if cl_sup_tr:
1772             print(f" Supervised Train ACC={cl_sup_tr['ACC']:.4f} "
1773                   f"NMI={cl_sup_tr['NMI']:.4f} ARI={cl_sup_tr['ARI']:.4f}")
1774
1775         if plot_evolution and ite > 0 and (ite % pinterv == 0):
1776             try:
1777                 self.plot_cluster_evolution(
1778                     feats_tr, y_pred, ite,
1779                     texts=texts_train if has_texts else None,
1780                     save_dir=plot_dir, method=plot_method, show_plot=False,
1781                     plot_tfidf_version=has_texts)
1782             except Exception as e:
1783                 print(f" Warning: plot failed at iter {ite}: {e}")
1784
1785         ig_top_dim_idx = ig_top_dim_score = ig_mean_attr = 0
1786         if plot_integrated_gradients and ite > 0 and (ite % pinterv == 0):
1787             try:
1788                 idx_ig = np.random.choice(n_train, min(ig_max_samples, n_train), replace=False)
1789                 ig_attrs = self.compute_integrated_gradients(
1790                     X_train[idx_ig], target_class=ig_target_class, n_steps=ig_n_steps)
1791                 self.plot_integrated_gradients(
1792                     ig_attrs, y_pred[idx_ig], epoch=ite,
1793                     save_dir=plot_dir, top_dims=ig_top_dims, show_plot=False)
1794                 abs_ig = np.abs(ig_attrs)
1795                 gi = abs_ig.mean(0)
1796                 ig_top_dim_idx = int(gi.argmax())
1797                 ig_top_dim_score = float(gi.max())
1798                 ig_mean_attr = float(gi.mean())
1799             except Exception as e:
1800                 print(f" Warning: IG plot failed at iter {ite}: {e}")
1801
1802         writer.writerow({
1803             "iter": ite, "split": "train",
1804             "train_acc": round(train_acc, 5),
1805             "train_f1": round(train_f1, 5),
1806             "train_precision": round(train_prec, 5),
1807             "train_recall": round(train_rec, 5),
1808             "L": round(avg_L, 5),
1809             "Lc": round(avg_Lc, 5),
1810             "Ls": round(avg_Ls, 5),
1811             "ACC": round(cl_sup_tr.get('ACC', 0.0), 5),
1812             "NMI": round(cl_sup_tr.get('NMI', 0.0), 5),
1813             "ARI": round(cl_sup_tr.get('ARI', 0.0), 5),
1814             "Homogeneity": round(cl_sup_tr.get('Homogeneity', 0.0), 5),
1815             "Completeness": round(cl_sup_tr.get('Completeness', 0.0), 5),
1816             "V-measure": round(cl_sup_tr.get('V-measure', 0.0), 5),
1817             "Train_Coherence": round(train_coh, 5),
1818             "Train_Diversity": round(train_div, 5),
1819             "Train_Cluster_Score": round(train_cluster_score, 5),
1820             "Cluster_Balance": round(cov_tr['cluster_balance'], 5),
1821             "Min_Cluster_Size": round(cov_tr['min_cluster_size'], 5),
1822             "Max_Cluster_Size": round(cov_tr['max_cluster_size'], 5),
1823             "val_acc": round(val_metrics.get('val_acc_sentiment', 0.0), 5),
1824             "val_f1": round(val_metrics.get('val_f1', 0.0), 5),
1825             "val_precision": round(val_metrics.get('val_precision', 0.0), 5),
1826             "val_recall": round(val_metrics.get('val_recall', 0.0), 5),
1827             "val_nmi": round(val_metrics.get('val_nmi', 0.0), 5),
1828             "val_ari": round(val_metrics.get('val_ari', 0.0), 5),
1829             "val_acc_cluster": round(val_metrics.get('val_acc_cluster', 0.0), 5),
1830             "val_coherence": round(val_metrics.get('val_coherence', 0.0), 5),
1831             "val_diversity": round(val_metrics.get('val_diversity', 0.0), 5),
1832             "val_cluster_score": round(val_metrics.get('val_cluster_score', 0.0), 5),
1833             "val_primary_score": round(val_metrics.get('val_primary_score', 0.0), 5),
1834             "is_best": int(is_best),
1835             "IG_TopDim": ig_top_dim_idx,
1836             "IG_TopDim_Score": round(ig_top_dim_score, 6),
1837             "IG_Mean_Attribution": round(ig_mean_attr, 6),
1838         })
1839
1840         tot_L = Lc = Ls = 0.0 # Lr removed
1841         iter_count = 0
1842
1843         # FIX 4: only trigger early stop after at least one full update interval
1844         if ite >= update_interval and delta < tol:
1845             print(f" label={delta:.6f} < tol={tol} early stop.")
1846             break
1847

```

```

1848         if has_labels:
1849             train_loader = DataLoader(
1850                 TensorDataset(X_train, p_all, Y_train),
1851                 batch_size=batch_size, shuffle=True)
1852         else:
1853             train_loader = DataLoader(
1854                 TensorDataset(X_train, p_all),
1855                 batch_size=batch_size, shuffle=True)
1856         self.train()
1857
1858         assert train_loader is not None
1859         for batch in tqdm(train_loader, desc=f"Train {ite}", leave=False):
1860             if has_labels and len(batch) == 3:
1861                 xb, pb, yb = batch
1862                 if yb.dim() > 1 and yb.shape[1] > 1: yb = yb.argmax(1)
1863                 yb = yb.long().to(dev)
1864             else:
1865                 xb, pb = batch; yb = None
1866                 xb = xb.to(dev); pb = pb.to(dev)
1867
1868             # Forward: encoder only no decode(), no reconstruction loss
1869             z = self.autoencoder.encode(xb)
1870             q = self.clustering(z)
1871             s = torch.softmax(self.sentiment(z), 1)
1872
1873             cl = kld_loss((q + 1e-8).log(), pb)
1874             sl = ce_loss(s, yb) if yb is not None else \
1875                 torch.zeros(1, device=dev).squeeze()
1876
1877             loss = gamma * cl + eta * sl # no alpha * rl
1878
1879             optimizer.zero_grad(); loss.backward(); optimizer.step()
1880             tot_L += float(loss)
1881             Lc += float(cl)
1882             Ls += float(sl)
1883             iter_count += 1
1884
1885             if ite % save_interval == 0 and ite > 0:
1886                 self.save_weights(os.path.join(save_dir, f"SEMTGPU_{ite}.weights.pth"))
1887
1888         # -- Post-training -----
1889         print("\n" + "=" * 60)
1890         print("Training complete. Restoring best model checkpoint")
1891         self.load_best_weights()
1892
1893         best_summary = {
1894             "best_iter": self._best_val_iter,
1895             "best_val_score": round(self._best_val_score, 5),
1896             "primary_metric": _primary,
1897             "val_ratio": val_ratio,
1898             "n_train": n_train,
1899             "n_val": n_val,
1900         }
1901         with open(os.path.join(save_dir, "best_model.json"), "w") as f:
1902             json.dump(best_summary, f, indent=2)
1903         print(f"Best model info saved {save_dir}/best_model.json")
1904         print(f"Best iter={self._best_val_iter}, {_primary}={self._best_val_score:.4f}")
1905         self.save_weights(os.path.join(save_dir, "SEMTGPU_final.weights.pth"))
1906
1907         # -- Final cluster assignments -----
1908         self.eval()
1909         with torch.no_grad():
1910             q_all_f = torch.cat([self(X_train[i:i+batch_size])[0]
1911                                 for i in range(0, n_train, batch_size)], 0)
1912             y_final = q_all_f.argmax(1).cpu().numpy()
1913
1914         # -- FIX 2: Final metrics BEFORE IG / token-attribution -----
1915         self.eval()
1916         with torch.no_grad():
1917             qs, ss = [], []
1918             for i in range(0, n_train, batch_size):
1919                 qb, sb = self(X_train[i:i+batch_size])
1920                 qs.append(qb); ss.append(sb)
1921             q_all_f2 = torch.cat(qs, 0); s_all_f = torch.cat(ss, 0)
1922             y_pred_cl = q_all_f2.argmax(1).cpu().numpy()
1923             y_pred_sent = s_all_f.argmax(1).cpu().numpy()
1924

```

```

1925 metrics = {}
1926 if has_labels:
1927     y_true = Y_train.cpu().numpy()
1928     if y_true.ndim == 2 and y_true.shape[1] > 1: y_true = y_true.argmax(1)
1929     metrics["sentiment"] = {
1930         "accuracy": float((y_pred_sent == y_true).mean()),
1931         "precision": float(precision_score(y_true, y_pred_sent,
1932                                           average="binary", zero_division=0)),
1933         "recall": float(recall_score(y_true, y_pred_sent,
1934                                    average="binary", zero_division=0)),
1935         "f1_score": float(f1_score(y_true, y_pred_sent,
1936                                  average="binary", zero_division=0)),
1937     }
1938     print("\n" + "=" * 60)
1939     print("\nFINAL (BEST MODEL) SENTIMENT METRICS TRAIN SPLIT")
1940     print("=" * 60)
1941     for k, v in metrics["sentiment"].items():
1942         print(f" {k:12s}: {v:.4f}")
1943     print("=" * 60)
1944
1945 val_final = self._evaluate_val(X_val, Y_val, texts_val, batch_size)
1946 metrics["val_final"] = val_final
1947 print("\nFINAL VAL METRICS (best model):")
1948 print(f" Sentiment Acc={val_final.get('val_acc_sentiment',0):.4f} "
1949       f"F1={val_final.get('val_f1',0):.4f} "
1950       f"P={val_final.get('val_precision',0):.4f} "
1951       f"R={val_final.get('val_recall',0):.4f}")
1952 print(f" Cluster Coh={val_final.get('val_coherence',0):.4f} "
1953       f"Div={val_final.get('val_diversity',0):.4f} "
1954       f"Score={val_final.get('val_cluster_score',0):.4f}")
1955
1956 # -- Token attribution (after metrics FIX 2) -----
1957 if plot_token_attribution and has_texts:
1958     try:
1959         print(f"\n[Token Attribution Final Best Model (v3.7)]")
1960         ta_final = self.compute_token_attribution_per_cluster_with_embeddings(
1961             texts_train, X_train.cpu().numpy(), y_final,
1962             bert_model_name=token_attr_bert_name,
1963             top_k=token_attr_top_k,
1964             max_length=token_attr_max_length,
1965             max_samples_per_cluster=token_attr_max_samples,
1966             tfidf_vocab_size=token_attr_tfidf_vocab,
1967         )
1968         ta_dir = os.path.join(save_dir, "token_attribution")
1969         os.makedirs(ta_dir, exist_ok=True)
1970         self.plot_token_attribution_pos_grid(
1971             ta_final, epoch=last_ite, save_dir=ta_dir,
1972             top_k=token_attr_top_k, show_plot=False)
1973         self.plot_token_attribution_neg_grid(
1974             ta_final, epoch=last_ite, save_dir=ta_dir,
1975             top_k=token_attr_top_k, show_plot=False)
1976         self.plot_token_attribution_per_cluster(
1977             ta_final, epoch=last_ite, save_dir=ta_dir,
1978             top_k=token_attr_top_k, show_plot=False)
1979         self.plot_token_attribution_heatmap(
1980             ta_final, epoch=last_ite, save_dir=ta_dir, show_plot=False)
1981         ta_json_path = os.path.join(ta_dir, "token_attribution_final.json")
1982         with open(ta_json_path, "w") as f:
1983             json.dump({str(k): v for k, v in ta_final.items()}, f, indent=2)
1984         print(f"\n Token attribution complete {ta_dir}")
1985     except Exception as e:
1986         import traceback
1987         print(f"Warning: final token attribution failed: {e}")
1988         traceback.print_exc()
1989
1990 # -- Integrated Gradients (after metrics FIX 2) -----
1991 if plot_integrated_gradients and has_labels:
1992     try:
1993         idx_ig = np.random.choice(n_train, min(ig_max_samples, n_train), replace=False)
1994         ig_f = self.compute_integrated_gradients(
1995             X_train[idx_ig], target_class=ig_target_class, n_steps=ig_n_steps)
1996         self.plot_integrated_gradients(
1997             ig_f, y_final[idx_ig], epoch=last_ite,
1998             save_dir=plot_dir, top_dims=ig_top_dims, show_plot=False)
1999         print(" Final IG plot saved.")
2000     except Exception as e:
2001         print(f"Warning: final IG failed: {e}")

```

```

2002
2003     if has_labels:
2004         return y_pred_cl, s_all_f.cpu().numpy(), metrics
2005     return y_pred_cl

```

Listing 2: Full implementation of JDST-DEC (SEMTGPU).

JDST DEC Training Loop (IMDB and Yelp)

```

1  """
2  SEMT Training Script - IMDB & Yelp
3  Loop over 2 datasets x 3 n_clusters configurations.
4  Results printed as a table and saved to results_summary.csv.
5  """
6
7  import subprocess
8  import re
9  import pandas as pd
10 from sklearn.feature_extraction.text import ENGLISH_STOP_WORDS
11
12 # Clone SEMT repo (skip if already cloned)
13 subprocess.run(
14     ["git", "clone", "https://github.com/zzeiidann/SEMT.git"],
15     check=False,
16 )
17
18 from SEMT.SEMT_Gpu import SEMTGPU, CachedBERTDataset # noqa: E402
19
20
21 # Config
22
23 BERT_MODEL = "bert-base-uncased"
24 MAX_LENGTH = 512
25 DIMS = [768, 2022, 2022, 256]
26 N_CLUSTERS_LIST = [30, 50, 80]
27 PRETRAIN_EPOCHS = 30
28 PRETRAIN_BATCH = 128
29
30 FIT_PARAMS = dict(
31     tol = 1e-100,
32     update_interval = 10,
33     learning_rate = 1e-3,
34     compute_metrics = True,
35     gamma = 0.5,
36     eta = 0.5,
37     maxiter = 100,
38     batch_size = 256,
39     val_ratio = 0.2,
40     token_attr_bert_name = BERT_MODEL,
41 )
42
43 # Dataset registry - name, url, text column, label column, label mapping
44 DATASETS = [
45     {
46         "name": "IMDB",
47         "url": "https://raw.githubusercontent.com/zzeiidann/Data/main/IMDB%20Dataset.csv",
48         "text_col": "review",
49         "label_col": "sentiment",
50         "label_map": {"positive": 1, "negative": 0},
51     },
52     {
53         "name": "Yelp",
54         "url": "https://raw.githubusercontent.com/zzeiidann/Data/refs/heads/main/yelp_sentiment_30k_balanced.csv",
55         "text_col": "review",
56         "label_col": "sentiment",
57         "label_map": {"positive": 1, "negative": 0},
58     },
59 ]
60
61 BASE_STOP = set(ENGLISH_STOP_WORDS).union({
62     "the", "and", "to", "was", "it", "of", "is", "in",

```

```

63     "for", "that", "we", "be", "are", "has", "have",
64     "this", "with", "but", "not", "he", "she", "they",
65     "his", "her", "their", "an", "at", "by", "from",
66     "good", "great", "bad", "really", "just", "like",
67     "love", "hate", "make", "made", "get", "one", "even",
68     "still", "well", "time", "way", "think", "people",
69     "much", "many", "back", "watch",
70 })
71
72 DOMAIN_STOP = {
73     "IMDB": {
74         "movie", "film", "story", "character", "characters",
75         "scene", "scenes", "plot", "show", "act", "acting",
76         "actor", "actress", "director", "cinema", "screen",
77         "role", "cast", "watched", "films", "movies", "series",
78         "episode", "sequel", "ending", "version", "seen", "part",
79     },
80     "Yelp": {
81         "restaurant", "food", "place", "order", "ordered", "came",
82         "got", "went", "come", "going", "said", "told", "asked",
83         "service", "staff", "table", "menu", "wait", "waitress",
84         "waiter", "eat", "eating", "lunch", "dinner", "breakfast",
85         "price", "prices", "experience", "location", "area",
86     },
87 }
88
89
90 # Text cleaning
91
92 def clean_text(text: str) -> str:
93     text = str(text).lower()
94     text = re.sub(r"<br\s*/?>", " ", text)
95     text = re.sub(r"http\S+|www\S+", " ", text)
96     text = re.sub(r"&\w+", "", text)
97     text = re.sub(r"\.{2,}", "", text)
98     text = re.sub(r"^[a-z\s]", "", text)
99     text = re.sub(r"s+", "", text).strip()
100     return text
101
102
103 # Load and preprocess dataset
104
105 def load_dataset(cfg: dict) -> tuple:
106     """Return (texts, labels) Series ready for CachedBERTDataset."""
107     print(f"Loading {cfg['name']} ...")
108     df = pd.read_csv(cfg["url"])
109
110     if cfg["label_map"]:
111         df[cfg["label_col"]] = df[cfg["label_col"]].map(cfg["label_map"])
112
113     df[cfg["text_col"]] = df[cfg["text_col"]].apply(clean_text)
114
115     texts = df[cfg["text_col"]].reset_index(drop=True)
116     labels = df[cfg["label_col"]].reset_index(drop=True)
117     print(f"{cfg['name']} ready - {len(df):,} rows")
118     return texts, labels
119
120
121 # Main loop: dataset -> n_clusters
122
123 all_results = []
124
125 for ds_cfg in DATASETS:
126     ds_name = ds_cfg["name"]
127     print(f"\n[DATASET: {ds_name}]")
128
129     texts, labels = load_dataset(ds_cfg)
130     stop_words = BASE_STOP | DOMAIN_STOP.get(ds_name, set())
131
132     print(f"Building CachedBERTDataset ({BERT_MODEL}) ...")
133     dataset = CachedBERTDataset(
134         texts = texts,
135         labels = labels,
136         bert_model = BERT_MODEL,
137         max_length = MAX_LENGTH,
138         cuda = True,
139         testing_mode = False,

```

```

140         return_texts = True,
141     )
142     print("Embeddings cached.\n")
143
144     for n_clusters in N_CLUSTERS_LIST:
145         print(f"[{ds_name}] n_clusters = {n_clusters}")
146
147         model = SEMTGPU(dims=DIMS, n_clusters=n_clusters)
148         model.set_stop_words(stop_words)
149
150         print(f"Pre-training autoencoder (epochs={PRETRAIN_EPOCHS}) ...")
151         model.pretrain_autoencoder(dataset, epochs=PRETRAIN_EPOCHS, batch_size=PRETRAIN_BATCH)
152
153         print("Fitting SEMT ...")
154         y_pred, s_probs, metrics = model.fit(dataset=dataset, **FIT_PARAMS)
155
156         row = {"dataset": ds_name, "n_clusters": n_clusters}
157         if isinstance(metrics, dict):
158             row.update(metrics)
159         all_results.append(row)
160         print(f"Done. Metrics: {metrics}\n")
161
162
163 # Results table
164
165 print("\nRESULTS SUMMARY")
166
167 results_df = (
168     pd.DataFrame(all_results)
169     .set_index(["dataset", "n_clusters"])
170     .sort_index()
171 )
172
173 print(results_df.to_string(float_format=lambda x: f"{x:.4f}"))
174
175 output_csv = "results_summary.csv"
176 results_df.to_csv(output_csv)
177 print(f"\nSaved -> {output_csv}")

```

Listing 3: Full implementation of JDST DEC training loop.

BERT Topic Evaluation Loop (IMDB and Yelp)

```

1  """
2  BERTopic Training Script - IMDB & Yelp
3  Loop over 2 datasets x 3 n_topics configurations.
4  Results printed as a table and saved to bertopic_results.csv.
5  """
6
7  import re
8  import random
9  import os
10 import numpy as np
11 import pandas as pd
12 import torch
13 from itertools import combinations
14 from collections import defaultdict
15 from sklearn.feature_extraction.text import ENGLISH_STOP_WORDS, CountVectorizer
16 from sentence_transformers import SentenceTransformer
17 from bertopic import BERTopic
18 import umap
19 import hdbscan
20
21 # Reproducibility
22 SEED = 42
23 random.seed(SEED)
24 np.random.seed(SEED)
25 os.environ["PYTHONHASHSEED"] = str(SEED)
26 torch.manual_seed(SEED)
27 if torch.cuda.is_available():
28     torch.cuda.manual_seed_all(SEED)

```

```

29
30
31 # Config
32
33 BERT_MODEL = "bert-base-uncased"
34 N_TOPICS_LIST = [30, 50, 80]
35 SAMPLE_FRAC = 1.0 # pakai semua data, set < 1.0 kalau mau subsample
36
37 UMAP_PARAMS = dict(
38     n_neighbors = 15,
39     n_components = 5,
40     min_dist = 0.0,
41     metric = "cosine",
42     random_state = SEED,
43 )
44
45 HDBSCAN_PARAMS = dict(
46     min_cluster_size = 10,
47     min_samples = 5,
48     metric = "euclidean",
49     cluster_selection_method="eom",
50     prediction_data = True,
51 )
52
53 VECTORIZER_PARAMS = dict(
54     stop_words = "english",
55     ngram_range = (1, 2),
56     min_df = 10,
57     max_df = 0.8,
58 )
59
60 DATASETS = [
61     {
62         "name": "IMDB",
63         "url": "https://raw.githubusercontent.com/zzeidann/Data/main/IMDB%20Dataset.csv",
64         "text_col": "review",
65         "label_col": "sentiment",
66         "label_map": {"positive": 1, "negative": 0},
67     },
68     {
69         "name": "Yelp",
70         "url": "https://raw.githubusercontent.com/zzeidann/Data/refs/heads/main/yelp_sentiment_30k_balanced.csv",
71         "text_col": "review",
72         "label_col": "sentiment",
73         "label_map": {"positive": 1, "negative": 0},
74     },
75 ]
76
77 BASE_STOP = set(ENGLISH_STOP_WORDS).union({
78     "the", "and", "to", "was", "it", "of", "is", "in",
79     "for", "that", "we", "be", "are", "has", "have",
80     "this", "with", "but", "not", "he", "she", "they",
81     "his", "her", "their", "an", "at", "by", "from",
82     "good", "great", "bad", "really", "just", "like",
83     "love", "hate", "make", "made", "get", "one", "even",
84     "still", "well", "time", "way", "think", "people",
85     "much", "many", "back", "watch",
86 })
87
88 DOMAIN_STOP = {
89     "IMDB": {
90         "movie", "film", "story", "character", "characters",
91         "scene", "scenes", "plot", "show", "act", "acting",
92         "actor", "actress", "director", "cinema", "screen",
93         "role", "cast", "watched", "films", "movies", "series",
94         "episode", "sequel", "ending", "version", "seen", "part",
95     },
96     "Yelp": {
97         "restaurant", "food", "place", "order", "ordered", "came",
98         "got", "went", "come", "going", "said", "told", "asked",
99         "service", "staff", "table", "menu", "wait", "waitress",
100         "waiter", "eat", "eating", "lunch", "dinner", "breakfast",
101         "price", "prices", "experience", "location", "area",
102     },
103 }
104
105

```

```

106 # Text cleaning
107
108 def clean_text(text: str) -> str:
109     text = str(text).lower()
110     text = re.sub(r"<br\s*/?>", " ", text)
111     text = re.sub(r"http\S+|www\S+", " ", text)
112     text = re.sub(r"&\w+", " ", text)
113     text = re.sub(r"\.[2,]", " ", text)
114     text = re.sub(r"^[a-z\s]", " ", text)
115     text = re.sub(r"\s+", " ", text).strip()
116     return text
117
118 def tokenize(text: str, stopwords: set) -> str:
119     tokens = [w for w in clean_text(text).split() if w not in stopwords and len(w) > 2]
120     return " ".join(tokens)
121
122
123 # Load dataset
124
125 def load_dataset(cfg: dict, stopwords: set) -> tuple:
126     """Return (texts_raw, texts_clean, labels) ready for BERTopic."""
127     print(f>Loading {cfg['name']} ...")
128     df = pd.read_csv(cfg["url"])
129
130     if cfg["label_map"]:
131         df[cfg["label_col"]] = df[cfg["label_col"]].map(cfg["label_map"])
132
133     if SAMPLE_FRAC < 1.0:
134         df = df.sample(frac=SAMPLE_FRAC, random_state=SEED).reset_index(drop=True)
135
136     texts_raw = df[cfg["text_col"]].astype(str).tolist()
137     texts_clean = [tokenize(t, stopwords) for t in texts_raw]
138     labels = df[cfg["label_col"]].tolist()
139
140     print(f>{cfg['name']} ready - {len(df):,} rows")
141     return texts_raw, texts_clean, labels
142
143
144 # Evaluation metrics
145
146 def tc_npmi_for_topic(words: list, topic_docs: list) -> float:
147     """Compute TC-NPMI for a single topic (identical to SEMENTGPU's compute_topic_coherence)."""
148     if len(words) < 2 or len(topic_docs) == 0:
149         return 0.0
150
151     n = len(topic_docs)
152     df_w = {w: sum(1 for doc in topic_docs if w in doc.lower()) for w in words}
153
154     npmi_scores = []
155     for w1, w2 in combinations(words, 2):
156         co = sum(1 for doc in topic_docs if w1 in doc.lower() and w2 in doc.lower())
157         if co > 0 and df_w[w1] > 0 and df_w[w2] > 0:
158             p12 = co / n
159             p1 = df_w[w1] / n
160             p2 = df_w[w2] / n
161             pmi = np.log((p12 + 1e-10) / (p1 * p2 + 1e-10))
162             npmi = pmi / (-np.log(p12 + 1e-10))
163             npmi_scores.append(npmi)
164
165     return float(np.mean(npmi_scores)) if npmi_scores else 0.0
166
167
168 def compute_metrics(topic_model, texts_raw: list, topics: list, top_n: int = 10) -> dict:
169     """Return n_topics, TC_NPMI, Diversity, Coverage."""
170     topic_ids = [t for t in topic_model.get_topics().keys() if t != -1]
171     topic_words = [[w for w, _ in topic_model.get_topic(tid)[:top_n]] for tid in topic_ids]
172
173     # Diversity
174     all_words = [w for tw in topic_words for w in tw]
175     diversity = len(set(all_words)) / (len(topic_ids) * top_n) if topic_ids else 0.0
176
177     # Coverage
178     topics_arr = np.array(topics)
179     coverage = float(np.sum(topics_arr != -1) / len(topics_arr))
180
181     # TC-NPMI
182     docs_per_topic = defaultdict(list)

```

```

183     for doc, tid in zip(texts_raw, topics):
184         if tid != -1:
185             docs_per_topic[tid].append(doc)
186
187     tc_npmi_vals = [
188         tc_npmi_for_topic(words, docs_per_topic.get(tid, []))
189         for tid, words in zip(topic_ids, topic_words)
190     ]
191     tc_npmi_mean = float(np.mean(tc_npmi_vals)) if tc_npmi_vals else 0.0
192
193     return {
194         "n_topics" : len(topic_ids),
195         "TC_NPMI" : tc_npmi_mean,
196         "Diversity": diversity,
197         "Coverage" : coverage,
198     }
199
200 # Main loop: dataset -> n_topics
201
202 all_results = []
203
204 for ds_cfg in DATASETS:
205     ds_name = ds_cfg["name"]
206     stopwords = BASE_STOP | DOMAIN_STOP.get(ds_name, set())
207
208     print(f"\n[DATASET: {ds_name}]\n")
209     texts_raw, texts_clean, labels = load_dataset(ds_cfg, stopwords)
210
211     # Embed once per dataset
212     print(f"Encoding embeddings with {BERT_MODEL} ...")
213     embedding_model = SentenceTransformer(BERT_MODEL)
214     embeddings = embedding_model.encode(texts_raw, show_progress_bar=True, batch_size=64)
215     print("Embeddings done.\n")
216
217     for n_topics in N_TOPICS_LIST:
218         print(f"[{ds_name}] n_topics = {n_topics}")
219
220         umap_model = umap.UMAP(**UMAP_PARAMS)
221         hdbscan_model = hdbscan.HDBSCAN(**HDBSCAN_PARAMS)
222         vectorizer = CountVectorizer(**VECTORIZER_PARAMS)
223
224         topic_model = BERTopic(
225             embedding_model = embedding_model,
226             umap_model = umap_model,
227             hdbscan_model = hdbscan_model,
228             vectorizer_model = vectorizer,
229             calculate_probabilities = False,
230             verbose = False,
231         )
232
233         print("Fitting BERTopic ...")
234         topics, _ = topic_model.fit_transform(texts_raw, embeddings=embeddings)
235
236         raw_n = len([t for t in set(topics) if t != -1])
237         print(f"Initial topics found: {raw_n}, reducing to {n_topics} ...")
238
239         reduced_model = topic_model.reduce_topics(texts_raw, nr_topics=n_topics)
240         reduced_topics = reduced_model.topics_
241
242         metrics = compute_metrics(reduced_model, texts_raw, reduced_topics)
243         print(f"Done. Metrics: {metrics}\n")
244
245         row = {"dataset": ds_name, "nr_topics": n_topics}
246         row.update(metrics)
247         all_results.append(row)
248
249 # Results table
250
251 print("\nRESULTS SUMMARY")
252
253 results_df = (
254     pd.DataFrame(all_results)
255     .set_index(["dataset", "nr_topics"])
256     .sort_index()
257 )

```

```

260
261 print(results_df.to_string(float_format=lambda x: f"{x:.4f}"))
262
263 output_csv = "bertopic_results.csv"
264 results_df.to_csv(output_csv)
265 print(f"\nSaved -> {output_csv}")

```

Listing 4: Full implementation of BERT Topic evaluation loop.

MLTGUI Training Loop (IMDB and Yelp)

```

1  """
2  MLT JointModel Training Script - IMDB & Yelp
3  HAN + Neural Topic Model (VAE/NGTM) trained jointly.
4  Loop over 2 datasets x 3 K-topics configurations.
5  Results saved to mlt_results.csv and mlt_results.png.
6  """
7
8  import re
9  import math
10 import random
11 import os
12 import numpy as np
13 import pandas as pd
14 from collections import Counter, defaultdict
15 from itertools import combinations
16
17 import torch
18 import torch.nn as nn
19 import torch.nn.functional as F
20 from torch.utils.data import Dataset, DataLoader
21 from torch.nn.utils.rnn import pack_padded_sequence, pad_packed_sequence
22
23 from sklearn.feature_extraction.text import ENGLISH_STOP_WORDS
24 from sklearn.metrics import accuracy_score, f1_score, precision_score, recall_score
25 from sklearn.preprocessing import LabelEncoder
26
27 from tqdm import tqdm
28 import matplotlib.pyplot as plt
29
30 DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
31 print(f"Device: {DEVICE}")
32
33 SEED = 42
34 random.seed(SEED)
35 np.random.seed(SEED)
36 os.environ["PYTHONHASHSEED"] = str(SEED)
37 torch.manual_seed(SEED)
38 if torch.cuda.is_available():
39     torch.cuda.manual_seed_all(SEED)
40
41
42 # Config
43
44 NUM_TOPICS_LIST = [30, 50, 80]
45 EPOCHS = 50
46 BATCH_SIZE = 32
47 TOPN = 10
48
49 DATASETS = [
50     {
51         "name": "IMDB",
52         "url": "https://raw.githubusercontent.com/zzeidann/Data/main/IMDB%20Dataset.csv",
53         "text_col": "review",
54         "label_col": "sentiment",
55         "label_map": {"positive": 1, "negative": 0},
56     },
57     {
58         "name": "Yelp",
59         "url": "https://raw.githubusercontent.com/zzeidann/Data/refs/heads/main/yelp_sentiment_30k_balanced.csv",
60         "text_col": None, # auto-detected below

```

```

61         "label_col": None,
62         "label_map": None,
63     },
64 ]
65
66
67 # Preprocessing
68
69 STOPWORDS = set(ENGLISH_STOP_WORDS)
70
71 def clean_text(text: str) -> str:
72     text = str(text).lower()
73     text = re.sub(r"<br\s*/?>", " ", text)
74     text = re.sub(r"http\S+|www\S+", " ", text)
75     text = re.sub(r"&w+;", " ", text)
76     text = re.sub(r"\.{2,}", " ", text)
77     text = re.sub(r"^[a-z\s]", " ", text)
78     text = re.sub(r"\s+", " ", text).strip()
79     return text
80
81 def tokenize(text: str):
82     text = clean_text(text)
83     return [w for w in text.split() if w not in STOPWORDS and len(w) > 2]
84
85 def split_into_sentences(text: str, max_sent_len: int = 50, max_sents: int = 10):
86     raw_sents = re.split(r"(?<=[!?.])\s+", text)
87     sents = []
88     for s in raw_sents:
89         tokens = tokenize(s)
90         if tokens:
91             sents.append(tokens[:max_sent_len])
92         if len(sents) >= max_sents:
93             break
94     if not sents:
95         tokens = tokenize(text)
96         sents = [tokens[:max_sent_len]] if tokens else [{"unknown"}]
97     return sents
98
99
100 # Load datasets
101
102 def load_dataset(cfg: dict) -> pd.DataFrame:
103     print(f"Loading {cfg['name']} ...")
104     df = pd.read_csv(cfg["url"])
105     df.columns = [c.strip().lower() for c in df.columns]
106
107     if cfg["text_col"] is None:
108         text_col = [c for c in df.columns if "review" in c or "text" in c][0]
109     else:
110         text_col = cfg["text_col"].lower()
111
112     if cfg["label_col"] is None:
113         label_col = [c for c in df.columns if "sentiment" in c or "label" in c or "stars" in c][0]
114     else:
115         label_col = cfg["label_col"].lower()
116
117     df["review"] = df[text_col].apply(clean_text)
118
119     if cfg["label_map"]:
120         df["label"] = df[label_col].map(cfg["label_map"])
121     else:
122         le = LabelEncoder()
123         df["label"] = le.fit_transform(df[label_col].astype(str))
124
125     print(f"{cfg['name']} ready - {len(df):,} rows | labels: {df['label'].value_counts().to_dict()}")
126     return df[["review", "label"]]
127
128
129 # Vocabulary
130
131 class Vocabulary:
132     PAD, UNK = "<pad>", "<unk>"
133
134     def __init__(self, max_size: int = 15000, min_freq: int = 2):
135         self.max_size = max_size
136         self.min_freq = min_freq
137         self.word2id = {}

```

```

138     self.id2word = {}
139
140     def build(self, texts):
141         counter = Counter()
142         for text in texts:
143             counter.update(tokenize(text))
144         special = [self.PAD, self.UNK]
145         words = [w for w, c in counter.most_common(self.max_size) if c >= self.min_freq]
146         all_words = special + words
147         self.word2id = {w: i for i, w in enumerate(all_words)}
148         self.id2word = {i: w for w, i in self.word2id.items()}
149         print(f"Vocab size: {len(self.word2id)}")
150         return self
151
152     def encode(self, word):
153         return self.word2id.get(word, self.word2id[self.UNK])
154
155     def __len__(self):
156         return len(self.word2id)
157
158
159 # Dataset & DataLoader
160
161 class ReviewDataset(Dataset):
162     def __init__(self, texts, labels, vocab: Vocabulary,
163                 topic_vocab_size: int,
164                 max_sents: int = 10, max_sent_len: int = 50):
165         self.vocab = vocab
166         self.topic_vocab_size = topic_vocab_size
167         self.samples = []
168         for text, label in zip(texts, labels):
169             sents = split_into_sentences(text, max_sent_len, max_sents)
170             enc_sents = [[vocab.encode(w) for w in s] for s in sents]
171             bow = self._make_bow(sents)
172             self.samples.append((enc_sents, bow, int(label)))
173
174     def _make_bow(self, sents):
175         bow = np.zeros(self.topic_vocab_size, dtype=np.float32)
176         for sent in sents:
177             for w in sent:
178                 idx = self.vocab.encode(w)
179                 if idx < self.topic_vocab_size:
180                     bow[idx] += 1
181         s = bow.sum()
182         if s > 0:
183             bow /= s
184         return bow
185
186     def __len__(self):
187         return len(self.samples)
188
189     def __getitem__(self, idx):
190         return self.samples[idx]
191
192
193 def collate_fn(batch):
194     enc_sents_list, bows, labels = zip(*batch)
195     batch_size = len(batch)
196     max_sents = max(len(d) for d in enc_sents_list)
197     max_sent_len = max((len(s) for d in enc_sents_list for s in d), default=1)
198
199     doc_lengths = torch.LongTensor([len(d) for d in enc_sents_list])
200     padded_docs = torch.zeros(max_sents, batch_size, max_sent_len, dtype=torch.long)
201     sent_lengths = torch.ones(max_sents, batch_size, dtype=torch.long)
202
203     for b, doc in enumerate(enc_sents_list):
204         for s_i, sent in enumerate(doc):
205             l = len(sent)
206             padded_docs[s_i, b, :l] = torch.LongTensor(sent)
207             sent_lengths[s_i, b] = max(l, 1)
208
209     return (
210         padded_docs,
211         sent_lengths,
212         doc_lengths,
213         torch.FloatTensor(np.stack(bows)),
214         torch.LongTensor(labels),

```

```

215         )
216
217
218     # Model Architecture
219
220     class TopicVAE(nn.Module):
221     def __init__(self, vocab_size, hidden_dim, num_topics, dropout=0.2):
222         super().__init__()
223         self.num_topics = num_topics
224         self.en1 = nn.Linear(vocab_size, hidden_dim)
225         self.en2 = nn.Linear(hidden_dim, hidden_dim)
226         self.drop = nn.Dropout(dropout)
227         self.mu_fc = nn.Linear(hidden_dim, num_topics)
228         self.lv_fc = nn.Linear(hidden_dim, num_topics)
229         self.g1 = nn.Linear(num_topics, num_topics)
230         self.g2 = nn.Linear(num_topics, num_topics)
231         self.g3 = nn.Linear(num_topics, num_topics)
232         self.g4 = nn.Linear(num_topics, num_topics)
233         self.g_drop = nn.Dropout(dropout)
234         self.de = nn.Linear(num_topics, vocab_size)
235
236     def encode(self, x):
237         h = F.relu(self.en1(x))
238         h = F.relu(self.en2(h))
239         h = self.drop(h)
240         return self.mu_fc(h), self.lv_fc(h)
241
242     def reparameterize(self, mu, logvar):
243         if self.training:
244             return mu + torch.exp(0.5 * logvar) * torch.randn_like(mu)
245         return mu
246
247     def generate(self, z):
248         h = torch.tanh(self.g1(z))
249         h = torch.tanh(self.g2(h))
250         h = torch.tanh(self.g3(h))
251         return self.g_drop(self.g4(h))
252
253     def forward(self, x):
254         mu, logvar = self.encode(x)
255         z = self.reparameterize(mu, logvar)
256         h = self.generate(z)
257         return mu, logvar, F.softmax(self.de(h), dim=-1)
258
259     def get_topic_words(self, topn=10):
260         W = self.de.weight.detach().cpu().numpy()
261         return [np.argsort(W[:, k])[:-1][:topn].tolist() for k in range(self.num_topics)]
262
263
264     class HAN(nn.Module):
265     def __init__(self, vocab_size, emb_dim, num_topics,
266                 word_rnn_size, sent_rnn_size, num_classes, dropout=0.3):
267         super().__init__()
268         self.num_topics = num_topics
269         self.embedding = nn.Embedding(vocab_size, emb_dim, padding_idx=0)
270         self.word_gru = nn.GRU(emb_dim, word_rnn_size, num_layers=2,
271                                batch_first=False, bidirectional=True, dropout=dropout)
272         word_out = word_rnn_size * 2
273         self.word_att1 = nn.Linear(word_out, num_topics, bias=False)
274         self.word_att2 = nn.Linear(num_topics, 1, bias=False)
275         self.sent_gru = nn.GRU(word_out, sent_rnn_size, num_layers=1,
276                                batch_first=False, bidirectional=True, dropout=dropout)
277         sent_out = sent_rnn_size * 2
278         self.sent_att = nn.Linear(sent_out, 1, bias=False)
279         self.classifier = nn.Sequential(
280             nn.Linear(sent_out, 100),
281             nn.LeakyReLU(),
282             nn.Dropout(dropout),
283             nn.Linear(100, num_classes),
284             nn.Tanh(),
285         )
286
287     def forward(self, padded_docs, sent_lengths, doc_lengths):
288         max_sents, batch_size, _ = padded_docs.shape
289         word_att_dict = {}
290         sent_repr_list = []
291

```

```

292     for s_i in range(max_sents):
293         words = padded_docs[s_i]
294         emb = self.embedding(words).permute(1, 0, 2)
295         out, _ = self.word_gru(emb)
296         att_raw = self.word_att1(out)
297
298         word_ids = words.permute(1, 0)
299         for w_i in range(word_ids.shape[0]):
300             for b_i in range(batch_size):
301                 wid = int(word_ids[w_i, b_i].item())
302                 if wid > 0:
303                     if wid not in word_att_dict:
304                         word_att_dict[wid] = att_raw[w_i, b_i].detach()
305                     else:
306                         word_att_dict[wid] = (word_att_dict[wid] + att_raw[w_i, b_i].detach()) / 2
307
308             att_score = F.softmax(F.relu(self.word_att2(att_raw)), dim=0)
309             sent_repr_list.append((out * att_score).sum(dim=0))
310
311         sent_stack = torch.stack(sent_repr_list, dim=0)
312         sent_out, _ = self.sent_gru(sent_stack)
313         sent_att = F.softmax(F.relu(self.sent_att(sent_out)), dim=0)
314         doc_repr = (sent_out * sent_att).sum(dim=0)
315         return self.classifier(doc_repr, word_att_dict)
316
317
318 class JointModel(nn.Module):
319     def __init__(self, vocab_size, topic_vocab_size, emb_dim, hidden_dim,
320                 num_topics, word_rnn_size, sent_rnn_size, num_classes, dropout=0.3):
321         super().__init__()
322         self.topic_model = TopicVAE(topic_vocab_size, hidden_dim, num_topics, dropout)
323         self.han = HAN(vocab_size, emb_dim, num_topics,
324                       word_rnn_size, sent_rnn_size, num_classes, dropout)
325
326     def forward(self, padded_docs, sent_lengths, doc_lengths, bows):
327         mu, logvar, recon = self.topic_model(bows)
328         logit, word_att_dict = self.han(padded_docs, sent_lengths, doc_lengths)
329         return mu, logvar, recon, logit, word_att_dict
330
331
332 # Loss functions
333
334 def topic_loss(bow, mu, logvar, recon):
335     nll = -torch.sum(bow * torch.log(recon + 1e-10), dim=1)
336     kld = -0.5 * torch.sum(1 + logvar - mu.pow(2) - logvar.exp(), dim=1)
337     return (nll + kld).mean(), nll.mean(), kld.mean()
338
339 def attention_sharing_loss(word_att_dict, topic_model, kl_loss_fn):
340     if not word_att_dict:
341         return torch.tensor(0.0, requires_grad=True)
342     de_weight = topic_model.de_weight
343     total, count = torch.tensor(0.0).to(de_weight.device), 0
344     for wid, han_att in word_att_dict.items():
345         if wid >= de_weight.shape[0]:
346             continue
347         tm_att = de_weight[wid]
348         han_att = han_att.to(de_weight.device)
349         k11 = kl_loss_fn(F.log_softmax(tm_att, dim=0), F.softmax(han_att.detach(), dim=0))
350         k12 = kl_loss_fn(F.log_softmax(han_att, dim=0), F.softmax(tm_att.detach(), dim=0))
351         total = total - 2.0 / (2.0 + k11 + k12)
352         count += 1
353     return total / max(count, 1)
354
355
356 # Metrics
357
358 def classification_metrics(y_true, y_pred):
359     return {
360         "Accuracy": accuracy_score(y_true, y_pred),
361         "F1_Macro": f1_score(y_true, y_pred, average="macro", zero_division=0),
362         "Precision_Macro": precision_score(y_true, y_pred, average="macro", zero_division=0),
363         "Recall_Macro": recall_score(y_true, y_pred, average="macro", zero_division=0),
364     }
365
366 def compute_tc_npmi(topic_word_ids, corpus_token_ids, topn=10, window=10):
367     N = len(corpus_token_ids)
368     if N == 0:

```

```

369         return 0.0
370     word_freq, pair_freq = Counter(), Counter()
371     for doc in corpus_token_ids:
372         for w in set(doc):
373             word_freq[w] += 1
374         for i in range(len(doc)):
375             for j in range(i + 1, min(i + window + 1, len(doc))):
376                 wi, wj = doc[i], doc[j]
377                 if wi != wj:
378                     pair_freq[(min(wi, wj), max(wi, wj))] += 1
379     scores = []
380     for topic in topic_word_ids:
381         t_scores = []
382         for wi, wj in combinations(topic[:topn], 2):
383             p_wi = word_freq[wi] / N
384             p_wj = word_freq[wj] / N
385             p_co = pair_freq[(min(wi, wj), max(wi, wj))] / N
386             if p_wi > 0 and p_wj > 0 and p_co > 0:
387                 pmi = math.log(p_co / (p_wi * p_wj))
388                 npmi = pmi / (-math.log(p_co))
389                 t_scores.append(npmi)
390         if t_scores:
391             scores.append(np.mean(t_scores))
392     return float(np.mean(scores)) if scores else 0.0
393
394 def compute_topic_diversity(topic_word_ids, topn=10):
395     all_words = [w for topic in topic_word_ids for w in topic[:topn]]
396     return len(set(all_words)) / len(all_words) if all_words else 0.0
397
398
399 # Training & evaluation
400
401 def train_epoch(model, loader, opt, opt_att, clf_loss_fn, kl_loss_fn,
402                 clf_w, tm_w, att_interval, device):
403     model.train()
404     total_loss = 0.0
405     for step, batch in enumerate(loader):
406         padded_docs, sent_lengths, doc_lengths, bows, labels = [x.to(device) for x in batch]
407         mu, logvar, recon, logit, word_att_dict = model(padded_docs, sent_lengths, doc_lengths, bows)
408         loss_clf = clf_loss_fn(logit, labels)
409         loss_tm, _, _ = topic_loss(bows, mu, logvar, recon)
410         loss = clf_w * loss_clf + tm_w * loss_tm
411         opt.zero_grad()
412         loss.backward()
413         nn.utils.clip_grad_norm_(model.parameters(), max_norm=5.0)
414         opt.step()
415         total_loss += loss.item()
416
417         if (step + 1) % att_interval == 0:
418             mu, logvar, recon, logit, word_att_dict = model(padded_docs, sent_lengths, doc_lengths, bows)
419             att_loss = attention_sharing_loss(word_att_dict, model.topic_model, kl_loss_fn)
420             opt_att.zero_grad()
421             att_loss.backward()
422             nn.utils.clip_grad_norm_(model.parameters(), max_norm=5.0)
423             opt_att.step()
424
425     return total_loss / max(len(loader), 1)
426
427
428 @torch.no_grad()
429 def evaluate_model(model, loader, device):
430     model.eval()
431     all_preds, all_labels = [], []
432     for batch in loader:
433         padded_docs, sent_lengths, doc_lengths, bows, labels = [x.to(device) for x in batch]
434         _, _, logit, _ = model(padded_docs, sent_lengths, doc_lengths, bows)
435         all_preds.extend(logit.argmax(dim=1).cpu().numpy().tolist())
436         all_labels.extend(labels.cpu().numpy().tolist())
437     return classification_metrics(all_labels, all_preds)
438
439
440 def run_experiment(df, dataset_name, num_topics_list,
441                  vocab_max=12000, topic_vocab=5000,
442                  emb_dim=200, hidden_dim=500,
443                  word_rnn=150, sent_rnn=150,
444                  clf_w=9.0, tm_w=1.0,
445                  epochs=EPOCHS, batch_size=BATCH_SIZE,

```

```

446         att_interval=50, topn=TOPN,
447         test_size=0.2, device=DEVICE):
448     print(f"\n[DATASET: {dataset_name}] rows: {len(df)}")
449
450     df = df.sample(frac=1, random_state=SEED).reset_index(drop=True)
451     n_test = int(len(df) * test_size)
452     test_df = df.iloc[:n_test]
453     train_df = df.iloc[n_test:]
454
455     print("Building vocabulary ...")
456     vocab = Vocabulary(max_size=vocab_max, min_freq=2)
457     vocab.build(train_df["review"].tolist())
458     topic_vocab_size = min(topic_vocab, len(vocab))
459
460     print("Building corpus index for TC-NPMI ...")
461     corpus_token_ids = [
462         [vocab.encode(w) for w in tokenize(t) if vocab.encode(w) < topic_vocab_size]
463         for t in tqdm(train_df["review"].tolist(), desc="corpus")
464     ]
465     corpus_token_ids = [d for d in corpus_token_ids if d]
466
467     num_classes = df["label"].nunique()
468     results = []
469
470     for K in num_topics_list:
471         print(f"\n[{dataset_name}] K = {K} topics")
472
473         train_ds = ReviewDataset(train_df["review"].tolist(), train_df["label"].tolist(),
474                                 vocab, topic_vocab_size)
475         test_ds = ReviewDataset(test_df["review"].tolist(), test_df["label"].tolist(),
476                                vocab, topic_vocab_size)
477         train_loader = DataLoader(train_ds, batch_size=batch_size, shuffle=True,
478                                  collate_fn=collate_fn, num_workers=2, pin_memory=True)
479         test_loader = DataLoader(test_ds, batch_size=batch_size, shuffle=False,
480                                 collate_fn=collate_fn, num_workers=2, pin_memory=True)
481
482         model = JointModel(
483             vocab_size = len(vocab),
484             topic_vocab_size = topic_vocab_size,
485             emb_dim = emb_dim,
486             hidden_dim = hidden_dim,
487             num_topics = K,
488             word_rnn_size = word_rnn,
489             sent_rnn_size = sent_rnn,
490             num_classes = num_classes,
491         ).to(device)
492
493         clf_loss_fn = nn.CrossEntropyLoss()
494         kl_loss_fn = nn.KLDivLoss(reduction="batchmean")
495         opt = torch.optim.Adam(model.parameters(), lr=1e-3, weight_decay=1e-8)
496         opt_att = torch.optim.Adam(model.parameters(), lr=1e-4, weight_decay=1e-8)
497
498         best_f1, best_metrics, best_state = 0.0, {}, {}
499         for ep in range(1, epochs + 1):
500             tr_loss = train_epoch(model, train_loader, opt, opt_att,
501                                   clf_loss_fn, kl_loss_fn, clf_w, tm_w, att_interval, device)
502             ev = evaluate_model(model, test_loader, device)
503             print(f"Epoch {ep}/{epochs} | loss={tr_loss:.4f} | acc={ev['Accuracy']:.4f} | f1={ev['F1_Macro']:.4f}")
504             if ev["F1_Macro"] > best_f1:
505                 best_f1 = ev["F1_Macro"]
506                 best_metrics = ev
507                 best_state = {k: v.cpu().clone() for k, v in model.state_dict().items()}
508
509         model.load_state_dict({k: v.to(device) for k, v in best_state.items()})
510         model.eval()
511
512         topic_word_ids = model.topic_model.get_topic_words(topn=topn)
513         tc_npmi = compute_tc_npmi(topic_word_ids, corpus_token_ids, topn=topn)
514         td = compute_topic_diversity(topic_word_ids, topn=topn)
515
516         row = {
517             "Dataset": dataset_name,
518             "K": K,
519             "Accuracy": round(best_metrics.get("Accuracy", 0), 4),
520             "F1_Macro": round(best_metrics.get("F1_Macro", 0), 4),
521             "Precision_Macro": round(best_metrics.get("Precision_Macro", 0), 4),
522             "Recall_Macro": round(best_metrics.get("Recall_Macro", 0), 4),

```

```

523         "TC_NPMI":          round(tc_npmi, 4),
524         "Topic_Diversity":  round(td, 4),
525     }
526     results.append(row)
527     print(f"\n[{dataset_name}] K={K} best results:")
528     for k, v in row.items():
529         if k not in ("Dataset", "K"):
530             print(f"    {k:20s}: {v}")
531
532     return results
533
534
535 # Main
536
537 all_results = []
538
539 for ds_cfg in DATASETS:
540     df = load_dataset(ds_cfg)
541     results = run_experiment(df=df, dataset_name=ds_cfg["name"], num_topics_list=NUM_TOPICS_LIST)
542     all_results.extend(results)
543
544
545 # Results table
546
547 results_df = pd.DataFrame(all_results)
548 print("\nRESULTS SUMMARY")
549 print(results_df.to_string(index=False))
550
551 results_df.to_csv("mlt_results.csv", index=False)
552 print("\nSaved -> mlt_results.csv")
553
554
555 # Plot
556
557 metrics_to_plot = ["Accuracy", "F1_Macro", "Precision_Macro", "Recall_Macro", "TC_NPMI", "Topic_Diversity"]
558 colors = {"IMDB": "#2196F3", "Yelp": "#FF5722"}
559 markers = {"IMDB": "o", "Yelp": "s"}
560
561 fig, axes = plt.subplots(2, 3, figsize=(16, 9))
562 axes = axes.flatten()
563
564 for i, metric in enumerate(metrics_to_plot):
565     ax = axes[i]
566     for ds in results_df["Dataset"].unique():
567         sub = results_df[results_df["Dataset"] == ds]
568         ax.plot(sub["K"], sub[metric],
569               marker=markers.get(ds, "o"), color=colors.get(ds, "gray"),
570               linewidth=2, markersize=8, label=ds)
571         for _, row in sub.iterrows():
572             ax.annotate(f"{row[metric]:.3f}", (row["K"], row[metric]),
573                       textcoords="offset points", xytext=(0, 8),
574                       ha="center", fontsize=8)
575     ax.set_title(metric.replace("_", " "), fontsize=12, fontweight="bold")
576     ax.set_xlabel("Number of Topics (K)", fontsize=10)
577     ax.set_xticks(NUM_TOPICS_LIST)
578     ax.legend(fontsize=9)
579     ax.grid(alpha=0.3)
580     ax.spines[["top", "right"]].set_visible(False)
581
582 plt.suptitle("MLT JointModel - IMDB vs Yelp x K Topics", fontsize=14, fontweight="bold", y=1.01)
583 plt.tight_layout()
584 plt.savefig("mlt_results.png", dpi=150, bbox_inches="tight")
585 print("Plot saved -> mlt_results.png")

```

Listing 5: Full implementation of MLTGUI training loop.